



E902

User Manual

This manual supports the CPU version R3S0

Revision 10

April 29, 2025

Copyright © 2025 Hangzhou C-SKY MicroSystems Co., Ltd. All rights reserved.

This document is the property of Hangzhou C-SKY MicroSystems Co., Ltd. and its affiliates ("C-SKY"). This document may only be distributed to: (i) a C-SKY party having a legitimate business need for the information contained herein, or (ii) a non-C-SKY party having a legitimate business need for the information contained herein. No license, expressed or implied, under any patent, copyright or trade secret right is granted or implied by the conveyance of this document. No part of this document may be reproduced, transmitted, transcribed, stored in a retrieval system, translated into any language or computer language, in any form or by any means, electronic, mechanical, magnetic, optical, chemical, manual, or otherwise without the prior written permission of Hangzhou C-SKY MicroSystems Co., Ltd.

Trademarks and Permissions

The C-SKY Logo and all other trademarks indicated as such herein (including XuanTie) are trademarks of Hangzhou C-SKY MicroSystems Co., Ltd. All other products or service names are the property of their respective owners.

Notice

The purchased products, services and features are stipulated by the contract made between C-SKY and the customer. All or part of the products, services and features described in this document may not be within the purchase scope or the usage scope. Unless otherwise specified in the contract, all statements, information, and recommendations in this document are provided "AS IS" without warranties, guarantees or representations of any kind, either express or implied.

The information in this document is subject to change without notice. Every effort has been made in the preparation of this document to ensure accuracy of the contents, but all statements, information, and recommendations in this document do not constitute a warranty of any kind, express or implied.

杭州中天微系统有限公司 Hangzhou C-SKY MicroSystems Co., LTD

Address: Room 201, 2/F, Building 5, No.699 Wangshang Road , Hangzhou, Zhejiang, China

Website: www.xrvm.cn

Copyright © 2025 杭州中天微系统有限公司，保留所有权利。

本文档的所有权及知识产权归属于杭州中天微系统有限公司及其关联公司（下称“中天微”）。本文档仅能分派给：(i) 拥有合法雇佣关系，并需要本文档的信息的中天微员工，或 (ii) 非中天微组织但拥有合法合作关系，并且其需要本文档的信息的合作方。对于本文档，未经杭州中天微系统有限公司明示同意，则不能使用该文档。在未经中天微的书面许可的情形下，不得复制本文档的任何部分，传播、转录、储存在检索系统中或翻译成任何语言或计算机语言。

商标申明

中天微的 LOGO 和其它所有商标（如 XuanTie 玄铁）归杭州中天微系统有限公司及其关联公司所有，未经杭州中天微系统有限公司的书面同意，任何法律实体不得使用中天微的商标或者商业标识。

注意

您购买的产品、服务或特性等应受中天微商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，中天微对本文档内容不做任何明示或默示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。杭州中天微系统有限公司不对任何第三方使用本文档产生的损失承担任何法律责任。

杭州中天微系统有限公司 Hangzhou C-SKY MicroSystems Co., LTD

地址: 中国浙江省杭州市网商路 699 号 5 号楼 2 楼 201 室

网址: www.xrvm.cn

Revision History

Revision	Description	Date
1.0.0	1. First official release;	2020-06-19
	2. Revised locking description in Chapter 5 based on C20011201-C1;	
	3. Corrected the reset value of MEXSTATUS.LPMD;	
	4. Refined the description of interrupt initialization in Chapter 13.	
01	Updated document name and version number, proofread the document.	2020-07-30
02	Optimized content formatting.	2020-08-10
03	Rearranged the directory structure, Swap the positions of Chapter 4 and Chapter 5.	2020-08-12
04	Corrected descriptions of MCAUSE and MTVAL registers.	2020-08-14
05	Refined descriptions in several chapters.	2020-09-07
06	Removed description of SYNC instruction.	2020-10-12
07	Updated template.	2021-05-14
08	Modified HAD to RV Debug.	2022-10-23
09	Added descriptions related to SYSMAP.	2023-02-23
10	Added the section for Synchronization Debug and Asynchronous Debug.	2025-03-12

Document Number

Xuantie CPU technical documents follow this format:

Product Name-Product Model-Product Version Number-Document Type

Terms

Logic 1: The level value corresponding to the Boolean logic value TRUE.

Logic 0: The level value corresponding to the Boolean logic value FALSE.

Set: The action of setting one or more bits to the level value corresponding to logic 1.

Clear: The action of setting one or more bits to the level value corresponding to logic 0.

Reserved bit: A bit reserved for feature extension. The value of a reserved bit is 0 unless otherwise specified.

Signal: An electrical value used to transfer information based on its state or state transition.

Pin: An external electrical and physical connection. One pin can connect to multiple signals.

Enable: The action of switching a discrete signal to a valid state:

- * Switch a valid low-level signal from a high level to a low level;

- * Switch a valid high-level signal from a low level to a high level.

Disable: The action of switching the state of an enabled signal:

- * Switch a valid low-level signal from a low level to a high level.

- * Switch a valid high-level signal from a high level to a low level.

LSB: the least significant bit.

MSB: the most significant bit.

Signal, bit field, control bit are represented by a general rule

Identifier followed by a value range indicates a group of signals from the most significant bit to the least significant bit.

- * `addr[4:0]` indicates a group of address buses;

- * `addr[4]` indicates the most significant bit, and `addr[0]` indicates the least significant bit.

Single identifier Indicates a single signal. For example, `pad_cpu_rst_b` indicates a single signal.

In some cases, an identifier followed by a number is used to express a specific meaning. For example, `addr15` indicates the 16th bit of a group of buses.

RAW indicates a read-after-write operation and WAW indicates a write-after-write operation.

Symbol

Symbol	Function
+	Addition
−	Subtraction
*	Multiplication
/	Division
>	Greater than
<	Less than
=	Equal to
≥	Greater than or equal to
≤	Less than or equal to
≠	Not equal
&	And
	Or
⊕	Exclusive-OR
NOT	Negation
:	Connection
⇒	Transmission
↔	Exchange
±	Error
0b0011	Binary number
0x0F	Hexadecimal number
rd	Integer Destination Register
rs1	Integer Source Register 1
rs2	Integer Source Register 2
rs3	Integer Source Register 3
fd	Floating Point Destination Register
fs1	Floating Point Source Register 1
fs2	Floating Point Source Register 2
fs3	Floating Point Source Register 3
vd	Vector Destination Register
vs1	Vector Source Register 1
vs2	Vector Source Register 2
vs3	Vector Source Register 3

1	Overview	1
1.1	Scope of Application	1
1.2	Introduction	1
1.3	Features	1
1.4	Configurable Options	2
1.5	Design For Debug (DFD)	3
1.6	Release Notes	3
2	E902 Overview	4
2.1	Structure	4
2.2	TCIP Architecture	5
2.3	Interface Overview	5
3	Programming Model	6
3.1	Operating Mode and Register View	6
3.2	General Purpose Registers	7
3.3	M-mode Control and Status Registers	8
3.4	Debug/Trace Registers (Shared with Debug Mode)	10
3.5	Debug Mode Registers	10
3.6	Exception Handling	10
3.7	Data Format	10
3.7.1	Integer Data Format	11
3.7.2	Little-Endian	11
4	Exceptions and Interrupts	12
4.1	Exceptions	13
4.1.1	Exception Responding	14
4.1.2	Exception Handling	15

4.1.3	Return from Exceptions	15
4.1.4	Lock	16
4.2	Interrupts	17
4.2.1	Vector Interrupts	17
4.2.1.1	Interrupt Priorities	17
4.2.1.2	Interrupt Response	17
4.2.1.3	Interrupt Handling	18
4.2.1.4	Return from Interrupts	18
4.2.2	Non-Vector Interrupts	19
4.2.2.1	Interrupt Priorities	19
4.2.2.2	Interrupt Response	19
4.2.2.3	Return from Interrupts	19
4.2.2.4	Interrupt Tail Chaining	19
4.3	NMI	20
4.3.1	NMI Response	20
4.3.2	Return from NMI	21
4.3.3	Lock	21
5	Instruction Set	22
5.1	RV32IMAFDC Instruction	22
5.1.1	RV32E Integer Instruction Set	22
5.1.2	RV32M Multiply and Divide Instruction Set	24
5.1.3	RVC Compressed Instruction Set	25
5.2	Xuantie Extended Instruction Set	26
5.2.1	Cache Instruction Set	26
6	Physical Memory Protection	28
6.1	PMP Overview	28
6.2	PMP CSR	28
6.2.1	Physical Memory Protection Configuration Register (PMPCFG0~PMPCFG3)	29
6.2.2	Physical Memory Protection Address Register (PMPADDR0~PMPADDR15)	30
6.3	Memory Access Handling	32
7	Memory Subsystem	34
7.1	Cache Operation	34
7.1.1	Extended Cache Register	34
7.1.2	Extended Cache Instructions	35
7.2	Memory Model	35
7.2.1	Memory Attribute	35
7.2.2	Configuration Reference of SYSMAP	36
8	Bus Matrix and Bus Interfaces	38
8.1	Introduction	38

8.2	System Bus Interfaces	40
8.3	Instruction Bus Interfaces	40
9	Core Local Interrupt	41
9.1	Register Address Mapping	41
9.2	Software Interrupts	42
9.3	Timer Interrupts	43
10	Core-Local Interrupt Controller	45
10.1	Interrupt Handling Mechanism	45
10.1.1	Interrupt Arbitration	45
10.1.2	Interrupt Requests and Responses	46
10.2	Address Mapping in CLIC	47
10.3	CLIC Registers	47
10.3.1	CLIC Configuration Register (CLICCFG)	47
10.3.2	CLIC Information Register (CLICINFO)	48
10.3.3	Interrupt Threshold Register (MINTTHRESH)	49
10.3.4	Interrupt Pending Register (CLICINTIP)	49
10.3.5	Interrupt Enable Register (CLICINTIE)	50
10.3.6	Interrupt Attribute Register (CLICINTATTR)	50
10.3.7	Interrupt CSR Register (CLICINTCTL)	51
11	Debug Interface	53
11.1	Overview	53
11.2	DM Register	54
11.3	Resource Configuration	56
11.4	Synchronization Debug and Asynchronous Debug	56
11.4.1	Synchronization Debug	56
11.4.2	Asynchronous Debug	56
12	Power Consumption Management	58
12.1	Low-Power Mode	58
12.2	Low-Power Wake-Up	58
13	Program Examples	60
13.1	PMP Setting Example	60
13.2	Cache Setting Example	63
13.3	Example of Interrupt Enable Initialization Setting	63
13.4	GPR Initialization Example	64
13.5	Initialization Example of Stack Pointer	65
13.6	Example of Setting the Entry Addresses of Exception and ISR	65
14	Appendix A Standard Instructions	69
14.1	Appendix A-1 E Instructions	69

14.1.1	ADD—The Signed Add Instruction	69
14.1.2	ADDI—The Signed Immediate Add Instruction	70
14.1.3	AND—The Bitwise AND Instruction	70
14.1.4	ANDI—The Immediate Bitwise AND Instruction	71
14.1.5	AUIPC—The Add Upper Immediate to PC Instruction	71
14.1.6	BEQ—The Branch-If-Equal Instruction	72
14.1.7	BGE—The Signed Branch-If-Greater-than-or-Equal Instruction	72
14.1.8	BGEU—The Unsigned Branch-If-Greater-than-or-Equal Instruction	73
14.1.9	BLT—The Signed Branch-If-Less-than Instruction	74
14.1.10	BLTU—The Unsigned Branch-if-Less-than Instruction	74
14.1.11	BNE—The Branch-if-not-Equal Instruction	75
14.1.12	CSRRC—The CSR Read/Clear Instruction	76
14.1.13	CSRRCI—The CSR Read/Clear Immediate Instruction	76
14.1.14	CSRRS—the CSR Read/Set Instruction	77
14.1.15	CSRRSI—The CSR Read/Set Immediate Instruction	78
14.1.16	CSR RW—The CSR Read/Write Instruction	78
14.1.17	CSR RWI—The CSR Read/Write Immediate Instruction	79
14.1.18	EBREAK—The Breakpoint Instruction	79
14.1.19	ECALL—The Environment Call Instruction	80
14.1.20	FENCE—The Memory Synchronization Instruction	80
14.1.21	FENCE.I—The Instruction Stream Synchronization Instruction	81
14.1.22	JAL—The Instruction for Directly Jumping to a Subroutine	81
14.1.23	JALR—The Jump and Link Register Instruction	82
14.1.24	LB—The Signed Extended Byte Load Instruction	82
14.1.25	LBU—The unsigned Extended Byte Load Instruction	83
14.1.26	LH—The Signed Extended Halfword Load Instruction	83
14.1.27	LHU—The Unsigned Extended Halfword Load Instruction	84
14.1.28	LUI—The Upper Immediate Load Instruction	84
14.1.29	LW—The Word Load Instruction	85
14.1.30	MRET—The Instruction to Return from Machine Exception	85
14.1.31	OR—Bitwise OR Instruction	86
14.1.32	ORI—The Immediate Bitwise OR Instruction	86
14.1.33	SB—The Byte Store Instruction	87
14.1.34	SH—The Halfword Store Instruction	87
14.1.35	SLL—The Logical Left Shift instruction	88
14.1.36	SLLI—The Immediate Logical Left Shift Instruction	88
14.1.37	SLT—The Signed Set-If-Less-than Instruction	88
14.1.38	SLTI—The Signed Set-If-less-than-Immediate Instruction	89
14.1.39	SLTIU—The Unsigned Set-If-less-than-Immediate Instruction	90
14.1.40	SLTU—The Unsigned Set-If-less-than Instruction	90
14.1.41	SRA—The Arithmetic Right Shift Instruction	91
14.1.42	SRAI—The Immediate Arithmetic Right Shift Instruction	91

14.1.43	SRL—The Logical Right Shift Instruction	92
14.1.44	SRLI—The Immediate Logical Right Shift Instruction	92
14.1.45	SUB—The Signed Subtract Instruction	92
14.1.46	SW—The Word Store Instruction	93
14.1.47	WFI—The Instruction for Entering the Low Power Mode	93
14.1.48	XOR—The Bitwise XOR Instruction	94
14.1.49	XORI—The Immediate Bitwise XOR Instruction	94
14.2	Appendix A-2 M Instructions	95
14.2.1	DIV—The Signed Divide Instruction	95
14.2.2	DIVU—The Unsigned Divide Instruction	95
14.2.3	MUL—The Signed Multiply Instruction	96
14.2.4	MULH—The Signed Multiply Upper Bit Extraction Instruction	96
14.2.5	MULHSU—The Signed and Unsigned Multiply Upper Bit Extraction Instruction	97
14.2.6	MULHU—The Unsigned Multiply Upper Bit Extraction Instruction	97
14.2.7	REM—The Signed Remainder Instruction	98
14.2.8	REMU—The Unsigned Remainder Instruction	98
14.3	Appendix A-3 C Instructions	99
14.3.1	C.ADD—The Signed Add Instruction	99
14.3.2	C.ADDI—The Signed Immediate Add Instruction	100
14.3.3	C.ADDI4SPN—The Instruction to Add Immediate Scaled by 4 to Stack Pointer	100
14.3.4	C.ADDI16SP—The Instruction to Add Immediate Scaled by 16 to Stack Pointer	101
14.3.5	C.AND—The Bitwise AND Instruction	101
14.3.6	C.ANDI—The Immediate Bitwise AND Instruction	102
14.3.7	C.BEQZ—The Branch-If-Equal-to-Zero Instruction	103
14.3.8	C.BNEZ—The Branch-If-not-Equal-to-Zero Instruction	104
14.3.9	C.EBREAK—The Breakpoint Instruction	105
14.3.10	C.J—The Unconditional Jump Instruction	105
14.3.11	C.JAL—The Instruction for Jumping to a Subroutine Unconditionally.	106
14.3.12	C.JALR—The Jump And Link Register Instruction	106
14.3.13	C.JR—The Jump to Register Instruction	107
14.3.14	C.LI—The Immediate Transfer Instruction	107
14.3.15	C.LUI—The Upper Bit Immediate Transfer Instruction	108
14.3.16	C.LW—The Word Load Instruction	109
14.3.17	C.LWSP—The Load Word from Stack Pointer Instruction	109
14.3.18	C.MV—The Data Transfer Instruction	110
14.3.19	C.NOP—The No-Operation Instruction	110
14.3.20	C.OR—The Bitwise OR Instruction	111
14.3.21	C.SLLI—The Immediate Logical Left Shift Instruction	112
14.3.22	C.SRAI—The Immediate Arithmetic Right Shift Instruction	112
14.3.23	C.SRLI—The Immediate Logical Right Shift Instruction	113
14.3.24	C.SW—The Word Store Instruction	114
14.3.25	C.SWSP—The Word Stack Store Instruction	115

14.3.26	C.SUB—The Signed Subtract Instruction	115
14.3.27	C.XOR—The Bitwise XOR Instruction	116
14.4	Appendix A-4 Pseudo Instructions	117
15	Appendix B Xuantie Extended Instructions	119
15.1	Appendix B-1 Cache Instructions	119
15.1.1	ICACHE.IALL—The Instruction to Invalidate All Table Entries in the I-Cache . .	119
15.1.2	ICACHE.IPA—The Instruction that Invalidates Page Table Entries in the I-Cache that Match the Specified Physical Address	120
16	Appendix C Machine Mode Control and Status Registers	121
16.1	M-mode Information Register Group	121
16.1.1	M-mode Vendor ID Register (MVENDORID)	121
16.1.2	M-mode Architecture ID register (MARCHID)	121
16.1.3	M-mode Hardware Implementation ID Register(MIMPID)	122
16.1.4	M-mode Hart ID Register (MHARTID)	122
16.2	M-mode Exception Configuration Register Group	122
16.2.1	M-Mode Status Register (MSTATUS)	122
16.2.2	M-mode Instruction Set Architecture Register (MISA)	123
16.2.3	M-mode Interrupt Enable Register (MIE)	123
16.2.4	M-Mode Exception Vector Base Address Register (MTVEC)	124
16.2.5	M-Mode Counter Enable Register (MCOUNTEREN)	125
16.2.6	M-mode Vector Interrupt Base Address Register (MTVT)	126
16.3	M-Mode Exception Handling Register Group	126
16.3.1	M-Mode Scratch Register (MSCRATCH)	126
16.3.2	M-Mode Multi-Mode Data Backup Register (MSCRATCHCSW)	126
16.3.3	M-Mode Interrupt Data Backup Register (MSCRATCHCSWL)	127
16.3.4	M-mode Interrupt Controller Base Address Register (MCLICBASE)	127
16.3.5	M-Mode Exception Program Counter (MEPC)	127
16.3.6	M-Mode Exception Cause Register (MCAUSE)	128
16.3.7	M-Mode Pending Interrupt Vector Address and Interrupt Enable Register (MNXTI) .	129
16.3.8	M-Mode Interrupt Status Register (MINTSTATUS)	130
16.3.9	M-Mode Exception Cause Register (MTVAL)	130
16.3.10	M-Mode Interrupt Pending Register (MIP)	131
16.4	M-Mode PMP Register Group	131
16.5	M-mode Exception Handling Register Group	131
16.5.1	M-mode Cycle Counter (MCYCLE)	131
16.5.2	M-mode Retired Instruction Counter (MINSTRET)	132
16.6	M-mode Extended Register Group	132
16.6.1	Extended Status Register (MXSTATUS)	132
16.6.2	M-mode Hardware Configuration Register (MHCR)	133
16.6.3	Reset Start Address Register (MRADDR)	133

16.6.4	Extended Exception Status Register (MEXSTATUS)	133
16.6.5	NMI Status Register (MNMICAUSE)	135
16.6.6	NMI Exception Program Counter (MNMIPC)	135
16.6.7	CPU Identifier Register	136
16.7	Debug/Trace Register Group (Shared with Debug Mode)	136
16.7.1	Debug/Trace Trigger Selection Register (TSELECT)	136
16.7.2	Debug/Trace Trigger Data Register 1 (TDATA1)	136
16.7.3	Debug/Trace Trigger Data Register 2 (TDATA2)	137
16.7.4	Debug/Trace Trigger Data Register 3 (TDATA3)	137
16.7.5	Debug/Trace Trigger Information Register (TINFO)	138
16.7.6	Debug/Trace Trigger CSR (TCONTROL)	138
16.7.7	M-mode Content Register (MCONTEXT)	139
16.8	Debug Mode Register Group	139
16.8.1	Debug CSR Register (DCSR)	139
16.8.2	Debug Program Counter (DPC)	141
16.8.3	Debug Mode Temporary Data Backup Register 0 (DSCRATCH0)	141
16.8.4	Debug Mode Temporary Data Backup Register 1 (DSCRATCH1)	141
16.9	Debug Extended Register Group	141
16.9.1	Xuantie Debug Cause Register (MHALTCAUSE)	142
16.9.2	Xunatie Debug Information Register (MDBGINFO)	142
16.9.3	XuanTie Branch Target Address Record Register (MPCFIFO)	142

1.1 Scope of Application

This document is designed for XuanTie E902, developed by Hangzhou C-SKY MicroSystems Co., Ltd.

1.2 Introduction

XuanTie E902 is an embedded CPU core with ultra-low power consumption and costs developed by Hangzhou C-SKY MicroSystems Co., Ltd. It has the running efficiency and performance of a 32-bit embedded CPU with the cost of 8-bit CPU. E902 is compatible with the RISC-V instruction architecture and utilizes the 16-bit/32-bit mixed encoding system. Its instruction system and pipeline hardware structure are simple and efficient for operation, and it features ultra-low costs and power consumption and high code density. E902 is oriented to smart cards, smart power grids, low-cost microcontrollers, wireless sensor networks, and other embedded applications.

1.3 Features

The key architectural features of E902:

- RISC-V RV32E[M]C instruction sets are supported;
- Sixteen 32-bit General Purpose Registers (GPRs);
- Two-stage in-order pipeline;

- RISC-V machine mode (M-mode) and user mode (U-mode) are supported;
- The RISC-V debug architecture, standard five-wire JTAG debugging interface and the two-wire CJTAG debugging interface are supported;
- Single-cycle hardware multiplier and multi-cycle hardware divider;
- Compatible with the RISC-V core local interrupt controller (CLIC) interrupt standard, and nested interrupts and up to 240 external interrupt sources are supported;
- Compatible with the RISC-V physical memory protection (PMP) standard, and 0-/4-/8-/12-/16-zone configuration are supported;
- AHB-Lite bus protocol, the instruction bus, and the system bus are supported;
- Configurable instruction cache (I-Cache) with 16-byte cache lines and 2 KiB/4 KiB/8 KiB capacity;
- Extended Xuantie programming models are supported;
- Configurable reset start address during hardware integration;
- Software reset operations are supported.

1.4 Configurable Options

Table 1.1 describes the configurable options of E902.

Table 1.1: E902 configurable options

Configurable Unit	Configurable Option	Description
I-Cache	None/2 KiB/4 KiB/8 KiB	The I-Cache may not be configured or configured with the capacity of 2 KiB, 4 KiB, or 8 KiB.
Hardware multiplier	No/Yes	If the hardware multiplier is configured, the single-cycle fast multiplier or multi-cycle (3-34) slow multiplier can also be configured.
PMP	0/4/8/12/16	0, 4, 8, 12, or 16 entries can be configured. 0 indicates that PMP is not implemented.
CLIC	Interrupt sources: 1-240 Number of valid bits for the interrupt priority: 2-5	1 to 240 interrupt sources can be configured, and the number of valid bits for the interrupt priority is 2 to 5, which corresponds to 4 to 32 priorities.
JTAG interface	2-wire/5-wire	The JTAG interface can be configured to operate in either 2-wire mode (CJTAG) or 5-wire mode (JTAG5).

Continued on next page

Table 1.1 – continued from previous page

Debug trigger re-source	Minimum Configuration/ Typical Configuration/ Maximum Configuration	The E902 incorporates a debug module compliant with the RISC-V Debug Standard, offering hardware configurability for Minimum, Typical, and Maximum setups. For detailed information, please refer to <i>Debug Interface</i> .
-------------------------	---	--

1.5 Design For Debug (DFD)

E902 supports the standard five-wire JTAG debugging interface and the two-wire CJTAG debugging interface. E902 supports all common debug features, including software breakpoints, memory breakpoints, register check and modification, storage check and modification, single-step and multi-step instruction tracking, program stream tracking and so on. For more information, see *Debug Interface*.

1.6 Release Notes

E902 is compatible with the following RISC-V standard versions:

- *The RISC-V Instruction Set Manual, Volume I: RISC-V User-Level ISA, Version 2.2*
- *The RISC-V Instruction Set Manual, Volume II: RISC-V Privileged Architecture, Version 1.10*
- *RISC-V Core-Local Interrupt Controller (CLIC) Version 0.8*
- *RISC-V External Debug Support Version 0.13.2*

2.1 Structure

Fig. 2.1 shows the E902 structure.

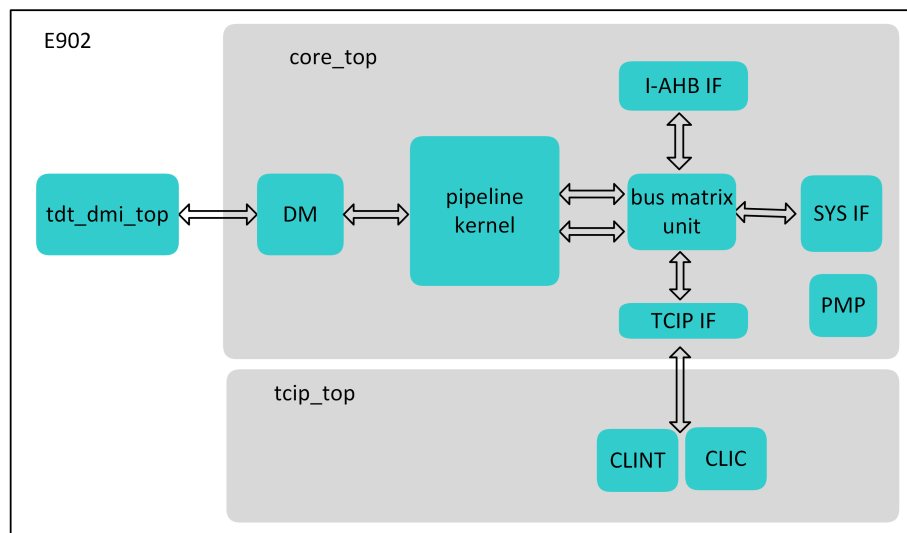


Fig. 2.1: E902 Structure

E902 utilizes a 2-stage pipeline structure involving instruction fetching and execution. In the instruction fetching stage, E902 obtains instructions from the memory. In the instruction execution stage, E902 decodes and executes instructions and writes back execution results.

The configurable physical memory protection (PMP) unit checks permissions of physical addresses to protect the memory. Permissions include unreadable/unwritable, read-only, readable/writable, executable, and non-executable.

Xuantie Debug Trace supports various debug methods, including software breakpoints, memory breakpoints, and single-step and multi-step instruction tracking, to debug the CPU, General Purpose Registers (GPRs), Control and Status Registers (CSRs), and memory online.

E902 is designed with the on-chip Tightly Coupled IP (TCIP) interface and the interface for multiple AHB-Lite buses. The on-chip TCIP interface integrates with the Processor Interrupt Controller and supports nested interrupts and Core Local Interrupt (CLINT). Up to 240 external interrupt sources can be configured, and 4/8/16/32 interrupt priorities can be set.

2.2 TCIP Architecture

E902 applies internal buses to integrate TCIPs, including the CLINT and vector interrupt controllers that are compatible with the RISC-V standard, to improve system integration of E902 and facilitate integration and development for users.

Vector interrupt controllers feature the following aspects.

- Support for 1 to 240 external interrupt sources,
- Support for nested interrupts,
- Support for level and pulse interrupts,
- Configurable 4/8/16/32 interrupt priorities.

2.3 Interface Overview

For E902 interface signal details, please refer to *XuanTie E902 Integration Manual*.

3.1 Operating Mode and Register View

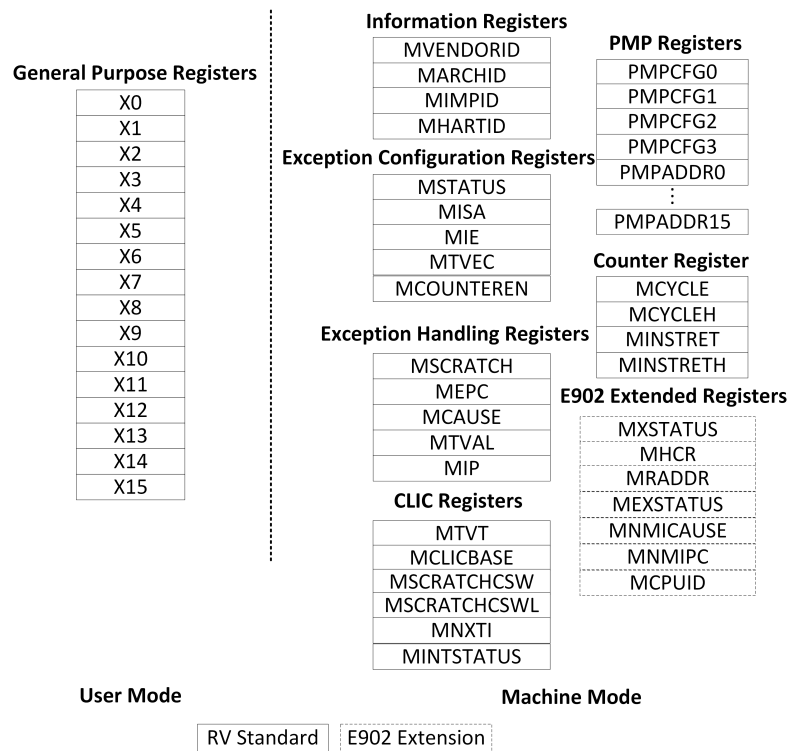


Fig. 3.1: Programming Model

E902 implements two RISC-V programming models: machine mode (M-mode) and user mode (U-mode). When the PM bit of the MXSTATUS register is set to 2'b11, E902 runs programs in M-mode. After the reset, it operates in M-mode.

The two programming models correspond to different operation privileges and differ in the following aspects:

1. Access to CSRs;
2. Application of privileged instructions;
3. Access to CSRs of Tightly Coupled IP (TCIP).

In U-mode, E902 can access only General Purpose Registers (GPRs). In M-mode, E902 can access all GPRs, CSRs, and TCIP CSRs.

In U-mode, E902 can use most privileged instructions except those have great impact on the system, such as WFI, MRET, CSR, and Xuantie extended CACHE instructions. In M-mode, E902 can apply all supported instructions. In U-mode, E902 can execute the ECALL instruction to switch to the M-mode.

3.2 General Purpose Registers

Table 3.1 lists sixteen 32-bit GPRs (X0 to X15) in U-mode, in which X0 is a zero register.

Table 3.1: General Purpose Registers

Register	ABI Name	Description
x0	zero	A zero register.
x1	ra	A return address register.
x2	sp	A stack pointer register.
3	gp	A global pointer register.
4	tp	A thread pointer register.
5	t0	A temporary/standby link register.
6-7	t1-2	Temporary registers.
8	s0/fp	A reserved register/frame pointer register.
9	s1	A reserved register.
10-11	a0-a1	Function argument/return value registers.
12-15	a2-a5	Function argument registers.

GPRs store instruction operands, instruction execution results, and address information in general. Hardware and software conventions designate these GPRs for purposes such as subroutine linkage, parameter passing, and stack pointer functionality.

Furthermore, in U-mode, the register group includes the Program Counter (PC), which holds the execution address for the processor.

3.3 M-mode Control and Status Registers

Table 3.2 describes the Control and Status Registers (CSRs) that are defined by the RISC-V standard and implemented in E902.

Table 3.2: M-mode CSRs List

Register	Read/Write Permission	ID	Description
M-mode Information Registers			
MVENDORID	Read-only in M-mode	0xF11	A vendor ID register.
MARCHID	Read-only in M-mode	0xF12	An architecture ID register.
MIMPID	Read-only in M-mode	0xF13	An M-mode hardware implementation ID register.
MHARTID	Read-only in M-mode	0xF14	A hart ID register.
M-mode Exception Configuration Registers			
MSTATUS	Read/Write in M-mode	0x300	An M-mode CPU status register.
MISA	Read/Write in M-mode	0x301	An M-mode CPU instruction set information register.
MIE	Read/Write in M-mode	0x304	An M-mode interrupt enable CSR.
MTVEC	Read/Write in M-mode	0x305	An M-mode exception vector base address register.
MCOUNTEREN	Read/Write in M-mode	0x306	An M-mode counter enable register.
MTVT	Read/Write in M-mode	0x307	An M-mode vector interrupt base address register.
M-mode Exception Handling Registers			
MSCRATCH	Read/Write in M-mode	0x340	An M-mode data backup register.
MEPC	Read/Write in M-mode	0x341	An M-mode exception program counter.
MCAUSE	Read/Write in M-mode	0x342	An M-mode exception cause register.
MTVAL	Read/Write in M-mode	0x343	An M-mode exception vector register.
MIP	Read/Write in M-mode	0x344	An M-mode interrupt pending register.
MNXTI	Read/Write in M-mode	0x345	An M-mode pending interrupt vector address and interrupt enable register.
MINTSTATUS	Read-only in M-mode	0x346	An M-mode interrupt status register.
MSCRATCHCSW	Read/Write in M-mode	0x348	An M-mode multi-mode data backup register.
MSCRATCHCSWL	Read/Write in M-mode	0x349	An M-mode interrupt data backup register.

Continued on next page

Table 3.2 – continued from previous page

MCLICBASE	Read-only in M-mode	0x350	An M-mode interrupt controller base address register.
M-mode Physical Memory Protection (PMP) Registers			
PMPCFG0	Read/Write in M-mode	0x3A0	PMP configuration register 0.
PMPCFG1	Read/Write in M-mode	0x3A1	PMP configuration register 1.
PMPCFG2	Read/Write in M-mode	0x3A2	PMP configuration register 2.
PMPCFG3	Read/Write in M-mode	0x3A3	PMP configuration register 3.
PMPADDR0	Read/Write in M-mode	0x3B0	PMP base address register 0.
... ..			
PMPADDR15	Read/Write in M-mode	0x3BF	PMP base address register 15.
M-mode Counters/Timers			
MCYCLE	Read/Write in M-mode	0xB00	An M-mode cycle counter.
MINSTRET	Read/Write in M-mode	0xB02	An M-mode retired instruction counter.
MCYCLEH	Read/Write in M-mode	0xB80	The upper 32 bits of an M-mode cycle counter.
MINSTRETH	Read/Write in M-mode	0xB82	The upper 32 bits of an M-mode retired instruction counter.

Table 3.3 describes extended CSRs of E902.

Table 3.3: Extended M-mode CSRs of E902

Register	Read/Write Permission	ID	Description
Extended M-mode Registers			
MXSTATUS	Read/Write in M-mode	0x7C0	An extended status register.
MHCR	Read/Write in M-mode	0x7C1	A hardware CSR.
MRADDR	Read-only in M-mode	0x7E0	A CPU reset start address register.
MEXSTATUS	Read/Write in M-mode	0x7E1	An extended exception status register.
MNMICAUSE	Read/Write in M-mode	0x7E2	A Non-Maskable Interrupt (NMI) context save register.
MNMIPC	Read/Write in M-mode	0x7E3	A NMI exception program counter.
Extended M-mode CPU Model Registers			
MCPUID	Read-only in M-mode	0xFC0	A CPU model register.

For specific register definitions and functions, please refer to *Appendix C Machine Mode Control and Status Registers*.

3.4 Debug/Trace Registers (Shared with Debug Mode)

Table 3.4: Debug/Trace Registers (Shared with Debug Mode)

Register	Read/Write Permission	ID	Description
tselect	Read/Write in M-mode	0x7A0	Debug/trace trigger select register.
tdata1	Read/Write in M-mode	0x7A1	Debug/trace trigger data register 1.
tdata2	Read/Write in M-mode	0x7A2	Debug/trace trigger data register 2.
tdata3	Read/Write in M-mode	0x7A3	Debug/trace trigger data register 3.
tinfo	Read-only in M-mode	0x7A4	Debug/trace trigger information register.
tcontrol	Read/Write in M-mode	0x7A5	Debug/trace trigger CSR.
mcontext	Read/Write in M-mode	0x7A8	M-mode content register.

3.5 Debug Mode Registers

Table 3.5: Debug Mode Registers

Register	Read/Write Permission	ID	Description
Debug Mode Registers			
dcsr	Read/Write in debug mode	0x7B0	Debug mode control and status register.
dpc	Read/Write in debug mode	0x7B1	Debug mode program Counter.
dscratch0	Read/Write in debug mode	0x7B2	Debug mode temporary data backup register 0
dscratch1	Read/Write in debug mode	0x7B3	Debug mode temporary data backup register 1
Extended Debug Registers			
mhaltcause	Read/Write in M-mode	0xFE0	Xuantie debug cause Register
mdbginfo	Read/Write in M-mode	0xFE1	Xuantie debug information register
mpcfifo	Read/Write in M-mode	0xFE2	Xuantie branch target address record register

3.6 Exception Handling

Exception handling is a core feature of a CPU. Exceptions include instruction exceptions and external interrupts. When exception events occur, exception handling enables the CPU to respond to and handle these events.

For specific information about the exception and interrupt handling processes, please refer to *Exceptions and Interrupts*.

3.7 Data Format

3.7.1 Integer Data Format

Values in a register are classified into the signed and unsigned types. And the format is consistently arranged from right to left, representing the least significant bit (LSB) to the most significant bit (MSB), as shown in Fig. 3.2 .

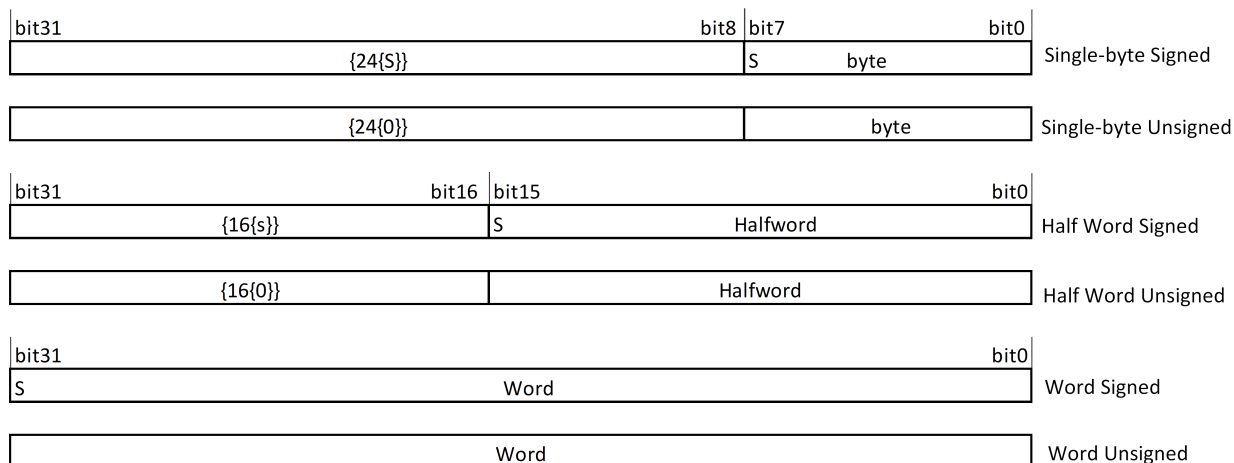


Fig. 3.2: Integer Data Structure in Registers

3.7.2 Little-Endian

Data in storage devices is classified into the big-endian and little-endian types. E902 only supports the little-endian scheme, which means upper bits of data are stored to the upper bits in the physical memory, as shown in Fig. 3.3 .

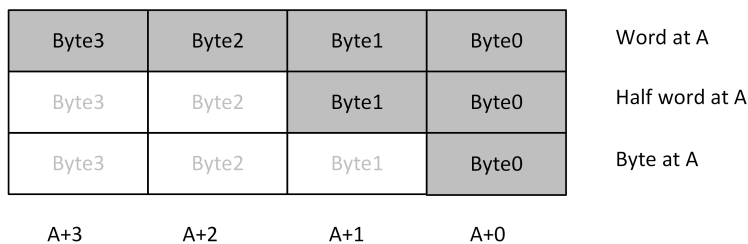


Fig. 3.3: Data Structure in Memory

Exceptions and Interrupts

Exception handling is a significant feature of a CPU, which includes handling of instruction exceptions and external interrupts. When some exception events occur, exception handling enables the CPU to respond to and handle these events.

When an exception occurs, the CPU switches from the normal program to the specific exception handling routine based on the internal or external exception event. External events that cause exceptions include interrupt requests and read/write access errors from external devices. Internal events that cause exceptions include invalid instructions and unaligned access errors. Exceptions may also occur when the ECALL and EBREAK instructions are executed properly. During exception handling, the CPU jumps to the exception service routine entry by the exception vector table.

The key to exception handling is to save the current operating status of the CPU when an exception occurs and restore the status when the CPU exits exception handling. Exceptions can be identified at all stages of the instruction pipeline. And the exception handling ensures that subsequent instructions do not change CPU status. Exceptions are handled at the boundary of an instruction. To be specific, the CPU responds to the exceptions when the instruction retires and saves the address of the to-be-executed instruction when it exits exception handling. Even if exceptions are identified before an instruction retires, the CPU handles the exceptions until the instruction retires. E902 determines which instruction's address to store in the exception address register, based on whether the instruction identifying the exception has finished executing or not. For example, if the exception event is an external interrupt service request, the interrupted instruction retires normally and changes the CPU status. The address of the next instruction is saved to the Machine Exception Program Counter (MEPC) as the instruction entry when the CPU returns from the interrupt service routine. The address of the next instruction is $PC + 2$ or $PC + 4$ based on whether the current instruction is 16-bit or 32-bit. If the exception event is caused by an access error

instruction, the instruction cannot be completed, retires abnormally, and does not change the CPU status (that is, the register value is not changed). The access error address instruction (PC) is saved to the MEPC register. When the CPU returns from the exception service routine, it continues to execute this access error instruction.

E902 is compatible with the exception vector IDs defined in RISC-V, as described in [Table 4.1](#) :

Table 4.1: Exception Vector ID

Interrupt Flag	Vector ID	Exception/Interrupt Type
1	0-2	Not implemented or reserved
1	3	M-mode software interrupt
1	4-6	Not implemented or reserved
1	7	M-mode timer interrupt
1	8-10	Not implemented or reserved
1	11	M-mode external interrupt
1	≥ 12	Reserved
1	16	CLIC external interrupt 0 (pad_clic_int_vld[0])
1	...	...
1	$16+i$	CLIC external interrupt i (pad_clic_int_vld[i])
1	...	...
1	255	CLIC external interrupt 239 (pad_clic_int_vld[239])
0	0	Not implemented
0	1	Fetch instruction access error exception
0	2	Illegal instruction exception
0	3	Debug breakpoint exception
0	4	Load instruction unaligned access exception
0	5	Load instruction access error exception
0	6	Store instruction unaligned access exception
0	7	Store instruction access error exception
0	8	U-mode environment call exception
0	9	Not implemented
0	10	Reserved
0	11	M-mode environment call exception
0	12~23	Not implemented or reserved
0	24	Non-Maskable Interrupt (NMI)

4.1 Exceptions

4.1.1 Exception Responding

There is no exception enable registers in RISC-V programming models. Therefore, exceptions can be responded to immediately once triggered. According to the RISC-V standard, the priority of interrupts is higher than that of exceptions. Table 4.2 describes the definition of internal priority for exceptions. The vector ID of NMI exceptions is defined as 24 in E902. NMI exceptions are given the highest priority among all interrupts and exceptions.

Table 4.2: Definition of Internal Priority for Exceptions

Priority	Vector ID	Type
Highest	3	Debug breakpoint exception
↓	1	Fetch instruction access error exception
↓	2	Illegal instruction exception
↓	8	U-mode environment call exception
↓	11	M-mode environment call exception
↓	6	Store instruction unaligned access exception
↓	4	Load instruction unaligned access exception
↓	7	Store instruction access error exception
↓	5	Load instruction access error exception

Exception responding will interrupt normal program execution of the CPU, which then handles exceptions. During exception responding, the CPU context need to be stored. The CPU will restore and execute the normal program flow prior to the exception response when it exits the exception service routine. During exception responding, store the CPU context in the following steps.

The exception is handled in the following steps, and all the steps are completed in one clock cycle of the CPU.

1. CPU saves the PC to the MEPC.
2. CPU updates the Exception Code field in the MCAUSE register to the vector ID of the current exception and marks the exception type. For more information about the vector IDs, please refer to Table 4.1 .
3. CPU saves the interrupt enable bit MIE of the MSTATUS register to MPIE.
4. CPU sets the MIE bit in the MSTATUS register to 0 to disable responding to interrupts.
5. CPU saves the PM field in the MXSTATUS register to the MPP field in the MSTATUS register and sets the PM field to the M-mode, which indicates the CPU enters M-mode after the exception responding.
6. CPU updates the cause of the exception event to the MTVAL register, as described in Table 4.3 .
7. CPU obtains the entry address of the exception service routine base on the base address in the M-mode Exception Vector Base Address Register (MTVEC) and jumps to the exception service

routine.

The jump entries of all exceptions are defined by the MTVEC register. The first data entry obtained from the jump entry is the first instruction of the exception service routine. E902 implements only MTVEC.mode=3. Therefore, the entry address of the exception service routine must be 64-byte aligned. For specific information, please refer to *M-Mode Exception Vector Base Address Register (MTVEC)*.

Table 4.3: The Definition of Updated Values in the MTVAL Register

Exception Vector ID	Exception Type	Updated Value in MTVAL
1	Fetch instruction access error exception	Address accessed by the fetch instruction
2	Illegal instruction exception	Instruction code
3	Debug breakpoint exception	0
4	Load instruction unaligned access exception	Address accessed by the load instruction
5	Load instruction access error exception	Address accessed by the load instruction
6	Store instruction unaligned access exception	Address accessed by the store instruction
7	Store instruction access error exception	Address accessed by the store instruction
8	U-mode environment call exception	0
11	M-mode environment call exception	0

4.1.2 Exception Handling

As described in the preceding section, the jump entries of all exceptions are specified by the MTVEC register. After jumping to this entry, the software can determine whether to jump to corresponding service routines based on the exception vector ID in the MCAUSE register. It is noted that, in addition to saving necessary Control and Status Register (CSR) and Program Counter (PC) states during exception handling, software must also stack General Purpose Registers (GPRs) and other registers that will be applied at the entry of the exception service routine.

4.1.3 Return from Exceptions

CPU can execute the MRET instruction to return from the exception service routine. When the MRET instruction is executed, it restores the CPU context saved during the exception response, primarily encompassing the following aspects:

1. The PC value is restored to the value in the MEPC register, ensuring that upon returning from the exception service routine, the CPU resumes execution from the point where the exception was

triggered. This necessitates that any events triggering the exception be corrected during the preceding exception handling process, to prevent reoccurrence of the exception.

2. The MIE bit in the MSTATUS register is restored to the value of the MPIE bit in the MSTATUS register, and the MPIE bit is set to 1.
3. The PM field in the MXSTATUS register is restored to the value of the MPP field in the MSTATUS register. The MPP field is set to 2'b00 (when the hardware implements the U-mode) or 2'b11 (when the hardware implements only the M-mode).

4.1.4 Lock

As described in the preceding section, no exception enable register is defined in the RISC-V programming models. When the CPU returns from the exception service routine to the normal program flow, the exception vector ID in the MCAUSE register is not cleared. As a result, software developers can't figure out whether the CPU is handling exceptions or running programs normally based on the registers defined in the programming model. E902 implements *Extended Exception Status Register (MEXSTATUS)*. When the CPU responds to exceptions, the EXPT_VLD field of this register is set to 1. When the CPU executes the MRET instruction to return from the exception service routine, the field is cleared.

When the CPU responds to exceptions, it determines whether the current program stream is in the exception service routine, based on the EXPT_VLD field in the MEXSTATUS register. If the CPU is handling an exception and does not exit the exception service routine while a new exception is triggered, the CPU is locked. In this scenario, the CPU top-layer output signal (cpu_pad_lockup) is set to a high level, to signal the SoC that the CPU is locked. At the same time, CPU stops instruction fetch and execution, executes and maintains the PC at the value of the instruction PC that triggered the CPU lock. The LOCKUP field in the MEXSTATUS register is set to 1.

Debug requests and NMI requests described below can interrupt the CPU lock state. The lock indication signal sent from the CPU to the SoC is set to a low level. The CPU remains in a locked state even after returning from the NMI handler. There is a distinction in CPU behavior upon exiting debug mode. When in debug mode, if software sets the PC value for the CPU to exit the debug mode to 0xefffffff, the CPU remains in a locked state even after exiting debug mode. Conversely, if the PC value is not set to 0xefffffff, the CPU's locked state is cleared upon exiting debug mode, including the active signal indicating CPU LOCKUP to the SoC, and the MEXSTATUS register flag MEXSTATUS.LOCKUP, along with the exception information (MEXSTATUS.EXPT_VLD) that led to the CPU lockup. The CPU then proceeds to execute instructions.

When the CPU is locked, it is recommended that system design personnel reset the CPU core.

In particular, exceptions triggered actively by ECALL or EBREAK do not invoke CPU lockup when nested with other types of exceptions.

4.2 Interrupts

E902 implements the core local interrupts (CLINTs) and a core local interrupt controller (CLIC) that are compatible with the RISC-V standard. The former includes M-mode software interrupts, M-mode timer interrupts, and M-mode external interrupts. The CLIC controls sampling, priority arbitration, and distribution of interrupt sources.

The CLIC implemented in E902 is compatible with the CLIC SPEC-0.8. In terms of the SPEC definition, when the hardware implements the CLIC, `MTVEC.mode[1:0]` extends the 2'b11 mode to support hardware vector interrupts and non-vector interrupts. The entry of the interrupt service routine (ISR) can be specified by the `MTVT` register. E902 only implements `MTVEC.mode[1:0]=3`. This section mainly describes handling of vector interrupts and non-vector interrupts in this mode.

4.2.1 Vector Interrupts

You can set the `shv` field in the `CLICINTATTR` register of each interrupt in the CLIC to 1 to specify the interrupt as a hardware vector interrupt.

4.2.1.1 Interrupt Priorities

E902 supports 2 to 5 valid bits for the interrupt priority and up to 32 interrupt priorities. The interrupt priority is set by the following 2 steps:

1. Set the `nlbits` field in the `CLICCFG` register. This field specifies the configurable number of interrupt priorities.
2. Set `int_ctl` field of the `CLICINTCTL` register corresponding to each interrupt. This field specifies the valid value of the interrupt in priority arbitration and the valid value stored in the `MIL` field of the `MINTSTATUS` register.

For specific information of register configuration, please refer to *CLIC Registers*.

4.2.1.2 Interrupt Response

To ensure that the CPU responds to interrupts properly, both the global interrupt enable bit (`MIE`) in the `MSTATUS` register and the enable bit in the `CLICINTIE` register of the interrupt must be set to 1.

The CPU responds to interrupts in the following steps, and all the steps are completed in one clock cycle of the CPU.

1. The CPU saves the PC value to the M-mode exception program counter (`MEPC`). If the instruction for interrupt responding simultaneously triggers an exception, the value saved in the `MEPC` register serves as the instruction for interrupt responding. Otherwise, it serves as the instruction following the one that responded to the interrupt.

2. The CPU updates the Exception Code field in the MCAUSE register to the current valid interrupt ID. For specific vector IDs, please refer to [Table 4.1](#) . In addition, the CPU sets the most significant bit of the MCAUSE register to 1, indicating that the CPU responds to an interrupt.
3. The CPU saves the MIE bit in the MSTATUS register to MPIE.
4. The CPU clears the MIE bit in the MSTATUS register to disable responding to interrupts.
5. The CPU saves the PM field in the MXSTATUS register to the MPP field in the MSTATUS register and sets the PM field to the M-mode. In other words, the CPU enters M-mode after it responds to the interrupt.
6. The CPU updates the MPIL field in the MCAUSE register to the MIL field in the MINTSTATUS register. The MIL field in the MINTSTATUS register is updated to the priority of the current responded interrupt.
7. For pulse interrupts, the CPU automatically clears the pending bit in the corresponding CLICINTIP register. Whereas for level-sensitive interrupts, it does not clear this bit.

When MTVEC.mode is set to 3, the entry for hardware vector interrupts is determined by an offset specified by the result of adding the base address stored in the MTVT register to the offset specified by the interrupt ID. For more information, please refer to *M-Mode Exception Vector Base Address Register (MTVEC)*.

4.2.1.3 Interrupt Handling

At the entry of ISR, stacking is required based on the number of GPRs applied within the ISR. Once the stacking is completed, setting the MSTATUS.MIE bit to 1 enables interrupts, facilitating further interrupt responsiveness. Additionally, the CLIC SPEC introduces the MINTSTATUS.MIL field to represent the priority of the interrupt currently being handled by the CPU. This is compared against the priority of newly arriving interrupts transferred by the CLIC to the pipeline core, to determine if nested interrupt handling can be initiated. It is noted that, for nested interrupt response, registers such as MSTATUS and MCAUSE must also be stacked and saved at the entry of ISR.

4.2.1.4 Return from Interrupts

The CPU must execute the MRET instruction to return from the ISR. Before executing MRET instruction, the software must pop the stacked context saved at the ISR entry. The CPU context before responding to an interrupt is restored by executing the MRET instruction with the following key operations:

1. The PC value is restored to the value in the MEPC register.
2. The PM field in the MXSTATUS register is restored to the value of the MPP field in the MSTATUS register. The MPP field is set to 2'b00 (when the hardware implements the U-mode) or 2'b11 (when the hardware implements only the M-mode).

3. The MIE bit in the MSTATUS register is restored to the value of the MPIE bit in the MSTATUS register.
4. The MIL bit in the MINTSTATUS register is updated to the value of the MPIL bit in the MCAUSE register.

4.2.2 Non-Vector Interrupts

You can set the shv field in the configuration register CLICINTATTR of each interrupt in the CLIC to 0, to specify the interrupt as a non-vector interrupt.

4.2.2.1 Interrupt Priorities

For more information, please refer to *Interrupt Priorities*.

4.2.2.2 Interrupt Response

Similar to vector interrupt response, the CPU saves the CPU context during interrupt responding. However, the CPU does not clear the corresponding pending bit of the interrupts in the CLIC, whether they are pulse or level interrupts. The software needs to read the MNXTI register in the ISR to clear the pending bit. For more information, please refer to *Interrupt Tail Chaining*.

In addition, ISR entries for non-vector interrupts are uniformly specified by the MTVEC register, which is different from hardware vector interrupts whose ISR entries are specified by both the MTVT register and interrupt ID offset. And the data obtained from the entry is the first instruction of the ISR.

4.2.2.3 Return from Interrupts

For more information, please refer to *Return from Interrupts*.

4.2.2.4 Interrupt Tail Chaining

CLIC SPEC defines interrupt tail chaining in non-vector interrupt mode and supports the CPU to respond to new interrupts with higher priorities at the ISR entry.

In the non-hardware vector interrupt mode, where the entry point of ISR is uniformly specified by the MTVEC register, a unified stack operation for registers can be performed at the ISR entry. Upon completing the stack operation, the value of the MNXTI register can be read. If a non-zero value is returned, it signifies the highest priority interrupt currently pending, as arbitrated by the CLIC, which could either be the one currently being processed or a more recent, higher-priority one. In addition, the pending bit of the interrupt in the CLIC is cleared (only for pulse interrupts). Meanwhile, read and write operations on the MNXTI register will set the MIE bit of the MSTATUS register to 1.

Similarly, following the completion of the interrupt handling for a non-vector interrupt itself, prior to performing the stack pop and executing the MRET instruction to return, by executing instructions that read and write to the MNXTI register, the entry address of an interrupt with a priority higher than that in MCAUSE.MPIL field through CLIC arbitration. This address is determined by an offset specified by the interrupt number added to the MTVT register. Software can then use this address to load ISR entry point for the pending interrupt from memory and proceed to jump to execute it. If a zero value is returned, indicating no need to handle an interrupt as arbitrated by the CLIC (or there are no pending interrupts in the CLIC), the process can directly proceed with stack pop and interrupt return.

4.3 NMI

NMIs are external events, and E902 defines the exception vector ID of NMIs to 24, which has the highest among priorities of all interrupts and exceptions. NMIs are not controlled by the MIE bit in the MSTATUS register. So E902 extends and implements the MNMICAUSE and MNMIPC registers to save the CPU context during NMI responding, to ensure that the CPU can restore to the normal program flow after it responds to and returns from a NMI anytime.

4.3.1 NMI Response

1. Save the value of MEPC register to the MNMIPC register.
2. Save the PC value of the instruction for responding to the NMI request to the MEPC register, so that the CPU returns to the normal program flow after handling the NMI.
3. Save the Exception Code field and interrupt indication bit of the MCAUSE register to the MNMICAUSE register, update Exception Code in the MCAUSE register to 12'h18, and set the interrupt enable bit (bit[31]) in the MCAUSE register to 0.
4. Save the MPP and MPIE fields in the MSTATUS register to the MNMICAUSE register, update the PM field in the MXSTATUS register to the MPP field, and save the MIE field in the MSTATUS register to the MPIE field.

When saving the preceding CPU status, the CPU jumps to and executes the NMI service routine based on the unified exception entry address specified in the MTVEC register.

New NMIs as well as regular interrupt requests are not responded to until the CPU returns to its normal program flow from responding to NMIs. When an exception is triggered in the NMI service routine, the CPU is locked, as shown in [Lock](#) . NMI status maintenance depends on the Exception Code field in the MCAUSE register. So the software needs to avoid modifying this register's field in the NMI service routine.

4.3.2 Return from NMI

To return from the NMI service routine, the CPU needs to execute the MRET instruction to restore the CPU context. The main operations are as follows:

1. Restore the PC value to the value in the MEPC register, and restore the value in the MEPC register to the value in the MNMIPC register.
2. Restore MCAUSE.exception_code and the interrupt indication bit in the MCAUSE register to the values saved in the MNMICAUSE register.
3. Restore the PM field in the MXSTATUS register to the MPP field in the MSTATUS register.
4. Restore the MPIE and MPP fields in the MSTATUS register to the values saved in the MNMICAUSE register.
5. Restore the NMI_MPP field in the MNMICAUSE register to 2'b00 (when the hardware implements the U-mode) or 2'b11 (when the hardware implements only the M-mode).

4.3.3 Lock

If an exception is triggered in the NMI service routine, the CPU will enter a locked state. As described in *Lock*, the CPU stops fetching or executing instructions and sets the LOCKUP field in the MEXSTATUS register to 1. In this case, even if the CPU receives a new NMI request, the lock state cannot be interrupted. While, the input reset request signals can comprehensively break the locked state of the CPU, forcing it to restart execution from the reset boot address.

E902 applies RV32E[M]C instruction sets with 16-bit/32-bit mixed encoding, and extends Xuantie custom instruction sets. E902 extended instruction sets can be used only when the THEADISAE bit in the Machine Mode Extended Status (MXSTATUS) register is enabled. Otherwise, illegal instruction exceptions will occur.

5.1 RV32IMAFDC Instruction

This chapter mainly describes the RV32EMC instruction sets implemented in E902, including the standard integer instruction set (RV32E), RV multiply and divide instruction set (RV32M), and RV compressed instruction set (RVC).

5.1.1 RV32E Integer Instruction Set

This section describes 32-bit basic integer instructions. As a basic instruction set, RV32E implements the same instructions as RV32I. However, RV32E implements only 16 General Purpose Registers (GPRs). Access to additional GPRs will cause illegal instruction exceptions. The basic integer instruction set can be categorized by features as follows:

- Add and subtract instructions
- Logical operation instructions
- Shift instructions

- Compare instructions
- Data transfer instructions
- Branch jump instructions
- Memory access instructions
- CSR operation instructions
- Low power instructions
- Exception-return instructions
- Special function instructions

Table 5.1: Integer Instruction Set (RV32E) List

Instruction	Description	Execution Delay
Add and Subtract Instructions		
ADD	A signed add instruction.	1
ADDI	A signed immediate add instruction.	1
SUB	A signed subtract instruction.	1
Logical Operation Instructions		
AND	A bitwise AND instruction.	1
ANDI	An immediate bitwise AND instruction.	1
OR	A bitwise OR instruction.	1
ORI	An immediate bitwise OR instruction.	1
XOR	A bitwise XOR instruction.	1
XORI	An immediate bitwise XOR instruction.	1
Shift Instructions		
SLL	A logical left shift instruction.	1
SLLI	An immediate logical left shift instruction.	1
SRL	A logical right shift instruction.	1
SRLI	An immediate logical right shift instruction.	1
SRA	An arithmetic right shift instruction.	1
SRAI	An immediate arithmetic right shift instruction.	1
Compare Instructions		
SLT	A signed set-if-less-than instruction.	1
SLTU	An unsigned set-if-less-than instruction.	1
SLTI	A signed set-if-less-than-immediate instruction.	1
SLTIU	An unsigned set-if-less-than-immediate instruction.	1
Data Transfer Instructions		
LUI	A load upper immediate instruction.	1
AUIPC	An add upper immediate to PC instruction.	1

Continued on next page

Table 5.1 – continued from previous page

Branch Jump Instructions		
BEQ	A branch-if-equal instruction.	1
BNE	A branch-if-not-equal instruction.	1
BLT	A signed branch-if-less-than instruction.	1
BGE	A signed branch-if-greater-than-or-equal instruction.	1
BLTU	An unsigned branch-if-less-than instruction.	1
BGEU	An unsigned branch-if-greater-than-or-equal instruction.	1
JAL	An instruction for directly jumping to a subroutine.	1
JALR	A jump and link register instruction	1
Memory Access Instructions		
LB	A sign-extended byte load instruction.	2 (hit cache)
LBU	An unsign-extended byte load instruction.	
LH	A sign-extended halfword load instruction.	
LHU	An unsign-extended halfword load instruction.	
LW	A sign-extended word load instruction.	
SB	A byte store instruction.	
SH	A halfword store instruction.	
SW	A word store instruction.	
CSR Operation Instructions		
CSRRW	A CSR read/write instruction.	Blocked
CSRRS	A CSR read/set instruction.	
CSRRC	A CSRs read/clear instruction.	
CSRRWI	A CSR read/write immediate instruction.	
CSRRSI	A CSR read/set immediate instruction.	
CSRRCI	A CSR read/clear immediate instruction.	
Low-Power Instructions		
WFI	An instruction for entering the low-power mode.	Uncertain duration
Exception-Return Instructions		
MRET	An instruction for returning from exceptions in M-mode.	Blocked
Special Function Instructions		
FENCE	A memory synchronization instruction.	Uncertain duration
FENCE.I	An instruction stream synchronization instruction.	Blocked
ECALL	An environment exception call instruction.	1
EBREAK	A breakpoint instruction.	1

For specific instruction descriptions and definitions, please refer to *Appendix A-1 E Instructions* .

5.1.2 RV32M Multiply and Divide Instruction Set

RV32M multiply and divide instruction set is listed in *Table 5.2* .

Table 5.2: RV32M Instruction List

Instruction	Description	Execution Delay
Multiply and Divide Instructions		
MUL	A signed multiply instruction.	1/3-33
MULH	A signed multiply instruction that extracts upper bits.	1/3-33
MULHSU	A signed-unsigned multiply instruction that extracts upper bits.	1/3-33
MULHU	An unsigned multiply instruction that extracts upper bits.	1/3-33
DIV	A signed divide instruction.	1-33
DIVU	An unsigned divide instruction.	1-33
REM	A signed remainder instruction.	1-33
REMU	An unsigned remainder instruction.	1-33

For specific instruction descriptions and definitions, please refer to *Appendix A-2 M Instructions*.

5.1.3 RVC Compressed Instruction Set

This section mainly describes 16-bit compressed instructions. The compressed instruction set can be categorized by features as follows:

- Add and subtract instructions
- Logical operation instructions
- Shift instructions
- Data transfer instructions
- Branch jump instructions
- Immediate offset access instructions

Table 5.3: RVC Compressed Instruction Set List

Instruction	Description	Execution Delay
Add and Subtract Instructions		
C.ADD	A signed add instruction.	1
C.ADDI	A signed immediate add instruction.	1
C.SUB	A signed subtract instruction.	1
C.ADDI16SP	An instruction to add immediate scaled by 16 to stack pointer.	1
C.ADDI4SPN	An instruction to add immediate scaled by 4 to stack pointer.	1
Logic Operation Instructions		

Continued on next page

Table 5.3 – continued from previous page

Instruction	Description	Execution Delay
C.AND	A bitwise AND instruction.	1
C.ANDI	An immediate bitwise AND instruction.	1
C.OR	A bitwise OR instruction.	1
C.XOR	A bitwise XOR instruction.	1
Shift Instructions		
C.SLLI	An immediate logical left shift instruction.	1
C.SRLI	An immediate logical right shift instruction.	1
C.SRAI	An immediate arithmetic right shift instruction.	1
Data Trasfer Instructions		
C.MV	A data transfer instruction.	1
C.LI	An immediate transfer instruction	1
C.LUI	An upper bit immediate transfer instruction	1
Branch Jump Instructions		
C.BEQZ	A branch-if-equal-to-zero instruction.	1
C.BNEZ	A branch-if-not-equal-to-zero instruction.	1
C.J	An unconditional jump instruction.	1
C.JR	A jump-to-register instruction	1
C.JAL	An instruction for jumping to a subroutine unconditionally.	1
C.JALR	A jump and link register instruction	1
Memory Access Instructions		
C.LW	A word load instruction.	2 (no bus delay)
C.SW	A word store instruction.	
C.LWSP	A load Word from stack pointer instruction.	
C.SWSP	A word stack store instruction.	
Special Instructions		
C.NOP	A no-operation instruction.	1
C.EBREAK	A debug breakpoint instruction.	1

For specific instruction descriptions and definitions, please refer to *Appendix A-3 C Instructions*.

5.2 Xuantie Extended Instruction Set

5.2.1 Cache Instruction Set

Table 5.4: Cache Instruction List

Instruction	Description	Execution Delay	Remarks
ICACHE.IALL	An instruction that invalidates all page table entries in the I-Cache.	M/2	M: number of cache lines in the I-Cache
ICACHE.IPA	An instruction that invalidates page table entries in the I-Cache that match the specified physical address.		

Table 5.4 describes the extended Xuantie custom cache instruction subsets implemented in E902. Cache instructions can be executed normally when the CPU is in M-mode and the THEADISAE bit in the MXSTATUS register is enabled. Otherwise, illegal instruction exceptions occur.

For specific instruction descriptions and definitions, please refer to *Appendix B-1 Cache Instructions* .

6.1 PMP Overview

The physical memory protection (PMP) of E902 complies with the RISC-V standard. The PMP mainly protects two types of system resources: memory and device. And it checks the access validity of memory and device and determines whether the CPU has the read/write/execute permissions to the memory address in the current operation mode.

The PMP unit can be configured with 0 (PMP not implemented)/4/8/12/16 entries. In M-mode, the CPU can configure the access permissions to addresses. Each entry is identified and indexed by a number ranging from 0 to 15.

The PMP unit of E902 provides the following key features:

- Supports 0/4/8/12/16 PMP entries for hardware.
- Supports the minimum address split granularity of 4 bytes.
- Supports four address matching modes: OFF, Top of Range (TOR), Naturally Aligned 4-Byte Region (NA4), Naturally Aligned Power-of-Two Region (NAPOT);
- Supports three permissions: readable, writable, and executable.
- Supports entry lock.

6.2 PMP CSR

6.2.1 Physical Memory Protection Configuration Register (PMPCFG0~PMPCFG3)

PMPCFG0~PMPCFG3 is applied to configuring the lock mode, operation mode, and access permissions of PMP entries. E902 provides four PMPCFG registers in total. Each register can be configured with four entries, with up to 16 entries can be configured in total.

Each register is 32-bit wide and is readable and writable only in M-mode. Accesses in other modes will cause an illegal instruction exception.

Each 32-bit PMPCFG register provides permission settings of four entries, and the overall layout is illustrated in Fig. 6.1 .

	31	24	23	16	15	8	7	0		
	pmp3cfg			pmp2cfg			pmp1cfg		pmp0cfg	PMPCFG0
Reset	0			0			0		0	

	31	24	23	16	15	8	7	0		
	pmp7cfg			pmp6cfg			pmp5cfg		pmp4cfg	PMPCFG1
Reset	0			0			0		0	

	31	24	23	16	15	8	7	0		
	pmp11cfg			pmp10cfg			pmp9cfg		pmp8cfg	PMPCFG2
Reset	0			0			0		0	

	31	24	23	16	15	8	7	0		
	pmp15cfg			pmp14cfg			pmp13cfg		pmp12cfg	PMPCFG3
Reset	0			0			0		0	

Fig. 6.1: Overall Layout Figure of PMPCFG Register

The format of each PMPCFG register is illustrated in Fig. 6.2 .

Reset	7	6	5	4	3	2	1	0
	L	-	A		X	W	R	
	0	0	0		0	0	0	0

Fig. 6.2: — PMPCFG Register Format

L-Lock bit:

When L is 0, the PMPCFG register and address register of this entry can both be modified. In M-mode, the access permissions are not constrained by the R/W/X bit. While in U-mode, the access permissions are controlled by the R/W/X bit.

When L is 1, all content of the entry, including the L bit, cannot be modified before a CPU reset. Access permissions in M-mode and U-mode are controlled by the R/W/X bit. When the PMPCFG register of the entry is configured for Top of Range (TOR) address matching mode, the PMPCFG register of the previous entry cannot be modified either.

The reset value is 0.

A-address matching mode:

When A is 00, the entry is invalid.

When A is 01, the TOR mode is applied, and the addresses of the adjacent entries are applied as the matching range.

When A is 10, the NA4 mode is applied, and the matching range is 4 bytes.

When A is 11, the NAPOT mode is applied, and the matching range is a power of 2.

The reset value is 0.

X-executable attribute:

When X is 0, the region is non-executable.

When X is 1, the region is executable.

The reset value is 0.

W-writable attribute:

When W is 0, the region is unwritable.

When W is 1, the region is writable.

The reset value is 0.

R-readable attribute:

When R is 0, the region is unreadable.

When R is 1, the region is readable.

The reset value is 0.

6.2.2 Physical Memory Protection Address Register (PMPADDR0~PMPADDR15)

PMPADDR0~PMPADDR15 is applied to configuring the addresses and region sizes of PMP entries. E902 provides 16 PMPADDR registers in total, and each register corresponds to one entry.

Each register is 32-bit wide and is readable and writable only in M-mode. Accesses in other modes will cause an illegal instruction exception.

The entry region size is determined by the PMPADDR and PMPCFG registers.

- In TOR mode, the region size controlled by entry i is $(\{PMPADDR_{i-1}[31:0], 2'b0\}, \{PMPADDR_i[31:2], 2'b0\})$. Entry 0 applies 0x0 as the lower boundary of the address space, that is $(0, \{PMPADDR_0[31:0], 5'b0\})$; If the lower boundary of this range is greater than or equal to the upper boundary, the entry is considered OFF, meaning it is disabled, and all access requests will not hit this entry.
- In NA4 mode, the region size of entry i is $(\{PMPADDR_{i-1}, 2'b0\}, \{PMPADDR_i, 2'b0\})$.
- The relationship between addresses and corresponding protection region sizes in NAPOT mode is illustrated in Table 6.1 .

In NAPOT mode, the region size and base address of entry i are determined by $PMPADDR_i$.

Assuming that the first occurrence of a bit with a value of 0 in $PMPADDR_i$ from bit[0] to bit[31] is the n th bit, that is, $PMPADDR_i[n]$. Then the base address of this entry is $\{PMPADDR_i[31:n+1], (n+3)\{1'b0\}\}$, and the corresponding space is 2^{n+3} bytes (excluding the address corresponding to the space size upper limit). Take

$PMPADDR_i = 32'b\text{aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_aaa0_1111}$ (a is 1'b0 or 1'b1) as an example, its base address is $34'b\text{aa_aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_a000_0000}$, and the space size is 128 bytes.

A PMPADDR register supports up to 16 GiB physical space. However, E902 applies 32-bit addresses and supports up to 4 GiB. Therefore, even though the two upper bits of $PMPADDR_i$ can be read and written by software, they do not participate in logical operations for address matching.

Table 6.1: Protection Region Code

PMPADDR _i	PMPCFG.A	Size of Matching Region
aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_aaaa	NAPOT	4B
aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_aaa0	NAPOT	8B
aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_aa01	NAPOT	16B
aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_a011	NAPOT	32B
aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_0111	NAPOT	64B
aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_aaa0_1111	NAPOT	128B
aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_aa01_1111	NAPOT	256B
aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_a011_1111	NAPOT	512B
aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_0111_1111	NAPOT	1KiB
aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_aaa0_1111_1111	NAPOT	2KiB
aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_aa01_1111_1111	NAPOT	4KiB
aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_a011_1111_1111	NAPOT	8KiB
aaaa_aaaa_aaaa_aaaa_aaaa_aaaa_0111_1111_1111	NAPOT	16KiB

Continued on next page

Table 6.1 – continued from previous page

PMPADDRi	PMPCFG.A	Size of Matching Region
aaaa_aaaa_aaaa_aaaa_aaa0_1111_1111_1111	NAPOT	32KiB
aaaa_aaaa_aaaa_aaaa_aa01_1111_1111_1111	NAPOT	64KiB
aaaa_aaaa_aaaa_aaaa_a011_1111_1111_1111	NAPOT	128KiB
aaaa_aaaa_aaaa_aaaa_0111_1111_1111_1111	NAPOT	256KiB
aaaa_aaaa_aaaa_aaa0_1111_1111_1111_1111	NAPOT	512KiB
aaaa_aaaa_aaaa_aa01_1111_1111_1111_1111	NAPOT	1MiB
aaaa_aaaa_aaaa_a011_1111_1111_1111_1111	NAPOT	2MiB
aaaa_aaaa_aaaa_0111_1111_1111_1111_1111	NAPOT	4MiB
aaaa_aaaa_aaa0_1111_1111_1111_1111_1111	NAPOT	8MiB
aaaa_aaaa_aa01_1111_1111_1111_1111_1111	NAPOT	16MiB
aaaa_aaaa_a011_1111_1111_1111_1111_1111	NAPOT	32MiB
aaaa_aaaa_0111_1111_1111_1111_1111_1111	NAPOT	64MiB
aaaa_aaa0_1111_1111_1111_1111_1111_1111	NAPOT	128MiB
aaaa_aa01_1111_1111_1111_1111_1111_1111	NAPOT	256MiB
aaaa_a011_1111_1111_1111_1111_1111_1111	NAPOT	512MiB
aaaa_0111_1111_1111_1111_1111_1111_1111	NAPOT	1GiB
aaa0_1111_1111_1111_1111_1111_1111_1111	NAPOT	2GiB
aa01_1111_1111_1111_1111_1111_1111_1111	NAPOT	4GiB
a011_1111_1111_1111_1111_1111_1111_1111	NAPOT	8GiB
0111_1111_1111_1111_1111_1111_1111_1111	NAPOT	16GiB

6.3 Memory Access Handling

After the CPU configures PMP entries, the PMP unit checks memory access addresses and determines whether the accessed address is in the protection regions. When the accessed address hits one or more PMP entries, the entry with a higher index is preferentially matched (0 indicates the highest index, and 15 indicates the lowest one).

For hit entries, the L, R, W, and X bits jointly determine whether the access is successful. When L is 0, the access permissions in M-mode are not constrained by the R/W/X bit and all access requests will be successful; In U-mode, the access permissions are controlled by the X/W/R bit. Only when the access type matches the access attribute of an entry, the access is successful; otherwise, the access fails. When L is 1, access permissions in M-mode and U-mode are controlled by the R/W/X bit. Only when the access type matches the access attribute of the entry, is successful; otherwise, the access fails.

When no PMP entry is enabled, that is, the address matching mode is OFF (2'b00) or at least one PMP entry is enabled while the accessed address does not match any of the enabled entries, the current mode of operation for CPU determines whether the access is successful. If the CPU is in M-mode, the access is successful. If the CPU is in U-mode, the access fails.

When access fails, memory access is suspended and the CPU reports an exception based on the access type, such as a load access exception, a store access exception, or an instruction access exception.

It should be noted that for memory access operations by fetch instructions, the access permissions are related only to the mode the CPU is in when the request is made; For load and store memory operations, the access permissions are determined by both the mode of operation for CPU and the MPRV and MPP bits in the MSTATUS register. For specific information, please refer to *M-Mode Status Register (MSTATUS)*.

E902 implements the configurable instruction cache (I-Cache), which provides the following main features:

- I-Cache size is hardware configurable, supporting options of 2KiB, 4KiB, and 8KiB.
- 2-way set-associative with a cache line size of 16 bytes;
- Data width for access: 32 bits;
- First-in, first-out (FIFO) strategy;
- Supports invalidation for all I-Cache and the invalidation for a single cache line.

7.1 Cache Operation

The I-Cache is disabled by default after the CPU is reset. Based on RISC-V programming models, E902 extends CSRs and instructions related to cache operations, and supports cache enabling, disabling, invalidation and other operations.

7.1.1 Extended Cache Register

E902 supports the extended cache register: M-mode hardware configuration register (MHCR) to enable and disable the I-Cache.

For more information about CSRs, please refer to *M-mode Hardware Configuration Register (MHCR)* in Appendix C Extended M-mode register group.

7.1.2 Extended Cache Instructions

E902 extends cache instructions, as listed in Table 7.1 .

Table 7.1: Extended Cache Instructions

Instruction	Description
ICACHE.IALL	An instruction that invalidates all page table entries in the I-Cache.
ICACHE.IPA	An instruction that invalidates page table entries in the I-Cache that match the specified physical addresses.

7.2 Memory Model

7.2.1 Memory Attribute

Xuantie E902 supports configuration of two memory attributes, whether the memory is cacheable or non-cacheable and whether it is bufferable or non-bufferable. For cacheable memory address spaces, data accessed from this region by the E902 can be cached in its internal instruction or D-caches. In terms of bufferable memory address spaces, when the E902 performs write operations to this address space, the bus write response (HRESP) can be returned by any node on the bus path, instead of waiting for the write operation to complete at the slave device accessed by the E902. Conversely, for non-bufferable memory address spaces, the response to write operations of this address space by E902 must be returned to the E902 by the slave device upon completion of the write operation at the accessed slave.

All memory address space attributes are configured in the Xuantie custom sysmap.h file. The sysmap.h file is delivered to customers together with E902 codes. Customers need to configure memory space attributes during system design and save the file to the project file list. The software cannot modify the attributes after they are configured.

The configuration of memory attributes across the entire address space is accomplished in the customized sysmap.h file by Xuantie. This sysmap.h file is delivered alongside the E902 code to customers, who need to finalize the settings for their memory space attributes during the system design phase, and then include this file in their project's source list. Once set, the software cannot modify these memory attribute configurations.

sysmap.h file supports to configure attributes for up to 8 address spaces. The upper address boundary (excluded) of address space i ($i = 0$ to 7) is defined by the macro `SYSMAP_BASE_ADDR i` ($i = 0$ to 7). The lower address boundary (included) is defined by the macro `SYSMAP_BASE_ADDR(i - 1)`, that is, `SYSMAP_BASE_ADDR(i - 1) <= Address of address space i < SYSMAP_BASE_ADDR i`. The lower address boundary of address space 0 is 0x0. Attributes of memory addresses beyond the eight address spaces defined in the sysmap.h file are cacheable and bufferable by default. The upper and lower boundaries of each address space are 4 KiB aligned. Therefore, the macro `SYSMAP_BASE_ADDR i` defines the upper 20 bits of an address.

Attributes of memory addresses within address space i ($i = 0$ to 7) are defined by the macro `SYSMAP_FLAG i` ($i = 0$ to 7), as shown in Fig. 7.1 :

4	3	2	1	0
-	Cacheable	Bufferable	-	-

Fig. 7.1: Address Attribute in sysmap.h File

7.2.2 Configuration Reference of SYSMAP

- The attribute definition of address space 0: $32'h0 \leq \text{addr0}[31:0] < 32'h3fff_f000$, $\text{flg0} = 5'b01111$. This address space segment includes parts of INST_RAM and DATA_RAM, configured with cacheable attribute, and the attribute definition is as follows:

```
`define CR_SYSMAP_BASE_ADDR0    20'h3ffff
`define CR_SYSMAP_FLG0          5'b01100
```

- The attribute definition of address space 1: $32'h3fff_f000 \leq \text{addr1}[31:0] < 32'hafff_f000$, $\text{flg1} = 5'b00000$. This address space segment mainly corresponds to the write addresses of special function (i.e., the print function) and APB interface, configured with non_cacheable attribute, ensuring that the operations are guaranteed to be issued from the core to the bus. The attribute definition is as follows:

```
`define CR_SYSMAP_BASE_ADDR1    20'haffff
`define CR_SYSMAP_FLG1          5'b00000
```

- The attribute definition of address space 2: $32'hafff_f000 \leq \text{addr2}[31:0] < 32'hbfff_f000$, $\text{flg2} = 5'b01100$. The attribute definition is as follows:

```
`define CR_SYSMAP_BASE_ADDR2    20'hbffff
`define CR_SYSMAP_FLG2          5'b01100
```

- The attribute definition of address space 3: $32'hbfff_f000 \leq \text{addr3}[31:0] < 32'hffff_f000$, $\text{flg3} = 5'b00000$. The attribute definition is as follows:

```
`define CR_SYSMAP_BASE_ADDR3    20'hfffff
`define CR_SYSMAP_FLG3          5'b00000
```

- The attribute definition of address space 4: $32'hffff_f000 \leq \text{addr4}[31:0] < 32'hefff_f000$, $\text{flg4} = 5'b00000$. The attribute definition is as follows:

```
`define CR_SYSMAP_BASE_ADDR4    20'heffff
`define CR_SYSMAP_FLG4          5'b00000
```

- The attribute definition of address space 5: $32'h\text{heff_f000} \leq \text{addr5}[31:0] < 32'h\text{hff_f000}$, $\text{flg5} = 5'b01100$. The attribute definition is as follows:

```
`define CR_SYSMAP_BASE_ADDR5    20'hffeff
`define CR_SYSMAP_FLG5          5'b01100
```

- The attribute definition of address space 6: $32'h\text{hff_f000} \leq \text{addr6}[31:0] < 32'h\text{fff5_f000}$, $\text{flg6} = 5'b01100$. The attribute definition is as follows:

```
`define CR_SYSMAP_BASE_ADDR6    20'hfff5f
`define CR_SYSMAP_FLG6          5'b01100
```

- The attribute definition of address space 7: $32'h\text{fff5_f000} \leq \text{addr7}[31:0] < 32'h\text{ffff_f000}$, $\text{flg7} = 5'b00000$. The attribute definition is as follows:

```
`define CR_SYSMAP_BASE_ADDR7    20'hfffff
`define CR_SYSMAP_FLG7          5'b00000
```

8.1 Introduction

E902 features multiple bus interfaces, comprising instruction bus and system bus, as well as a Tightly Coupled IP (TCIP) interface. The address space of the instruction bus can be configured by users according to the actual system requirements. The address space for the TCIP interface is fixed at 0xE0000000 to 0xEFFFFFFF, with the remaining address space allocated to the system bus.

The bus matrix provides interconnect functionality for CPU internal requests to access external bus interfaces. The connection relationship among bus matrix, the CPU's internal requests and the bus interfaces is illustrated in Fig. 8.1 .

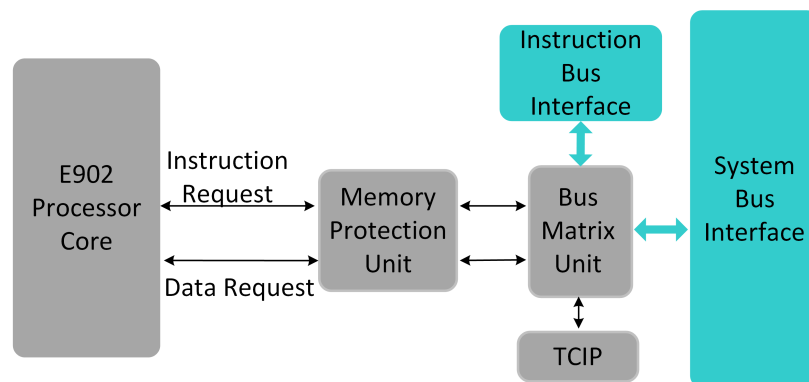


Fig. 8.1: E902 Bus Matrix

Instruction fetching and data access requests inside the CPU share the same bus access permissions, and

can access all bus interfaces. To resolve conflicts when both instruction fetch accesses and data accesses compete for the same bus interface during the same clock cycle, the bus matrix is also responsible for prioritizing these requests. In cases where instruction fetch requests and data requests contend for the same bus interface, data requests are granted higher priority.

In addition, E902 provides a group of interface signals (`pad_bmu_iahbl_base` and `pad_bmu_iahbl_mask`) to configure the instruction bus base address and space size during hardware integration.

`pad_bmu_iahbl_base` specifies the base address of the instruction bus; while `pad_bmu_iahbl_mask` specifies address alignment requirements in different address spaces.

The address space of the instruction bus is configurable from 1 MiB to 4 GiB. For example, when the instruction bus address space is set to 8 MiB, `pad_bmu_iahbl_base[2:0]` must be set to 3'b0 and `pad_bmu_iahbl_mask[11:0]` must be set to 12'b1111 1111 1000. For specific requirements on address spaces with different sizes, please refer to [Table 8.1](#).

Table 8.1: Requirements on the Base Address and Address Alignment of the Instruction Bus

Address Space Size	Requirements on <code>bmu_iahbl_base</code>	Requirements on <code>pad_bmu_iahbl_mask</code>
1MiB	No requirements	<code>bit[11:0]=12'b1111 1111 1111</code>
2MiB	<code>bit[0] =0</code>	<code>bit[11:0]=12'b1111 1111 1110</code>
4MiB	<code>bit[1:0] =2'b0</code>	<code>bit[11:0]=12'b1111 1111 1100</code>
8MiB	<code>bit[2:0] =3'b0</code>	<code>bit[11:0]=12'b1111 1111 1000</code>
16MiB	<code>bit[3:0] =4'b0</code>	<code>bit[11:0]=12'b1111 1111 0000</code>
32MiB	<code>bit[4:0] =5'b0</code>	<code>bit[11:0]=12'b1111 1110 0000</code>
64MiB	<code>bit[5:0] =6'b0</code>	<code>bit[11:0]=12'b1111 1100 0000</code>
128MiB	<code>bit[6:0] =7'b0</code>	<code>bit[11:0]=12'b1111 1000 0000</code>
256MiB	<code>bit[7:0] =8'b0</code>	<code>bit[11:0]=12'b1111 0000 0000</code>
512MiB	<code>bit[8:0] =9'b0</code>	<code>bit[11:0]=12'b1110 0000 0000</code>
1GiB	<code>bit[9:0] =10'b0</code>	<code>bit[11:0]=12'b1100 0000 0000</code>
2GiB	<code>bit[10:0] =11'b0</code>	<code>bit[11:0]=12'b1000 0000 0000</code>
4GiB	<code>bit[11:0] =12'b0</code>	<code>bit[11:0]=12'b0000 0000 0000</code>

Attention: Data must not be stored in the IAHB area during use.

The overlap between the instruction bus address space and TCIP interface is not supported in E902. When the bus interface is misconfigured, leading to a single address request simultaneously matching multiple locations within both the instruction bus address space and the TCIP interface, the CPU behavior becomes unpredictable. Consequently, when configuring hardware with multiple bus interfaces, it is advisable not to configure the instruction bus with a 4GiB address range.

8.2 System Bus Interfaces

The system bus interfaces of E902 support the AMBA3.0 AHB-Lite protocol. For more information, please refer to AMBA 3.0 Specification - *AMBA3 AHB-Lite Protocol Specification Rev 1.0*. The system bus interfaces of E902 implement only part of content in the AHB-Lite protocol.

According to the AHB-Lite protocol, as the master device, bus interface supports the following transfer types:

- The HBURST supports only SINGLE transfers; all other burst types are not supported.
- HTRANS supports only IDLE and NONSEQ; other transfer types are not supported.
- HSIZE supports word, byte, and half-word transfers; other sizes are not supported.
- HWRITE supports both read and write operations.

Within the AHB and AHB-Lite protocols, the bus interface accepts the following response types from slave devices:

- HREADY supports both Ready and Not Ready states.
- HRESP supports OKAY and ERROR responses; other response types are not supported.

8.3 Instruction Bus Interfaces

The instruction bus interfaces of E902 support only AMBA3.0 AHB-Lite protocol, as detailed in the AMBA 3.0 Specification- *AMBA3 AHB-Lite Protocol Specification Rev 1.0*. The instruction bus interfaces of E902 implement only part of content in the AHB-Lite protocol.

According to the AHB-Lite protocol, as the master device, the bus interface supports the following transfer types:

- The HBURST supports only SINGLE transfers; all other burst types are not supported.
- HTRANS supports only IDLE and NONSEQ; other transfer types are not supported.
- HSIZE supports word, byte, and half-word transfers; other sizes are not supported.
- HWRITE supports both read and write operations.

According to the the AHB-Lite protocol, the bus interface accepts the following response types from slave devices:

- HREADY supports both Ready and Not Ready states.
- HRESP supports OKAY and ERROR responses; other response types are not supported.

Core Local Interrupt

E902 implements the core local interrupts (CLINTs), including software interrupts and timer interrupts. This module's registers are mapped within the address space of the Tightly Coupled IP (TCIP).

9.1 Register Address Mapping

The CLINT register occupies 64 KiB memory space. The upper 16 bits are assigned with the fixed address 0xE000. While the mapping of the lower 16 bits is illustrated in [Table 9.1](#). All registers support only access to word-aligned addresses.

Table 9.1: Memory-Mapped Addresses for the CLINT Register

Address[15:0]	Name	Type	Initial Value	Description
0x0000	MSIP	Read/Write	0x00000000	An Machine Mode (M-mode) software interrupt configuration register: The upper bits are fixed to 0 for hardware, and bit[0] is valid.
Reserved	-	-	-	-
0x4000	MTIMECMPLO	Read/Write	0xFFFFFFFF	An M-mode counter: compare value register (lower 32 bits).

Continued on next page

Table 9.1 – continued from previous page

Address[15:0]	Name	Type	Initial Value	Description
0x4004	MTIMECMPHI	Read/Write	0xFFFFFFFF	An M-mode counter: compare value register (upper 32 bits).
Reserved	-	-	-	-
0xBFF8	MTIMELO	Read	0x00000000	An M-mode counter: the current value register (lower 32 bits). The register value is the value of signal pad_cpu_sys_cnt[31:0] *.
0xBFFC	MTIMEHI	Read	0x00000000	An M-mode counter: the current value register (upper 32 bits). The register value is the value of signal pad_cpu_sys_cnt[63:32] *.
Reserved	-	-	-	-

9.2 Software Interrupts

CLINT is applied to generating software interrupts during software configuration. M-mode software interrupts are controlled by the M-mode software interrupt pending (MSIP) register.

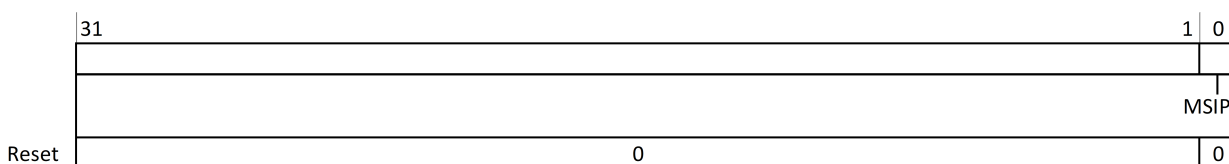


Fig. 9.1: MSIP Register

MSIP: M-mode software interrupt pending bit:

This bit indicates the status of M-mode software interrupts. CPU can read and write this bit in M-mode.

- When the MSIP bit is set to 1, valid M-mode software interrupt requests exist currently.
- When the MSIP bit is set to 0, no valid M-mode software interrupt requests exist currently.

9.3 Timer Interrupts

CLINT is applied to generating M-mode timer interrupts, which are used in conjunction with a 64-bit counter implemented externally by the E902 system design. The system counter needs to run in the always-on voltage domain. And it counts in each clock cycle after a reset. During E902 integration, the value of this 64-bit system counter needs to be transferred into E902 through the pad_cpu_sys_cnt[63:0] signal. Values of the upper 32 bits can be read from the MTIMEHI[31:0] register implemented in E902, while the values of the lower 32 bits can be read from the MTIMELO[31:0] register.

Meanwhile, E902 also implements 64-bit *M-mode timer compare value registers (MTIMECMPH and MTIMECMPL)*, which can be read and write through word-aligned address access.

CLINT compares the current value between {MTIMECMPH[31:0], MTIMECMPL[31:0]} and {MTIMEHI[31:0], MTIMELO[31:0]} of the system counter to determine whether to generate a timer interrupt.

- When the value of {MTIMECMPH[31:0], MTIMECMPL[31:0]} is greater than the value of the system counter, the CLINT controller does not generate an interrupt.
- When the value of {MTIMECMPH[31:0], MTIMECMPL[31:0]} is less than or equal to the value of the system counter, the CLINT controller generates a timer interrupt.

The software can rewrite the value of the MTIMECMPH/MTIMECMPL register to clear the corresponding timer interrupt.

All registers related to timer interrupts support accessing and modifying in M-mode, which can not be implemented in U-mode.

The two registers share the same structure, whose layout and definition is illustrated in Fig. 9.2 .

	63	32	31	0
	MTIMECMPH		MTIMECMPL	
Reset	0x0		0x0	

Fig. 9.2: MTIMECMPH/MTIMECMPL Registers (Upper/Lower)

MTIMECMPH/MTIMECMPL: registers that store the upper bits/lower bits of the M-mode timer interrupt compare value register

These registers store timer compare values.

- MTIMECMPH: stores the upper 32 bits of timer compare values.
- MTIMECMPL: stores the lower 32 bits of timer compare values.

	63	32	31	0
	MTIMEHI		MTIMELO	
Reset	0x0		0x0	

Fig. 9.3: MTIMEHI/MTIMELO Registers (Upper/Lower)

MTIMEHI/MTIMELO: registers that store the upper bits/lower bits of the current count value of the system counter used by the M-mode timer

These registers store the current values of system counter.

- MTIMEHI: stores the upper 32 bits of the current count value of the system counter.
- MTIMELO: stores the lower 32 bits of the current count value of the system counter.

Core-Local Interrupt Controller

The core local interrupt controller (CLIC) supports sampling, priority arbitration, and distribution of interrupt sources. The CLIC arbitration sources include interrupts triggered in different CPU modes. The CLIC of E902 provides the following fundamental features:

- Supports the RISC-V standard CLIC spec-0.8.
- Supports up to 240 external interrupt sources, level and pulse interrupts, 16 CLINT interrupts (only Machine Mode (M-mode) software interrupts and M-mode timer interrupts implemented currently), and a total of 256 interrupt handlings;
- Supports 2 to 5 valid bits for the interrupt priority (CLICINTCTLBITS) and up to 32 interrupt priorities.
- Supports four memory-mapped control and status registers (CSR) for each interrupt target.
- Supports writing CSRs of interrupt sources to configure the attributes for these interrupt sources.
- Supports the MSCRATCHCSW and MSCRATCHCSWL registers for quick stack pointer exchange in interrupt handling functions.

10.1 Interrupt Handling Mechanism

10.1.1 Interrupt Arbitration

For the CLIC, only interrupt sources that meet the specified conditions are involved in arbitration. The conditions include:

- The interrupt source is in the pending state ($IP = 1$).
- Interrupts with priority levels greater than the value set in the domain threshold register MINTTHRESH (only interrupts with non-zero priority levels can participate in the arbitration process normally).
- The interrupt enable bit in the CLIC is set to 1 ($IE = 1$).

When multiple interrupts in the CLIC are in the pending state, the CLIC selects the interrupt with the highest priority through arbitration. In the CLIC, the larger the value of the Interrupt Priority Configuration Register (CLICINTCTL), the higher the priority; interrupts with a priority of 0 are invalid. If multiple interrupts have the same priority, the interrupt with a larger interrupt ID is preferentially handled.

The CLIC will transfer the arbitration result, including the interrupt ID, priority, privilege state, and vector interrupt judgment to the CPU pipeline core. And the interrupt ID serves as the interrupt number for processing, with the corresponding interrupt IDs 0 to 15, and the interrupt IDs of CLIC external interrupt sources are 16 to 255.

10.1.2 Interrupt Requests and Responses

When a valid interrupt request is generated during CLIC arbitration and the priority of this interrupt is greater than the value set in the MINTTHRESH register, the CLIC triggers the interrupt request to the CPU.

When the CPU receives the valid interrupt request and the priority of this interrupt is higher than that of the interrupt responded inside the core (MINTSTATUS.MIL), the CPU sends an interrupt response message to CLIC based on the interrupt type. The interrupt response mechanism functions as follows:

- If the interrupt is a level interrupt, no interrupt response signal is sent. The software needs to clear the external interrupt source in the interrupt service routine (ISR).
- If the interrupt is a vector edge interrupt, the CPU sends an interrupt response signal after it responds to the interrupt. The CLIC clears the interrupt pending bit of the interrupt after receiving the signal.
- If the interrupt is a non-vector edge interrupt, read operations to the MNXTI register in the ISR will generate an ID from the CLIC and the corresponding entry address of the program, which indicates the entry address of the interrupt arbitrated by the CLIC. The CPU performs a jump operation based on the obtained address. If the value read from the MNXTI register is 0, it signifies that there is no valid interrupt request. Upon receiving a valid read operation from the CPU to the MNXTI register, the CLIC clears the corresponding interrupt pending bit based on the identified ID.

10.2 Address Mapping in CLIC

CLIC occupies a 20 KiB memory space, with the address mapping illustrated in Table 10.1, in which, CLICCFG, CLICINFO, and MINTTHRESH registers support only access to word-aligned addresses.

Table 10.1: Address Mapping in CLIC

Address	Name	Type	Initial Value	Description
0xE0800000	CLICCFG	RW	0x1	CLIC configuration register
0xE0800004	CLICINFO	RO	Please refer to <i>Timer Interrupts</i>	CLIC information register
0xE0800008	MINTTHRESH	RW	0x0	Interrupt threshold register
0xE0801000+4*i	CLICINTIP[i]	R or RW	0x0	Pending register of interrupt source i
0xE0801001+4*i	CLICINTIE[i]	RW	0x0	Enable register of interrupt source i
0xE0801002+4*i	CLICINTATTR[i]	RW	0x0	Attribute register of interrupt source i
0xE0801003+4*i	CLICINTCTRL[i]	RW	0x0	CSR of interrupt source i

i represents the interrupt vector ID. For instance, CLICINTIE[3] and CLICINTIE[7] are required to be applied respectively, to enable the corresponding software interrupt (vector ID = 3) and the timer interrupt (vector ID = 7).

10.3 CLIC Registers

10.3.1 CLIC Configuration Register (CLICCFG)

The CLIC registers contain an 8-bit global CLICCFG register, which defines the supported privilege modes for interrupt responding the segmentation of CLICINTCTL[i], and whether hardware vector interrupts are supported. Fig. 10.1 shows the bit layout and definition of the register.

	7	6	5	4	1	0
	-	nmbits	nbits		nvbits	
Reset	0	0	0		1	

Fig. 10.1: CLICCFG Register

nmbits: number of privilege mode valid bits

This bit indicates the number of valid bits in CLICINTATTR[i].mode. E902 supports handling interrupts only in M-mode. Therefore, this bit is fixed to 0. Regardless of the value in CLICINTATTR[i].mode, it is regarded that the CPU responds to interrupts in M-mode.

nlbits: number of interrupt priority valid bits

This bit indicates the number of bits in CLICINTCTL[i] serves as the interrupt priority. The priority of interrupts generated by the CLIC is fixed to 8 bits. When the number of valid bits is less than 8, the left empty parts are replaced with the digit 1.

nvbits: hardware vector interrupt implementation flag

This bit indicates whether the CLIC supports vector interrupts. This bit is fixed to 1, indicating that the CLIC supports hardware vector interrupts. To enable the hardware vector interrupt mode for an interrupt, the shv bit in the CLICINTATTR register of the interrupt needs to be set to 1. The ISR address of hardware vector interrupts is obtained through two-levels hardware jumping. Initially, after saving the processor context, the CPU fetches data from an address marked as $MTVT + (\text{Interrupt ID} * 4)$. This data is regarded as the ISR entry for that interrupt ID, and the CPU jumps to this address to execute the ISR. For non-hardware vector interrupts, the ISR entry is $MTVEC[31:6] < 6$, and the CPU jumps to this address to handle interrupts.

10.3.2 CLIC Information Register (CLICINFO)

The CLICINFO register is read-only and provides part of CLIC information. Fig. 10.2 shows the corresponding bit layout and definition of the register.

	31	25	24	21	20	13	12	0
Reset	-			CLICCTLBITS		version		num_interrupt

Fig. 10.2: CLICINFO Register

CLICCTLBITS: number of valid bits in CLICINTCTL

This bit indicates the number of valid bits of the priority in the CLICINTCTL register. Implemented valid bits are left-aligned in CLICINTCTL[i].

version: version information

The lower four bits indicate the revised version implemented by the hardware, and the upper four bits indicate the CLIC architecture version information.

num_interrupt: number of interrupt sources

This bit indicates the number of interrupt sources supported by the CLIC hardware with up to 256 interrupt sources supported.

10.3.3 Interrupt Threshold Register (MINTTHRESH)

The MINTTHRESH register defines the priority threshold of pending interrupt requests that can be sent to the CPU pipeline core. The MINTTHRESH register provides an 8-bit field as the corresponding interrupt threshold for each privilege mode. Only pending interrupt requests whose priority is greater than the threshold value defined in the MINTTHRESH register can be sent to the CPU.

The bit layout and definition of the register is illustrated in Fig. 10.3 .

	31	24	23	0
	mth		-	
Reset	-		0	

Fig. 10.3: MINTTHRESH Register

mth: M-mode threshold

This bit indicates the priority threshold of interrupt in M-mode. E902 supports interrupt responding only in M-mode, so only one threshold field is available.

10.3.4 Interrupt Pending Register (CLICINTIP)

When the least significant bit of the CLICINTIP register is set to 1, it indicates that there is an interrupt of the related interrupt source pending to be processed. Fig. 10.4 shows the bit layout and definition of the register.

	7	1	0
	-		IP
Reset	0		0

Fig. 10.4: CLICINTIP Register

IP-the interrupt pending bit

This bit indicates whether there is pending interrupts of the interrupt source. This bit has different setting and clearing logic for level and edge interrupts.

In level interrupt mode, the CLICINTIP register is read-only. Modifying the value of CLICINTIP requires direct manipulation of the external interrupt source. When the external interrupt source is set to a high level, the IP bit is set to 1. When the external interrupt source is set to a low level, the IP bit is set to 0.

In edge interrupt mode, the CLICINTIP register is readable and writable and can automatically clear the IP bit. When an interrupt is configured in hardware vector mode, the CPU automatically clears its IP bit upon interrupt response. For non-hardware vector mode interrupts, software clears the pending status by performing a CSR access instruction that effectively reads or writes to the MNXTI register, to obtain the interrupt pending state.

Meanwhile, the operations can also clear the pending interrupt and enable the CPU to jump to the ISR for interrupt handling. If the non-hardware vector mode interrupt does not rely on reading or writing the MNXTI register for its handling, but instead the interrupt directly delivers the interrupt information to the CPU pipeline core, the hardware will be unable to clear the interrupt pending status when responding to the interrupt. Consequently, software must intervene to clear this state.

Note: Effective read-write operations on the MNXTI register refer to instances where a write operation is performed on the MNXTI register, accompanied by a read operation that returns a non-zero value. If the CSR instruction merely reads the MNXTI register, it will only return the entry address of any pending interrupt responses without clearing the corresponding Interrupt Pending (IP) bit. This functionality enables querying whether there is an interrupt awaiting service.

10.3.5 Interrupt Enable Register (CLICINTIE)

The least significant bit of this register being set indicates that the corresponding interrupt source is enabled, allowing the CPU to respond to the interrupt when its conditions are met. The bit layout and definition of the register is illustrated in Fig. 10.5 .

	7	1	0
	-		IE
Reset	0		0

Fig. 10.5: CLICINTIE Register

IE: the interrupt enable bit

- 1: The corresponding interrupt is enabled.
- 0: The corresponding interrupt is not enabled.

10.3.6 Interrupt Attribute Register (CLICINTATTR)

The CLICINTATTR register is applied to configuring the attributes of different interrupt source, including the CPU privilege modes for interrupt responding, interrupt triggering modes, and judgment of hardware vector interrupts. Fig. 10.6 shows the bit layout and definition of the register.

	7	6	5	3	2	1	0
	mode		-		trig		shv
Reset	2'b11		0		0		0

Fig. 10.6: CLICINTATTR Register

mode: the privilege mode for interrupts

This field is applied to configuring the privilege mode for responding to an interrupt. E902 can respond to interrupts only in M-mode. Therefore, this field is fixed to 2'b11, indicating the interrupt in M-mode.

trig: interrupt triggering mode

This field is applied to distinguish between a pulse interrupt and level interrupt.

When trig[0] is set to 0, it indicates a level interrupt.

When trig[0] is set to 1, if trig[1] is set to 0, the interrupt is a rising edge interrupt; while if trig[1] is 1, the interrupt is a falling edge interrupt.

shv: the vector interrupt enable bit

This bit indicates whether an interrupt is a hardware vector interrupt.

10.3.7 Interrupt CSR Register (CLICINTCTL)

CLICINTCTL register indicates the priority of each interrupt source that participates in arbitration. Based on the nlbits field in the CLICCFG register, the register generates a interrupt priority sent to CPU, which is fixed to 8 bits. Fig. 10.7 shows the bit layout and definition of the register.

	7	8-CLICINTCTLBITS	7-CLICINTCTLBITS	0
	int_ctl		hardware tied to 1	
Reset	0		{(CLICINTCTLBITS){1b'1}}	

Fig. 10.7: CLICINTCTL Register**int_ctl: the priority of an interrupt that participates in arbitration**

This field is valid for the CLICINTCTLBITS bits (hardware-configurable 2-5). All the values in the valid bit field serve as the priority of interrupts for arbitration in CLIC. However, only upper bits of the nlbits field in the CLICCFG register transferred to the CPU pipeline core indicates the arbitrated interrupt priority. This priority is applied to updating the MIL field in MINTSTATUS register.

Some samples of CLICCFG.nlbits and the corresponding final interrupt priority code sent from CLIC to CPU is illustrated in Table 10.2 .

When nlbits is greater than CLICINTCTLBITS, it is regarded that nlbits is equal to CLICINTCTLBITS.

Table 10.2: Samples of nlbits and Interrupt Priority Code Saved in CPU

nlbits	CLICINTCTL Code	Configurable Interrupt Priority
0	xxxxxxxx	255

Continued on next page

Table 10.2 – continued from previous page

1	lxxxxxxx	127, 255
2	llxxxxxx	63, 127, 191, 255
3	lllxxxxx	31, 63, 95, 127, 159, 191, 223, 255
4	llllxxxx	15, 31, 47, 63, 79, 95, 111, 127, 143, 159, 175, 191, 207, 223, 239, 255

Note: In "CLICINTCTL code" column, the letter "I" indicates the valid upper bits of the interrupt priority sent from the CLIC to CPU. For an 8-bit priority, the remaining lower bits (bits indicated by "x" in the code) are set to 1.

Sample illustrating of the division between CLICCFG.nlbits and CLICINTCTL is described in Table 10.3 .

Table 10.3: Samples of the Division between nlbits and CLICINTCTL

CLICINTCTLBITS	nlbits	Code	Configurable Interrupt Priority
2	2	llxxxxxx	63, 127, 191, 255
2	1	lpxxxxxx	127, 255
2	0	ppxxxxxx	255
3	2	llpxxxxx	63, 127, 191, 255
4	2	llppxxxx	63, 127, 191, 255
5	1	lppppxxx	127, 255

Note: In "code" column, the letter "I" indicates the valid upper bits of the interrupt priority sent from the CLIC to CPU. The combination of letter "I" and letter "p" indicates the bits that participate in interrupt priority arbitration in the CLIC; The letter "x" indicates the actual bits in the CLICINTCTL which are hardware-fixed to 1.

11.1 Overview

The debug interface provides an interaction channel between software and the CPU. Users can obtain information through the debug interface, including the information of registers and memory contents in CPU, as well as other on-chip device information. Besides, operations such as program downloading can also be carried out through the debugging interface.

E902 is compatible with the standard of RISC-V External Debug Support Version 0.13.2 protocol. The external debugging interface supports two configurations: a 5-wire JTAG interface (conforming to the standard JTAG5 protocol) and a 2-wire CJTAG interface.

The debug interface provides the following features:

- Supports both 5-wire JTAG and 2-wire CJTAG configurations;
- Supports both synchronous and asynchronous debugging, ensuring that CPU enters debug mode even under extreme adverse conditions;
- Supports software breakpoints;
- Supports setting multiple memory breakpoints;
- Supports inspection and modification of CPU register values;
- Supports inspection and alteration of memory values;
- Supports single-step or multi-step instruction execution;
- Supports rapid program downloading;

- Supports entering debug mode following CPU reset.

Debug of E902 is jointly completed by the debug software, debug proxy, debugger, and debug interface.

Fig. 11.1 shows the location of the debug interface in the CPU debug environment. The debug software is connected to the debug proxy over the network. The debug proxy is connected to the debugger through a USB interface. The debugger communicates with the CPU debug interface in CJTAG or JTAG mode.

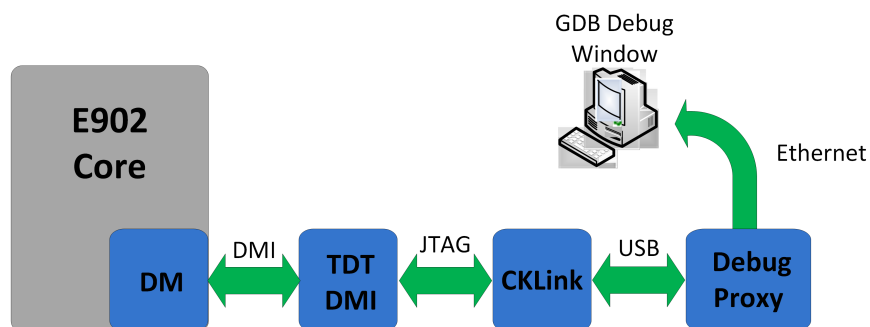


Fig. 11.1: Location of the Debug Interface in the CPU Debug Environment

11.2 DM Register

The registers implemented in E902 DM are illustrated in Table 11.1 . Besides, E902 extends and implements part of the DM registers on the DMI Address encoding domain.

Table 11.1: Mapping and Description of DM Registers

Address	Registers	Description
0x4	DATA0	Abstract DATA0
0x10	DMCONTROL	DM Control Register
0x11	DMSTATUS	DM Status Register
0x15	HAWINDOW	Hart Array Window
0x16	ABSTRACTCS	Abstract Control and Status Register (CSR)
0x17	COMMAND	Abstract Command Register
0x18	ABSTRACTAUTO	Abstract Command Autoexec Register
0x1D	NEXTDM	Next DM Base Address Register
0x20-0x2F	PROGBUF0-1F	Programmable Buffers 0-15
0x32	DMCS2	DM CSR Register 2
0x38	SBCS	System Bus Access (SBA) CSR
0x39	SBADDRESS0	SBA Address 31:0
0x3C	SBDATA0	SBA Data 31:0
0x40	HALTSUM0	HALT Summary Register 0

Continued on next page

Table 11.1 – continued from previous page

Address	Registers	Description
XuanTie Extended Registers		
0x1F	ITR	Instruction Transfer Register
0x70	CUSTOMCS	Custom CSR Register
0x71	CUSTOMCMD	Custom Command Register
0x72-0x79	CUSTOMBUF0-7	Custom Buffers 0-7
0x7F	COMPID	Component ID Register

Note: The DMCS2 constitutes a feature of the new version of *RISC-V Debug (Version 1.0.0-STABLE)*.

Instruction Transfer Register (ITR)

Compared to the Program Buffer, instructions written to the Instruction Transfer Register (ITR) are directly sent to the Load-Store Unit (LSU) for execution, bypassing address fetching and other operations, thereby offering enhanced robustness.

Extended Control and Status Register (CUSTOMCS)

It describes implementations and extensions of the abstract command in XuanTie E902. The CUSTOMCS registers are illustrated in the following table:

Table 11.2: DM CUSTOMCS Registers

Bitfield	Registers	Description
31:29	custcmderr	Error states when executing commands by CUSTOMCMD: 0: No Error 1: Command Not Supported
28:25	cusbufcnt	The number of implemented Buffer
24:28	-	-
17	cuscmdbusy	Status when executing commands by CUSTOMCMD: 0: No command is currently being processed. 1: Command is actively being carried out.
16	-	-
15:0	coredbginfo	Status of kernel debugging resources supported by E902.

Extended Command Register (CUSTOMCMD)

This register is applied to executing extended commands. If an unsupported command is input, the custcmderr bit within the CUSTOMCS register is set to 1.

Table 11.3: DM CUSTOMCMD Registers

Bitfield	Registers	Description
31:24	type	Specifies the types of CUSTOM commands, with the following supported: 0: Initiates an asynchronous debug request to the currently selected core. 1: Facilitates internal register movement within the CPU. 2: Program Counter (PC) Sampling. After command execution, the value of the next PC following the most recent jump instruction executed by the CPU is copied into the DATA Register. Others: Reserved
23:0	-	-

Extended Component ID Register (COMPID)

This register signifies the content and version details of the current DM implementation.

11.3 Resource Configuration

To facilitate user selection, E902 offers three debugging resource configuration options:

- Minimum Configuration: 2 Program Buffers implementing implicit EBREAK; 1 hardware breakpoint.
- Typical Configuration: 2 Program Buffers implementing implicit EBREAK; 4 hardware breakpoints.
- Maximum Configuration: 2 Program Buffers implementing implicit EBREAK, 8 hardware breakpoints, 2 interrupt exception breakpoints.

11.4 Synchronization Debug and Asynchronous Debug

11.4.1 Synchronization Debug

The debugger enters debug mode by writing 1 to the haltreq field in the dmcontrol register.

11.4.2 Asynchronous Debug

When the debugger writes 0 to the type field in the custom command register (belonging to the custom register of XuanTie debug system) within DM module, it indicates the debugger sends an asynchronous debug request to the target core. Then DM will send an asserted asynchronous debug request signal into the core. Unlike synchronous debugging, when the in-core RTU module receives an asynchronous debug request, it immediately enters debug mode. This feature is an extension of the XuanTie CPU debugging system. After the debugger initiates a synchronous debugging request, it will wait for the current

instruction to retire before entering the debugging mode. When the CPU is stuck (the current instruction has not been able to retire), it will not be able to respond to the synchronous debugging request, and the debugger will wait for a few seconds and find that the synchronous debugging request can not be responded to, then it will automatically initiate asynchronous debugging request to try to make the CPU enter the debugging mode even when the CPU is stuck. For the details of the registers related to the above debugging, please refer to the RISC-V official document [RISC-V External Debug Support V0.13.2](#) .

Power Consumption Management

E902 implements the wait for interrupt (WFI) instruction of RV32E to enable the CPU to switch from the normal operation mode to the low-power mode. In low-power mode, the gating clock management unit of E902 disables most register clocks. While the clock to the logic section associated with CPU wake-up feature remains active. WFI instruction can be executed normally only when E902 is in Machine Mode (M-mode). If it is executed in User Mode (U-mode), an illegal instruction exception occurs.

In low-power mode, E902 does not initiate data transfer requests to the bus with internal pipelines suspended.

12.1 Low-Power Mode

E902 extends and implements the LPMD field in the MEXSTATUS register, to indicate the low-power mode (deep sleep or light sleep) that the CPU enters through the WFI instruction. The mode is indicated to the SoC by the top-layer output signal `sysio_pad_lpmd_b[1:0]`. For specific information about register definition, please refer to *Extended Exception Status Register (MEXSTATUS)*.

Even though E902 implements the deep sleep and light sleep modes, the CPU core handles these two sleep modes in the same way. SoC design personnel can determine whether to implement different low-power policies based on the indication signal at the top layer of the CPU.

12.2 Low-Power Wake-Up

The low-power wake-up in E902 supports the following request types:

- Debug requests
- Non-Maskable Interrupt (NMI) requests
- Interrupt requests
- External events

The E902 extension implements the WFE (Wait For Event) field within the MEXSTATUS register to indicate the type of wake-up request for the CPU.

When this field is set to 1, all the above requests can awaken the CPU from the low-power mode.

When this field is set to 0, requests except external events can awaken the CPU from the low-power mode. When an interrupt request is applied for the wake-up, the CPU can be awakened only when the priority of the interrupt is greater than the value of the MIL field in the MINTSTATUS register. The reset value of this field is set to 1.

When a NMI request triggers the wake-up of the CPU, the CPU responds to and handles the NMI request. When a debug request triggers the wake-up of the CPU, the CPU enters the debug mode. When an interrupt request triggers the wake-up of the CPU, the CPU determines whether to respond to the interrupt or execute the instructions following the WFI instruction based on the comparison result between the interrupt priority and the MIL field in the MINTSTATUS register.

The top-layer output signal `sysio_pad_lpm�_b[1:0]` is changed from 2'b00 to 2'b11 after the wake-up.

CHAPTER 13

Program Examples

This chapter mainly describes various program examples, including the Physical Memory Protection (PMP) setting example, the cache setting example, the interrupt enable and initialization example, the General Purpose Register (GPR) initialization example, the stack pointer initialization example, and the example for setting the entry addresses of exception and interrupt service routine (ISR).

13.1 PMP Setting Example

```
/* Set the address mode, size, and start address for region 0 with the address mode as N
↳NAPOT and 128-KiB size.
/* Region 0 controls address range [0x0, 0x00020000).

li t1,0x3fff
csw pmpaddr0,t1
```

Note: The value of the PMPADDR register is calculated based on the address size and start address.

```
/* Set the address mode, size, and start address for region 1 with the address mode as N
↳NAPOT and 128-byte size .
/* Region 1 controls address range [0x01000000, 0x01000080)
```

(continues on next page)

(continued from previous page)

```
li t1,0x40000f
csrw pmpaddr1,t1
```

Note: li t1, 0x400000 can achieve the same effect.

```
/* Set the address mode, size, and start address for region 2 with the address mode as
↳TOR.
/* Region 2 controls address range [0x01000000, 0x01100000). Access requests to
↳[0x01000000, 0x01000080) will hit regions 1 and 2 simultaneously. In this scenario,
↳region 1 has a higher priority than region 2, and the access attribute of region 1
↳determines whether the access permission is granted. When the address matching mode of
↳region 1 is changed to OFF, access requests to [0x01000000, 0x01100000) all hit region
↳2. The access attribute of region 2 determines whether the access permission is
↳granted.

li t1,0x440000
csrw pmpaddr2,t1
```

```
/* Set the address mode, size, and start address for region 3 with the address mode as
↳NAPOT and 8-KiB size.
/* Region 3 controls address range [0x01200000, 0x01202000)

li t1,0x4803ff
csrw pmpaddr3,t1
```

```
/* Set attributes of regions 0 to 3.
/* R0x9f for region 0: lock = 1, executable, writable, and readable (address matching
↳mode: NAPOT)
/* 0x1b for region 1: lock = 0, non-executable, writable, and readable (address matching
↳mode: NAPOT)
/* 0x8e for region 2: lock = 1 , executable, writable, unreadable (address matching
↳mode: TOR)
/* 0x1d for region 3: lock = 0 , executable, unwritable, and readable (address matching
↳mode: NAPOT)

li t1, 0x1d8e1b9f
csrw pmpcfg0,t1
```

Note: The value of the pmpcfg register is calculated based on the address attribute.

```
/* Set the address mode, size, and start address for region 4 with the address mode as
↳OFF.
/* No address will hit this entry.

li t1,0x4c0000
csrw pmpaddr4,t1
```

```
/* Set the address mode, size, and start address for region 5 with the address mode as
↳NAPOT, and 16-KiB size.
/* Region 5 controls address range [0x01400000, 0x01404000)

li t1,0x5007ff
csrw pmpaddr5,t1
```

```
/* Set the address mode, size, and start address for region 6 with the address mode as
↳NAPOT, and 32-KiB size.
/* Region 6 controls address range [0x01500000, 0x01508000)

li t1,0x540fff
csrw pmpaddr6,t1
```

```
/* Set the address mode, size, and start address for region 7 with the address mode as
↳NAPOT, and 64-KiB sizes.
/* Region 7 controls address range [0x01600000, 0x01610000)

li t1,0x581fff
csrw pmpaddr7,t1
```

```
/* Sets attributes of regions 4 to 7.
/* 0x83 for region 4: lock = 1, non-executable, writable, and readable (address matching
↳mode: OFF)
/* 0x1e for region 5: lock = 0, executable, writable, and unreadable (address matching
↳mode: NAPOT).
/* 0x9d for region 6: lock = 1, executable, unwritable, and readable (address matching
↳mode: NAPOT).
/* 0x1b for region 7: lock = 0, non-executable, writable, and readable (address matching
↳mode: NAPOT).
```

(continues on next page)

(continued from previous page)

```
li t1,0x1b9d1e83
csrw pmpcfg1,t1

/* The configuration method for regions 8 through 15 is the same as above, each
↳corresponding to the following registers:
/* pmpaddr8, pmpaddr9, pmpaddr10, pmpaddr11, pmpcfg2
/* pmpaddr12, pmpaddr13, pmpaddr14, pmpaddr15, pmpcfg3
```

13.2 Cache Setting Example

E902 supports the extended cache register: Machine Mode (M-mode) hardware configuration register (MHCR), which enables and disables Instruction Cache (I-Cache). For more information about setting cacheable address spaces in the system, please refer to *Memory Model*.

```
// When IE is set to 0, the I-Cache is disabled. When IE is set to 1, the I-Cache is
↳enabled.

li t1, 1 // Enable the I-Cache. Instructions can be cached.
csrrs t3, mhcr, t1

// Set mxstatus.theadisae to 1 and invalidate the I-Cache by the cache instruction.

la t1, 0x400000
csrrs t3, mhcr, t1

//invalid icache

ICACHE.IALL
```

13.3 Example of Interrupt Enable Initialization Setting

After configuring the interrupt controller and interrupt vector table (For specific information, please refer to *Core-Local Interrupt Controller*), it is necessary to enable the interrupt bits. The specific settings are as follows:

```
// Configure the interrupt priority valid bits.
```

(continues on next page)

(continued from previous page)

```

li t1, 0x6 //nlbits = 3
li t2, 0xe0800000 //cliccfig register address
sw t1, 0(t2) // For the maximum bit width of the interrupt priority, please refer to the
↪ CLICCTLBITS field in the CLICINFO register.

// Set the interrupt type.

li t1, 0x7f010100

// [31:24] indicates CLICINTCTL with the priority of 0x7f. [23:16] indicates CLICINTATTR,
↪ so the interrupt is a vector level interrupt.
// indicates CLICINTIE, which enables the interrupt. [7:0] indicates CLICINTIP.

li t2, 0xe0801000 // Configure the base address for interrupt 0.
sw t1, 64(t2) // Configure interrupt 16.

// Enable the global interrupt enable bit mstatus.mie.

li t1, 0x8
csrrs t3, mstatus, x1

```

13.4 GPR Initialization Example

```

// Initialize GPRs x0 to x15.

li x1, 0
li x2, 0
li x3, 0
li x4, 0
li x5, 0
li x6, 0
li x7, 0
li x8, 0
li x9, 0
li x10, 0
li x11, 0
li x12, 0
li x13, 0

```

(continues on next page)

(continued from previous page)

```
li x14, 0
li x15, 0
```

13.5 Initialization Example of Stack Pointer

The setting of the stack pointer is as follows:

```
li x2, 0x01000000 // Set the stack pointer
```

13.6 Example of Setting the Entry Addresses of Exception and ISR

The entry addresses of the exception and ISR are set in the following 2 steps:

Step 1:

Set the base address and mode of the exception vector table and write them to the MTVEC register. In E902, the mode bit is fixed to 2'b11.

Step 2:

Write the entry address of ISR to the exception vector table address corresponding to the exception vector number in step 1.

// if clicintattr[i].shv = 0 clic direct mode (non-vector interrupt mode)

The entry address of exceptions and interrupts is $mtvec[31:6] \ll 6$

//if clicintattr[i].shv = 1 clic vector mode (vector interrupt mode):

The exception entry address is $mtvec[31:6] \ll 6$, and the interrupt entry address is $MEM[mtvt[31:0] + 4 * \text{Interrupt ID}]$.

Example:

```
li t3, (trap_handler) // The trap_handler is 2^6 byte aligned.
csw mtvec, t3 // Initialize the MTVEC register.
li t3, (vector_table) //vector_table 为 is 2^6 byte aligned.
addi t3, t3, 64
csw mtvt, t3 // Initialize the MTVT register.

.align 6
trap_handler:
addi sp, sp, -48
```

(continues on next page)

(continued from previous page)

```

sw t0, 44(sp)
sw t1, 40(sp)
sw t2, 36(sp)
sw t3, 32(sp)
sw t4, 28(sp)
sw t5, 24(sp)
sw t6, 20(sp)
sw ra, 16(sp)
csrr t0, mcause
sw t0, 12(sp)
csrr t1, mepc
sw t1, 8(sp)
csrr t2, mstatus
sw t2, 4(sp)
andi t1, t0, 0xfff // Obtain the cause number.
srli t0, t0, 0x1b
andi t0, t0, 0x10 // Obtain bit[31] of the MCAUSE register to determine whether it is
↳caused by an interrupt.
add t0, t0, t1 // The cause number increases by 16. Leave the space for the lower 16
↳exception vectors.
slli t0, t0, 0x2
la t1, vector_table // Store the entry addresses of the exception and ISR.
add t0, t0, t1
lw t1, 0(t0)
//handle with the exception or non vector int
jalr t1
//recovery the cpu field
lw t2, 4(sp)
csrw mstatus, t2
lw t1, 8(sp)
csrw mepc, t1
lw t0, 12(sp)
csrw mcause, t0
lw ra, 16(sp)
lw t6, 20(sp)
lw t5, 24(sp)
lw t4, 28(sp)
lw t3, 32(sp)
lw t2, 36(sp)
lw t1, 40(sp)

```

(continues on next page)

(continued from previous page)

```

lw t0, 44(sp)
addi sp, sp, 48
mret

.align 6
vector_table:
.long 0x0 //reserved
.long INST_FETCH_ERROR_HANDLER // Function for handling fetch instruction access
↳exception 1.
.long ILLEGAL_INST_ERROR_HANDLER // Function for handling illegal instruction exception
↳2.
.....
.long MACHINE_ECALL_HANDLER // Function for handling Machine Mode (M-mode) environment
↳call exception 11.
.rept 5
.long 0x0
.endr
.rept 3
.long 0x0 // CLINT interrupts 0 to 2 are not implemented.
.endr
.long MSOFT_INT_HANDLER // Function for handling M-mode software interrupt 3
.rept 3
.long 0x0 // CLINT interrupts 4 to 6 are not implemented.
.endr
.long MTIME_INT_HANDLER // Function for handling M-mode timer interrupt 7
.rept 3
.long 0x0 // CLINT interrupts 8 to 10 are not implemented.
.endr
.long EXTERNAL_INT_HANDLER
// External Interrupt 11 via pad_cpu_ext_int_b
.rept 4
.long 0x0 // CLINT interrupts 12 to 15 are not implemented.
.endr
.long CLIC_INT0_HANDLER

// Interrupt 0 accessed through the CLIC, that is, the pad_clic_int_vld[0] signal.
// The value of cause in the MCAUSE register is 16.

.long CLIC_INT1_HANDLER

```

(continues on next page)

(continued from previous page)

```
// Interrupt 1 accessed through the CLIC, that is, the pad_clic_int_vld signal.  
  
// Up to 240 external interrupts can be accessed.
```

In the above initialization process, for the same interrupt, both vector and non-vector ISRs share a common entry point, and the CPU jumps to the entry address to execute the ISR. The distinction lies in that under vector interrupt mode, the CPU hardware automatically fetches the ISR's entry address and jump for execution; conversely, in non-vector mode, this action is emulated by the CPU executing software instructions.

The saving and restoration of the ISR's context can be handled inside each ISR. To facilitate this, the ISR can be annotated with `__attribute__((isr))` during declaration, prompting the compiler to automatically insert code for context saving, restoring, as well as the interrupt return instruction during compilation.

Appendix A Standard Instructions

E902 implements the RV32EMC instruction set architecture, with the following sections detailing each instruction based on distinct instruction sets.

14.1 Appendix A-1 E Instructions

This section describes the RV32E instruction set implemented by E902 in detail. The instructions are sorted in alphabetic order with 32-bit width by default. However, in specific cases, the system assembles some instructions into 16-bit compressed instructions. For specific information about compressed instructions, please refer to *Appendix A-3 C Instructions*.

14.1.1 ADD—The Signed Add Instruction

Syntax:

dd rd, rs1, rs2

Operation:

$rd \leftarrow rs1 + rs2$

Execute Permission:

Machine Mode (M-mode)/ User Mode (U-mode)

Exception:

None

Instruction format:

31	25	24	20	19	15	14	12	11	7	6	0	
0000000			rs2		rs1		000		rd		0110011	

14.1.2 ADDI—The Signed Immediate Add Instruction

Syntax:

addi rd, rs1, imm12

Operation:

$rd \leftarrow rs1 + \text{sign_extend}(\text{imm12})$

Execute Permission:

M-mode/U-mode

Exception:

None

Instruction format:

31	20	19	15	14	12	11	7	6	0		
imm12[11:0]				rs1		000		rd		0010011	

14.1.3 AND—The Bitwise AND Instruction

Syntax:

and rd, rs1, rs2

Operation:

$rd \leftarrow rs1 \& rs2$

Execute Permission:

M-mode/U-mode

Exception:

None

Instruction format:

31	25	24	20	19	15	14	12	11	7	6	0
0000000		rs2		rs1		111		rd		0110011	

14.1.4 ANDI—The Immediate Bitwise AND Instruction

Syntax:

andi rd, rs1, imm12

Operation:

$rd \leftarrow rs1 \& \text{sign_extend}(imm12)$

Execute Permission:

M-mode/U-mode

Exception:

None

Instruction format:

31	20	19	15	14	12	11	7	6	0		
imm12[11:0]				rs1		111		rd		0010011	

14.1.5 AUIPC—The Add Upper Immediate to PC Instruction

Syntax:

auipec rd, imm20

Operation:

$rd \leftarrow \text{current pc} + imm20 \ll 12$

Execute Permission:

M-mode/U-mode

Exception:

None

Instruction format:

31	12	11	7	6	0
imm20[19:0]			rd		0010111

14.1.6 BEQ—The Branch-If-Equal Instruction

Syntax:

```
beq rs1, rs2, label
```

Operation:

```
if (rs1 == rs2)
```

```
    next pc = current pc + sign_extend(imm12 << 1)
```

```
else
```

```
    next pc = current pc + 4
```

Execute Permission:

M-mode/U-mode

Exception:

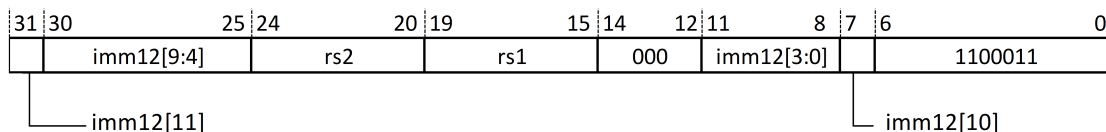
None

Note:

The assembler calculates imm12 based on the label.

The instruction jump range is $\pm 4\text{KiB}$ address space.

Instruction format:



14.1.7 BGE—The Signed Branch-If-Greater-than-or-Equal Instruction

Syntax:

```
bge rs1, rs2, label
```

Operation:

```
if (rs1 >= rs2)
```

```
    next pc = current pc + sign_extend(imm12 << 1)
```

```
else
```

```
    next pc = current pc + 4
```

Execute Permission:

M-mode/U-mode

Exception:

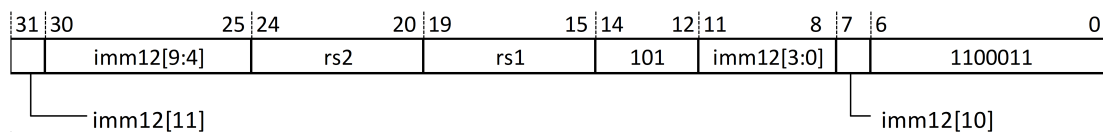
None

Note:

The assembler calculates imm12 based on the label.

The instruction jump range is $\pm 4\text{KiB}$ address space.

Instruction format:



14.1.8 BGEU—The Unsigned Branch-If-Greater-than-or-Equal Instruction

Syntax:

bgeu rs1, rs2, label

Operation:

if (rs1 $\geq$ rs2)

next pc = current pc + sign_extend(imm12<<1)

else

next pc = current pc + 4

Execute Permission:

M-mode/U-mode

Exception:

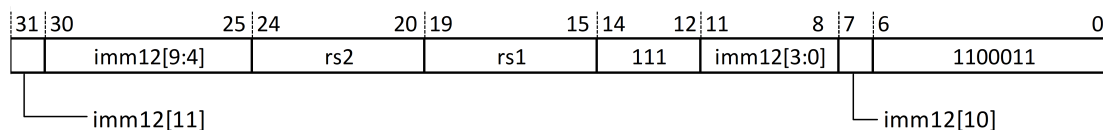
None

Note:

The assembler calculates imm12 based on the label.

The instruction jump range is $\pm 4\text{KiB}$ address space.

Instruction format:



14.1.9 BLT—The Signed Branch-If-Less-than Instruction

Syntax:

blt rs1, rs2, label

Operation:

if (rs1 < rs2)

next pc = current pc + sign_extend(imm12<<1)

else

next pc = current pc + 4

Execute Permission:

M-mode/U-mode

Exception:

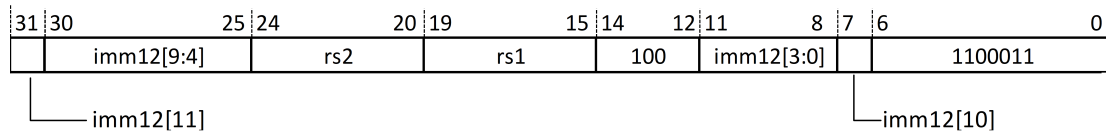
None

Note:

The assembler calculates imm12 based on the label.

The instruction jump range is $\pm 4\text{KiB}$ address space.

Instruction format:



14.1.10 BLTU—The Unsigned Branch-if-Less-than Instruction

Syntax:

bltu rs1, rs2, label

Operation:

if (rs1 < rs2)

next pc = current pc + sign_extend(imm12<<1)

else

next pc = current pc + 4

Execute Permission:

M-mode/U-mode

Exception:

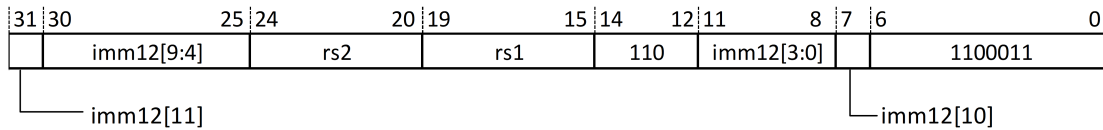
None

Note:

The assembler calculates imm12 based on the label.

The instruction jump range is $\pm 4\text{KiB}$ address space.

Instruction format:



14.1.11 BNE—The Branch-if-not-Equal Instruction

Syntax:

bne rs1, rs2, label

Operation:

if (rs1 $\neq$ rs2)

next pc = current pc + sign_extend(imm12<<1)

else

next pc = current pc + 4

Execute Permission:

M-mode/U-mode

Exception:

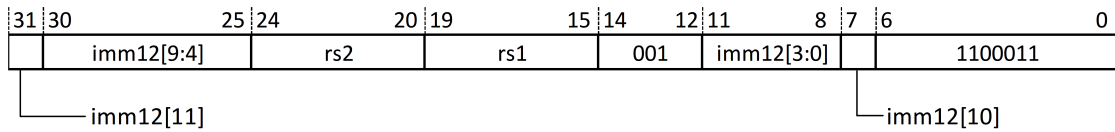
None

Note:

The assembler calculates imm12 based on the label.

The instruction jump range is $\pm 4\text{KiB}$ address space.

Instruction format:



14.1.12 CSRRC——The CSR Read/Clear Instruction

Syntax:

csrrc rd, csr, rs1

Operation:

$rd \leftarrow csr$

$csr \leftarrow csr \& (\sim rs1)$

Execute Permission:

M-mode/U-mode

Exception:

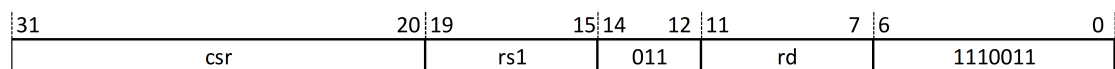
The illegal instruction exception

Note:

Access to CSRs varies depending on the privilege level, with specific details provided in the CSR chapter.

When rs1 is set to x0, the instruction does not perform a write operation or cause any exceptions related to write behavior.

Instruction format:



14.1.13 CSRRCI——The CSR Read/Clear Immediate Instruction

Syntax:

csrrci rd, csr, imm5

Operation:

$rd \leftarrow csr$

$csr \leftarrow csr \& \sim zero_extend(imm5)$

Execute Permission:

M-mode/U-mode

Exception:

The illegal instruction exception

Note:

Access to CSRs varies depending on the privilege level, with specific details provided in the CSR section.

When rs1 is set to x0, the instruction does not perform a write operation or cause any exceptions related to write behavior.

Instruction format:

31	20	19	15	14	12	11	7	6	0	
csr			imm5		111		rd		1110011	

14.1.14 CSRRS—the CSR Read/Set Instruction

Syntax:

csrrs rd, csr, rs1

Operation:

$rd \leftarrow csr$

$csr \leftarrow csr \mid rs1$

Execute Permission:

M-mode/U-mode

Exception:

The illegal instruction exception

Note:

Access to CSRs varies depending on the privilege level, with specific details provided in the CSR section.

When rs1 is set to x0, the instruction does not perform a write operation or cause any exceptions related to write behavior.

Instruction format:

31	20	19	15	14	12	11	7	6	0	
csr			rs1		010		rd		1110011	

14.1.15 CSRRSI—The CSR Read/Set Immediate Instruction

Syntax:

```
csrrsi rd, csr, imm5
```

Operation:

$$rd \leftarrow csr$$

$$csr \leftarrow csr \mid \text{zero_extend}(\text{imm5})$$

Execute Permission:

M-mode/U-mode

Exception:

The illegal instruction exception

Note:

Access to CSRs varies depending on the privilege level, with specific details provided in the CSR section.

When rs1 is set to x0, the instruction does not perform a write operation or cause any exceptions related to write behavior.

Instruction format:

31	20	19	15	14	12	11	7	6	0
csr				imm5		110	rd		1110011

14.1.16 CSRRW—The CSR Read/Write Instruction

Syntax:

```
csrrw rd, csr, rs1
```

Operation:

$$rd \leftarrow csr$$

$$csr \leftarrow rs1$$

Execute Permission:

M-mode/U-mode

Exception:

The illegal instruction exception

Note:

Access to CSRs varies depending on the privilege level, with specific details provided in the CSR section.

Instruction format:

31	20	19	15	14	12	11	7	6	0
csr				rs1		001	rd		1110011

14.1.17 CSRRWI—The CSR Read/Write Immediate Instruction

Syntax:

csrrwi rd, csr, imm5

Operation:

$rd \leftarrow csr$

$csr[4:0] \leftarrow imm5$

$csr[63:5] \leftarrow csr[63:5]$

Execute Permission:

M-mode/U-mode

Exception:

The illegal instruction exception

Note:

Access to CSRs varies depending on the privilege level, with specific details provided in the CSR section.

Instruction format:

31	20	19	15	14	12	11	7	6	0
csr				imm5		101	rd		1110011

14.1.18 EBREAK—The Breakpoint Instruction

Syntax:

ebreak

Operation:

Generates breakpoint exceptions or enters the debug mode.

Execute Permission:

M-mode/U-mode

Exception:

The breakpoint exception

Instruction format:

31	20	19	15	14	12	11	7	6	0		
000000000001				00000		000		00000		1110011	

14.1.19 ECALL—The Environment Call Instruction**Syntax:**

ecall

Operation:

Generates environment exceptions.

Execute Permission:

M-mode/U-mode

Exception:

U-mode and M-mode environment call exceptions

Instruction format:

31	20	19	15	14	12	11	7	6	0		
000000000000				00000		000		00000		1110011	

14.1.20 FENCE—The Memory Synchronization Instruction**Syntax:**

fence iorw, iorw

Operation:

Ensures that all memory or device read/write instructions before this instruction are observed earlier than those after this instruction.

Execute Permission:

M-mode/U-mode

Exception:

None

Note:

pi = 1, so = 1, and the instruction syntax is fence i,o, and so forth.

Instruction format:

31	28	27	26	25	24	23	22	21	20	19	15	14	12	11	7	6	0
0000	pi	po	pr	pw	si	so	sr	sw	00000		000		00000				0001111

14.1.21 FENCE.I—The Instruction Stream Synchronization Instruction

Syntax:

fence.i

Operation:

Clears the Instruction Cache (I-Cache) to ensure that all the data access results before this instruction can be accessed by the instruction's subsequent fetch operation.

Execute Permission:

M-mode/U-mode

Exception:

None

Instruction format:

31	28	27	24	23	20	19	15	14	12	11	7	6	0
0000		0000		0000		00000		001		00000			0001111

14.1.22 JAL—The Instruction for Directly Jumping to a Subroutine

Syntax:

jal rd, label

Operation:

next pc $\leftarrow$ current pc + sign_extend(imm20<<1)

rd $\leftarrow$ current pc + 4

Execute Permission:

M-mode/U-mode

Exception:

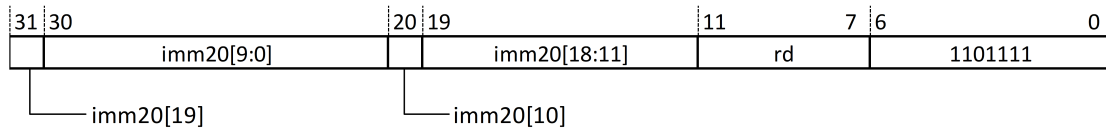
None

Note:

The assembler calculates imm20 based on the label.

The jump range of the instruction is ± 1 MiB address space.

Instruction format:



14.1.23 JALR—The Jump and Link Register Instruction

Syntax:

jalr rd, rs1, imm12

Operation:

$\text{next pc} \leftarrow (\text{rs1} + \text{sign_extend}(\text{imm12})) \& 32'\text{hfffffffe}$

$\text{rd} \leftarrow \text{current pc} + 4$

Execute Permission:

M-mode/U-mode

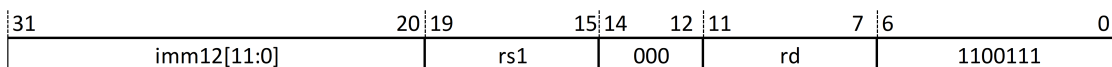
Exception:

None

Note:

The jump range of the instruction is the entire 4 GiB address space.

Instruction format:



14.1.24 LB—The Signed Extended Byte Load Instruction

Syntax:

lb rd, imm12(rs1)

Operation:

$\text{address} \leftarrow \text{rs1} + \text{sign_extend}(\text{imm12})$

$\text{rd} \leftarrow \text{sign_extend}(\text{mem}[(\text{address}+7):\text{address}])$

Execute Permission:

M-mode/U-mode

Exception:

Unaligned access exceptions and access error exceptions on load instructions.

Instruction format:

31	20	19	15	14	12	11	7	6	0
imm12[11:0]				rs1	000	rd	0000011		

14.1.25 LBU—The unsigned Extended Byte Load Instruction**Syntax:**

lbu rd, imm12(rs1)

Operation: $address \leftarrow rs1 + sign_extend(imm12)$ $rd \leftarrow zero_extend(mem[(address+7):address])$ **Execute Permission:**

M-mode/U-mode

Exception:

Unaligned access exceptions and access error exceptions on load instructions.

Instruction format:

31	20	19	15	14	12	11	7	6	0
imm12[11:0]				rs1	100	rd	0000011		

14.1.26 LH—The Signed Extended Halfword Load Instruction**Syntax:**

lh rd, imm12(rs1)

Operation: $address \leftarrow rs1 + sign_extend(imm12)$ $rd \leftarrow sign_extend(mem[(address+15):address])$ **Execute Permission:**

M-mode/U-mode

Exception:

Unaligned access exceptions and access error exceptions on load instructions.

Instruction format:

31	20	19	15	14	12	11	7	6	0	
imm12[11:0]			rs1		001		rd		0000011	

14.1.27 LHU—The Unsigned Extended Halfword Load Instruction**Syntax:**

lhu rd, imm12(rs1)

Operation:

address $\leftarrow$ rs1 + sign_extend(imm12)

rd $\leftarrow$ zero_extend(mem[(address+15):address])

Execute Permission:

M-mode/U-mode

Exception:

Unaligned access exceptions and access error exceptions on load instructions.

Instruction format:

31	20	19	15	14	12	11	7	6	0		
imm12[11:0]				rs1		101		rd		0000011	

14.1.28 LUI—The Upper Immediate Load Instruction**Syntax:**

lui rd, imm20

Operation:

rd $\leftarrow$ imm20 $\ll$ 12

Execute Permission:

M-mode/U-mode

Exception:

None

Instruction format:

31	12	11	7	6	0
imm20[19:0]					
			rd		0110111

14.1.29 LW——The Word Load Instruction

Syntax:

lw rd, imm12(rs1)

Operation:

$address \leftarrow rs1 + sign_extend(imm12)$

$rd \leftarrow mem[(address+31):address]$

Execute Permission:

M-mode/U-mode

Exception:

Unaligned access exceptions and access error exceptions on load instructions.

Instruction format:

31	20	19	15	14	12	11	7	6	0
imm12[11:0]				rs1		010	rd		0000011

14.1.30 MRET——The Instruction to Return from Machine Exception

Syntax:

mret

Operation:

$next\ pc \leftarrow mepc$

$mstatus.mie \leftarrow mstatus.mpie$

$mstatus.mpie \leftarrow 1$

Execute Permission:

M-mode

Exception:

The illegal instruction exception

Instruction format:

31	25	24	20	19	15	14	12	11	7	6	0
0011000	00010	00000	000	00000	1110011						

14.1.31 OR——Bitwise OR Instruction

Syntax:

or rd, rs1, rs2

Operation:

$rd \leftarrow rs1 \mid rs2$

Execute Permission:

M-mode/U-mode

Exception:

None

Instruction format:

31	25	24	20	19	15	14	12	11	7	6	0
0000000	rs2	rs1	110	rd	0110011						

14.1.32 ORI——The Immediate Bitwise OR Instruction

Syntax:

ori rd, rs1, imm12

Operation:

$rd \leftarrow rs1 \mid \text{sign_extend}(\text{imm12})$

Execute Permission:

M-mode/U-mode

Exception:

None

Instruction format:

31	20	19	15	14	12	11	7	6	0	
imm12[11:0]			rs1		110		rd		0010011	

14.1.33 SB——The Byte Store Instruction

Syntax:

sb rs2, imm12(rs1)

Operation:

address $\leftarrow$ rs1 + sign_extend(imm12)

mem[(address+7):address] $\leftarrow$ rs2[7:0]

Execute Permission:

M-mode/U-mode

Exception:

Unaligned access exceptions and access error exceptions on load instructions.

Instruction format:

31	25	24	20	19	15	14	12	11	7	6	0
imm12[11:5]				rs2		rs1		000	imm12[4:0]		0100011

14.1.34 SH——The Halfword Store Instruction

Syntax:

sh rs2, imm12(rs1)

Operation:

address $\leftarrow$ rs1 + sign_extend(imm12)

mem[(address+15):address] $\leftarrow$ rs2[15:0]

Execute Permission:

M-mode/U-mode

Exception:

Unaligned access exceptions and access error exceptions on load instructions.

Instruction format:

31	25	24	20	19	15	14	12	11	7	6	0
imm12[11:5]				rs2		rs1		001	imm12[4:0]		0100011

14.1.35 SLL—The Logical Left Shift instruction

Syntax:

sll rd, rs1, rs2

Operation:

$rd \leftarrow rs1 \ll rs2[4:0]$

Execute Permission:

M-mode/U-mode

Exception:

None

Instruction format:

31	25	24	20	19	15	14	12	11	7	6	0
0000000		rs2		rs1		001		rd		0110011	

14.1.36 SLLI—The Immediate Logical Left Shift Instruction

Syntax:

slli rd, rs1, shamt5

Operation:

$rd \leftarrow rs1 \ll shamt5$

Execute Permission:

M-mode/U-mode

Exception:

None

Instruction format:

31	26	25	20	19	15	14	12	11	7	6	0
000000		shamt6		rs1		001		rd		0010011	

14.1.37 SLT—The Signed Set-If-Less-than Instruction

Syntax:

slt rd, rs1, rs2

Operation:

if ($rs1 < rs2$)

$rd \leftarrow 1$

else

$rd \leftarrow 0$

Execute Permission:

M-mode/U-mode

Exception:

None

Instruction format:

31	25	24	20	19	15	14	12	11	7	6	0
0000000		rs2		rs1		010		rd		0110011	

14.1.38 SLTI—The Signed Set-If-less-than-Immediate Instruction**Syntax:**

slti rd, rs1, imm12

Operation:

if ($rs1 <_{\text{sign_extend}}(\text{imm12})$)

$rd \leftarrow 1$

else

$rd \leftarrow 0$

Execute Permission:

M-mode/U-mode

Exception:

None

Instruction format:

31	20	19	15	14	12	11	7	6	0		
imm12[11:0]				rs1		010		rd		0010011	

14.1.39 SLTIU—The Unsigned Set-If-less-than-Immediate Instruction

Syntax:

```
sltiu rd, rs1, imm12
```

Operation:

```
if (rs1 < sign_extend(imm12))
```

```
    rd ← 1
```

```
else
```

```
    rd ← 0
```

Execute Permission:

M-mode/U-mode

Exception:

None

Instruction format:

31	20	19	15	14	12	11	7	6	0
imm12[11:0]				rs1		011	rd		0010011

14.1.40 SLTU—The Unsigned Set-If-less-than Instruction

Syntax:

```
sltu rd, rs1, rs2
```

Operation:

```
if (rs1 < rs2)
```

```
    rd ← 1
```

```
else
```

```
    rd ← 0
```

Execute Permission:

M-mode/U-mode

Exception:

None

Instruction format:

31	25	24	20	19	15	14	12	11	7	6	0
0000000		rs2		rs1		011		rd		0110011	

14.1.41 SRA—The Arithmetic Right Shift Instruction

Syntax:

sra rd, rs1, rs2

Operation:

$rd \leftarrow rs1 \ggg rs2[4:0]$

Execute Permission:

M-mode/U-mode

Exception:

None

Instruction format:

31	25	24	20	19	15	14	12	11	7	6	0
0100000		rs2		rs1		101		rd		0110011	

14.1.42 SRAI—The Immediate Arithmetic Right Shift Instruction

Syntax:

srai rd, rs1, shamt5

Operation:

$rd \leftarrow rs1 \ggg shamt5$

Execute Permission:

M-mode/U-mode

Exception:

None

Instruction format:

31	26	25	20	19	15	14	12	11	7	6	0
010000		shamt6		rs1		101		rd		0010011	

14.1.43 SRL—The Logical Right Shift Instruction

Syntax:

srl rd, rs1, rs2

Operation:

$rd \leftarrow rs1 \gg rs2[4:0]$

Execute Permission:

M-mode/U-mode

Exception:

None

Instruction format:

31	25	24	20	19	15	14	12	11	7	6	0
0000000		rs2		rs1		101		rd		0110011	

14.1.44 SRLI—The Immediate Logical Right Shift Instruction

Syntax:

srli rd, rs1, shamt5

Operation:

$rd \leftarrow rs1 \gg shamt5$

Execute Permission:

M-mode/U-mode

Exception:

None

Instruction format:

31	26	25	20	19	15	14	12	11	7	6	0
000000		shamt6		rs1		101		rd		0010011	

14.1.45 SUB—The Signed Subtract Instruction

Syntax:

sub rd, rs1, rs2

Operation:

$$rd \leftarrow rs1 - rs2$$
Execute Permission:

M-mode/U-mode

Exception:

None

Instruction format:

31	25	24	20	19	15	14	12	11	7	6	0
0100000				rs2		rs1		000		rd	
										0110011	

14.1.46 SW—The Word Store Instruction**Syntax:**

sw rs2, imm12(rs1)

Operation:

$$\text{address} \leftarrow \text{rs1} + \text{sign_extend}(\text{imm12})$$

$$\text{mem}[(\text{address}+31):\text{address}] \leftarrow \text{rs2}[31:0]$$
Execute Permission:

M-mode/U-mode

Exception:

Unaligned access exceptions and access error exceptions on load instructions.

Instruction format:

31	25	24	20	19	15	14	12	11	7	6	0
imm12[11:5]				rs2		rs1		010		imm12[4:0]	
										0100011	

14.1.47 WFI—The Instruction for Entering the Low Power Mode**Syntax:**

wfi

Operation:

The processor enters a low-power mode, during which the CPU clock is disabled and most peripheral clocks are also disabled.

Execute Permission:

M-mode/U-mode

Exception:

None

Instruction format:

31	25	24	20	19	15	14	12	11	7	6	0	
0001000			00101		00000		000		00000		1110011	

14.1.48 XOR—The Bitwise XOR Instruction**Syntax:**

xor rd, rs1, rs2

Operation: $rd \leftarrow rs1 \oplus rs2$ **Execute Permission:**

M-mode/U-mode

Exception:

None

Instruction format:

31	25	24	20	19	15	14	12	11	7	6	0
0000000		rs2		rs1		100		rd		0110011	

14.1.49 XORI—The Immediate Bitwise XOR Instruction**Syntax:**

xori rd, rs1, imm12

Operation: $rd \leftarrow rs1 \oplus \text{sign_extend}(\text{imm12})$ **Execute Permission:**

M-mode/U-mode

Exception:

None

Instruction format:

31	20	19	15	14	12	11	7	6	0
imm12[11:0]				rs1		100	rd		0010011

14.2 Appendix A-2 M Instructions

This section describes the RV32M instruction set implemented by E902. The instructions are 32-bit wide and sorted in alphabetic order.

14.2.1 DIV—The Signed Divide Instruction

Syntax:

div rd, rs1, rs2

Operation: $rd \leftarrow rs1 / rs2$ **Execute permission:**

M-mode/U-mode

Exception:

None

Notes:

When the divisor is 0, the division result is 0xffffffff.

When overflow occurs, the division result is 0x80000000.

Instruction format:

31	25	24	20	19	15	14	12	11	7	6	0
0000001				rs2		rs1		100	rd		0110011

14.2.2 DIVU—The Unsigned Divide Instruction

Syntax:

divu rd, rs1, rs2

Operation:

$$rd \leftarrow rs1 / rs2$$
Execute permission:

M-mode/U-mode

Exception:

None

Notes:

When the divisor is 0, the division result is 0xffffffff.

Instruction format:

31	25	24	20	19	15	14	12	11	7	6	0
0000001				rs2				rs1			
								101			
								rd			
								0110011			

14.2.3 MUL—The Signed Multiply Instruction

Syntax:

mul rd, rs1, rs2

Operation:

$$rd \leftarrow (rs1 * rs2)[31:0]$$
Execute permission:

M-mode/U-mode

Exception:

None

Instruction format:

31	25	24	20	19	15	14	12	11	7	6	0
0000001				rs2				rs1			
								000			
								rd			
								0110011			

14.2.4 MULH—The Signed Multiply Upper Bit Extraction Instruction

Syntax:

mulh rd, rs1, rs2

Operation:

$$rd \leftarrow (rs1 * rs2)[63:32]$$

Execute permission:

M-mode/U-mode

Exception:

None

Instruction format:

31	25	24	20	19	15	14	12	11	7	6	0
0000001		rs2		rs1		001		rd		0110011	

14.2.5 MULHSU—The Signed and Unsigned Multiply Upper Bit Extraction Instruction

Syntax:

mulusu rd, rs1, rs2

Operation: $rd \leftarrow (rs1 * rs2)[63:32]$ **Execute permission:**

M-mode/U-mode

Exception:

None

Notes:

rs1: Signed number; rs2: Unsigned number

Instruction format:

31	25	24	20	19	15	14	12	11	7	6	0
0000001		rs2		rs1		010		rd		0110011	

14.2.6 MULHU—The Unsigned Multiply Upper Bit Extraction Instruction

Syntax:

mulhu rd, rs1, rs2

Operation: $rd \leftarrow (rs1 * rs2)[63:32]$ **Execute permission:**

M-mode/U-mode

Exception:

None

Instruction format:

31	25	24	20	19	15	14	12	11	7	6	0
0000001		rs2		rs1		011		rd		0110011	

14.2.7 REM—The Signed Remainder Instruction

Syntax:

rem rd, rs1, rs2

Operation:

$rd \leftarrow rs1 \% rs2$

Execute permission:

M-mode/U-mode

Exception:

None

Notes:

When the divisor is 0, the remainder operation result is the dividend.

When overflow occurs, the remainder operation result is 0x0.

Instruction format:

31	25	24	20	19	15	14	12	11	7	6	0
0000001		rs2		rs1		110		rd		0110011	

14.2.8 REMU—The Unsigned Remainder Instruction

Syntax:

remu rd, rs1, rs2

Operation:

$rd \leftarrow rs1 \% rs2$

Execute permission:

M-mode/U-mode

Exception:

None

Notes:

When the divisor is 0, the remainder operation result is the dividend.

Instruction format:

31	25	24	20	19	15	14	12	11	7	6	0
0000001		rs2		rs1		111		rd		0110011	

14.3 Appendix A-3 C Instructions

This section describes the RVC instructions implemented by E902. And the instructions are 16-bit wide, listed in alphabetic order.

14.3.1 C.ADD——The Signed Add Instruction

Syntax:

c.add rd, rs2

Operation:

$rd \leftarrow rs1 + rs2$

Execute permission:

M-mode/U-mode

Exception:

None

Note:

$rs1 = rd \neq 0$

$rs2 \neq 0$

Instruction format:

15	13	12	11	7	6	2	1	0
100	1	rs1/rd		rs2		10		

14.3.2 C.ADDI—The Signed Immediate Add Instruction

Syntax:

c.addi rd, nzimm6

Operation:

$rd \leftarrow rs1 + \text{sign_extend}(nzimm6)$

Execute permission:

M-mode/U-mode

Exception:

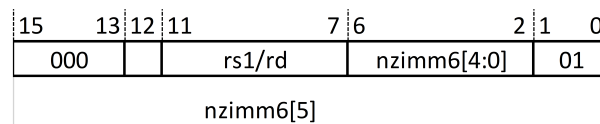
None

Note:

$rs1 = rd \neq 0$

$nzimm6 \neq 0$

Instruction format:



14.3.3 C.ADDI4SPN—The Instruction to Add Immediate Scaled by 4 to Stack Pointer

Syntax:

c.addi4spn rd, sp, nzuimm8<<2

Operation:

$rd \leftarrow sp + \text{zero_extend}(nzuimm8 \ll 2)$

Execute permission:

M-mode/U-mode

Exception:

None

Note:

$nzuimm8 \neq 0$

rd code represents the following registers:

- 000 x8
- 001 x9
- 010 x10
- 011 x11
- 100 x12
- 101 x13
- 110 x14
- 111 x15

Instruction format:

15	13	12	5	4	2	1	0
000	nzuimm8[3:2 7:4 0 1]				rd	00	

14.3.4 C.ADDI16SP——The Instruction to Add Immediate Scaled by 16 to Stack Pointer

Syntax:

c.addi16sp sp, nzuimm6<<4

Operation:

$sp \leftarrow sp + \text{sign_extend}(nzuimm6 \ll 4)$

Execute permission:

M-mode/U-mode

Exception:

None

Instruction format:

15	13	12	11	7	6	2	1	0
011			00010					01
nzimm6[5]					nzimm6[0 2 4:3 1]			

14.3.5 C.AND——The Bitwise AND Instruction

Syntax:

c.and rd, rs2

Operation:

$$rd \leftarrow rs1 \& rs2$$
Execute permission:

M-mode/U-mode

Exception:

None

Note:

$$rs1 = rd$$

rd/rs1, rs2 codes represent the following registers:

- 000: x8
- 001: x9
- 010: x10
- 011: x11
- 100: x12
- 101: x13
- 110: x14
- 111: x15

Instruction format:

15	13	12	11	10	9	7	6	5	4	2	1	0
100	0	11	rs1/rd		11	rs2		01				

14.3.6 C.ANDI—The Immediate Bitwise AND Instruction**Syntax:**

c.andi rd, imm6

Operation:

$$rd \leftarrow rs1 \& \text{sign_extend}(\text{imm6})$$
Execute permission:

M-mode/U-mode

Exception:

None

Note:

rs1 = rd

rd/rs1 code represents the following registers:

- 000: x8
- 001: x9
- 010: x10
- 011: x11
- 100: x12
- 101: x13
- 110: x14
- 111: x15

Instruction format:

15	13	12	11	10	9	7	6	2	1	0
100			10		rs1/rd		imm6[4:0]		01	
imm6[5]										

14.3.7 C.BEQZ—The Branch-If-Equal-to-Zero Instruction

Syntax:

c.beqz rs1, label

Operation:

if (rs1 == 0)

next pc = current pc + imm8<<1;

else

next pc = current pc + 2;

Execute permission:

M-mode/U-mode

Exception:

None

Note:

rs1 code represents the following registers:

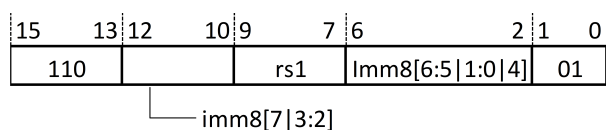
- 000: x8
- 001: x9

- 010: x10
- 011: x11
- 100: x12
- 101: x13
- 110: x14
- 111: x15

The assembler calculates imm8 based on the label.

The instruction jump range is $\pm 128\text{B}$ address space.

Instruction format:



14.3.8 C.BNEZ——The Branch-If-not-Equal-to-Zero Instruction

Syntax:

c.bnez rs1, label

Operation:

if (rs1 $\neq$ 0)

next pc = current pc + imm8 $\ll$ 1;

else

next pc = current pc + 2;

Execute permission:

M-mode/U-mode

Exception:

None

Note:

rs1 code represents the following registers:

- 000: x8
- 001: x9
- 010: x10

- 011: x11
- 100: x12
- 101: x13
- 110: x14
- 111: x15

The assembler calculates imm12 based on the label.

The instruction jump range is $\pm 256\text{B}$ address space.

Instruction format:

15	13	12	10	9	7	6	2	1	0
111		rs1			imm[6:5 1:0 4]			01	
imm8[7 3:2]									

14.3.9 C.EBREAK—The Breakpoint Instruction

Syntax:

c.ebreak

Operation:

Generates breakpoint exceptions or enters the debug mode.

Execute permission:

M-mode/U-mode

Exception:

Breakpoint exceptions

Instruction format:

15	13	12	11	7	6	2	1	0
100		1	00000			00000		10

14.3.10 C.J—The Unconditional Jump Instruction

Syntax:

c.j label

Operation:

next pc $\leftarrow$ current pc + sign_extend(imm<<1);

Execute permission:

M-mode/U-mode

Exception:

None

Note:

The assembler calculates imm11 based on the label.

The instruction jump range is $\pm 2\text{KB}$ address space.

Instruction format:

15	13	12		2	1	0
101	imm11[10 3 8:7 9 5 6 2:0 4]				01	

14.3.11 C.JAL—The Instruction for Jumping to a Subroutine Unconditionally.

Syntax:

c.jal label

Operation:

$\text{next pc} \leftarrow \text{current pc} + \text{sign_extend}(\text{imm} \ll 1);$

$\text{x1} \leftarrow \text{current pc} + 2;$

Execute permission:

M-mode/U-mode

Exception:

None

Note:

The assembler calculates imm11 based on the label.

The instruction jump range is $\pm 2\text{KB}$ address space.

Instruction format:

15	13	12		2	1	0
001	imm[11 4 9:8 10 6 7 3:1 5]				01	

14.3.12 C.JALR—The Jump And Link Register Instruction

Syntax:

c.jalr rs1

Operation:

next pc $\leftarrow$ rs1;

x1 $\leftarrow$ current pc + 2;

Execute permission:

M-mode/U-mode

Exception:

None

Note:

rs1 $\neq$ 0.

The jump range of the instruction is the entire 4 GB address space.

Instruction format:

15	13	12	11	7	6	2	1	0
100	1	rs1				00000	10	

14.3.13 C.JR—The Jump to Register Instruction

Syntax:

c.jr rs1

Operation:

next pc = rs1;

Execute permission:

M-mode/U-mode

Exception:

None

Note:

rs1 $\neq$ 0.

The jump range of the instruction is the entire 4 GB address space.

Instruction format:

15	13	12	11	7	6	2	1	0
100	0	rs1				00000	10	

14.3.14 C.LI—The Immediate Transfer Instruction

Syntax:

```
c.li rd, imm6
```

Operation:

```
rd ← sign_extend(imm6)
```

Execute permission:

M-mode/U-mode

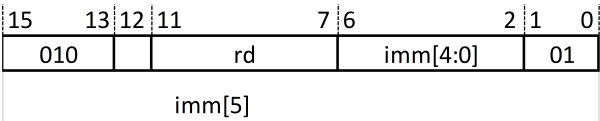
Exception:

None

Note:

```
rd != 0.
```

Instruction format:



14.3.15 C.LUI—The Upper Bit Immediate Transfer Instruction

Syntax:

```
c.lui rd, nzimm6
```

Operation:

```
rd ← sign_extend(nzimm6<<12)
```

Execute permission:

M-mode/U-mode

Exception:

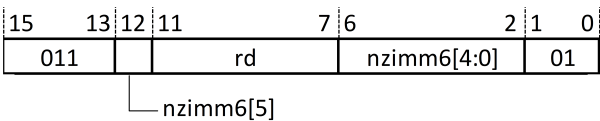
None

Note:

```
rd != 0.
```

```
nzimm6 != 0.
```

Instruction format:



14.3.16 C.LW—The Word Load Instruction

Syntax:

c.lw rd, uimm5<<2(rs1)

Operation:

address $\leftarrow$ rs1 + zero_extend(uimm5<<2)

rd $\leftarrow$ mem[address+31:address]

Execute permission:

M-mode/U-mode

Exception:

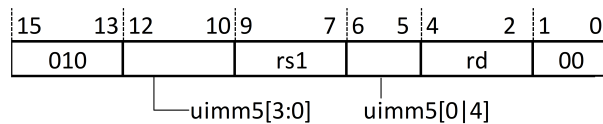
Unaligned access exceptions and access error exceptions on load instructions.

Note:

rs1/rd code represents the following registers:

- 000: x8
- 001: x9
- 010: x10
- 011: x11
- 100: x12
- 101: x13
- 110: x14
- 111: x15

Instruction format:



14.3.17 C.LWSP—The Load Word from Stack Pointer Instruction

Syntax:

c.lwsp rd, uimm6<<2(sp)

Operation:

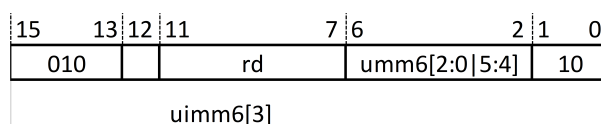
$$\text{address} \leftarrow \text{sp} + \text{zero_extend}(\text{uimm6} \ll 2)$$

$$\text{rd} \leftarrow \text{mem}[\text{address} + 31 : \text{address}]$$
Execute permission:

M-mode/U-mode

Exception:

Unaligned access exceptions and access error exceptions on load instructions.

Note: $\text{rd} \neq 0$ **Instruction format:****14.3.18 C.MV—The Data Transfer Instruction****Syntax:**

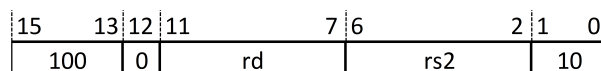
c.mv rd, rs2

Operation: $\text{rd} \leftarrow \text{rs2};$ **Execute permission:**

M-mode/U-mode

Exception:

None

Note: $\text{rs2} \neq 0, \text{rd} \neq 0.$ **Instruction format:****14.3.19 C.NOP—The No-Operation Instruction****Syntax:**

c.nop

Operation:

None operation

Execute permission:

M-mode/U-mode

Exception:

None

Instruction format:

15	13	12	11	7	6	2	1	0
000	0	00000	00000	01				

14.3.20 C.OR——The Bitwise OR Instruction**Syntax:**

c.or rd, rs2

Operation: $rd \leftarrow rs1 \mid rs2$ **Execute permission:**

M-mode/U-mode

Exception:

None

Note:

rs1 = rd

rd/rs1 code represents the following registers:

- 000: x8
- 001: x9
- 010: x10
- 011: x11
- 100: x12
- 101: x13
- 110: x14
- 111: x15

Instruction format:

15	13	12	11	10	9	7	6	5	4	2	1	0
100	0	11	rs1/rd			10	rs2			01		

14.3.21 C.SLLI—The Immediate Logical Left Shift Instruction

Syntax:

c.slli rd, shamt5

Operation:

$rd \leftarrow rs1 \ll shamt5$

Execute permission:

M-mode/U-mode

Exception:

None

Note:

$rs1 == rd$

$rd/rs1 \neq 0, shamt5 \neq 0, shamt[5] = 0$

Instruction format:

15	13	12	11	7	6	2	1	0
000		rs1/rd			nzuimm6[4:0]			10
nzuimm6[5]								

14.3.22 C.SRAI—The Immediate Arithmetic Right Shift Instruction

Syntax:

c.srli rd, shamt5

Operation:

$rd \leftarrow rs1 \ggg shamt6$

Execute permission:

M-mode/U-mode

Exception:

None

Note:

shamt5 != 0, shamt[5] = 0

rs1 == rd

rs1/rd code represents the following registers:

- 000: x8
- 001: x9
- 010: x10
- 011: x11
- 100: x12
- 101: x13
- 110: x14
- 111: x15

Instruction format:

15	13	12	11	10	9	7	6	2	1	0
100			01		rs1/rd		nzuimm6[4:0]		01	
nzuimm6[5]										

14.3.23 C.SRLI—The Immediate Logical Right Shift Instruction

Syntax:

c.srli rd, shamt5

Operation:

rd ← rs1 >> shamt5

Execute permission:

M-mode/U-mode

Exception:

None

Note:

shamt5 != 0, shamt[5] = 0

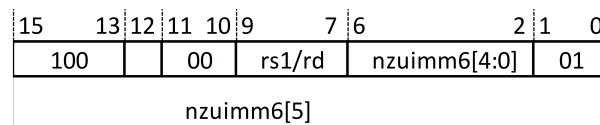
rs1 == rd

rs1/rd code represents the following registers:

- 000: x8
- 001: x9

- 010: x10
- 011: x11
- 100: x12
- 101: x13
- 110: x14
- 111: x15

Instruction format:



14.3.24 C.SW—The Word Store Instruction

Syntax:

c.sw rs2, uimm5<<2(rs1)

Operation:

address $\leftarrow$ rs1 + zero_extend(uimm5<<2)

mem[address+31:address] $\leftarrow$ rs2

Execute permission:

M-mode/U-mode

Exception:

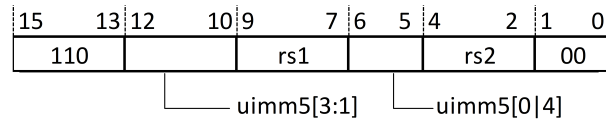
Unaligned access exceptions and access error exceptions on store instructions.

Note:

rs1/rs2 code represents the following registers:

- 000: x8
- 001: x9
- 010: x10
- 011: x11
- 100: x12
- 101: x13
- 110: x14
- 111: x15

Instruction format:



14.3.25 C.SWSP—The Word Stack Store Instruction

Syntax:

c.swsp rs2, uimm6<<2(sp)

Operation:

$\text{address} \leftarrow \text{sp} + \text{zero_extend}(\text{uimm6} \ll 2)$

$\text{mem}[\text{address}+31:\text{address}] \leftarrow \text{rs2}$

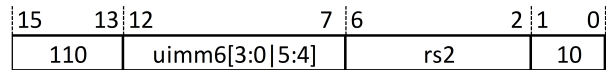
Execute permission:

M-mode/U-mode

Exception:

Unaligned access exceptions and access error exceptions on store instructions.

Instruction format:



14.3.26 C.SUB—The Signed Subtract Instruction

Syntax:

c.sub rd, rs2

Operation:

$\text{rd} \leftarrow \text{rs1} - \text{rs2}$

Execute permission:

M-mode/U-mode

Exception:

None

Note:

rs1 == rd

rs1/rd code represents the following registers:

- 000: x8
- 001: x9
- 010: x10
- 011: x11
- 100: x12
- 101: x13
- 110: x14
- 111: x15

Instruction format:

15	13	12	11	10	9	7	6	5	4	2	1	0
100	0	11	rs1/rd	00	rs2	01						

14.3.27 C.XOR—The Bitwise XOR Instruction

Syntax:

c.xor rd, rs2

Operation:

$rd \leftarrow rs1 \wedge rs2$

Execute permission:

M-mode/U-mode

Exception:

None

Note:

rs1 == rd

rs1/rd code represents the following registers:

- 000: x8
- 001: x9
- 010: x10
- 011: x11
- 100: x12
- 101: x13
- 110: x14

- 111: x15

Instruction format:

15	13	12	11	10	9	7	6	5	4	2	1	0
100	0	11	rs1/rd	01	rs2	01						

14.4 Appendix A-4 Pseudo Instructions

RISC-V implements a series of pseudo instructions, listed in this section are for reference only and sorted in alphabetic order.

Table 14.1: RISC-V Pseudo Instruction

Pseudo Instruction	Basic Instruction	Description
beqz rs, offset	beq rs, x0, offset	Takes the branch if the value in the rs register is zero.
bnez rs, offset	bne rs, x0, offset	Takes the branch if the value in the rs register is not zero.
blez rs, offset	bge x0,rs,offset	Takes the branch if the value in the rs register is less than or equal to zero.
bgez rs, offset	bge rs, x0, offset	Takes the branch if the value in the rs register is greater than or equal to zero.
bltz rs, offset	blt rs, x0, offset	Takes the branch if the value in the rs register is less than zero.
bgtz rs, offset	blt x0, xs, offset	Takes the branch if the value in the rs register is greater than zero.
bgt rs, rt, offset	blt rt, rs, offset	Takes the branch if the value in the rs register is greater than that of the rt register.
ble rs, rt, offset	bge rt, rs, offset	Takes the branch if the value in the rs register is less than or equal to that of the rt register.
bgtu rs, rt, offset	bltu rt, rs, offset	Takes the branch if the value in the rs register is greater than that of the rt register, using unsigned comparison.
bleu rs, rt, offset	bgeu rt, rs, offset	Takes the branch if the value in the rs register is less than or equal to that of the rt register, using unsigned comparison.
call offset	auipc x6, offset[31:12] jalr x1, x6, offset[11:0]	Function jump within a 4-Kilobyte to 4-Gigabyte Address Range.
csrc csr, rs	csrc x0, csr, rs	Clears the corresponding bits in the CSR.
csrci csr, imm	csrci x0, csr, imm	Clears the corresponding bits among the lower six bits in the CSR.

Continued on next page

Table 14.1 – continued from previous page

csrs csr, rs	csrrs x0, csr, rs	Sets the corresponding bits in the CSR.
csrsi csr, imm	csrrsi x0, csr, imm	Set the corresponding bits in the lower 5 bits of the CSR.
csrw csr, rs	csrrw x0, csr, rs	Writes the corresponding bits in the CSR.
csrwi csr, imm	csrrwi x0, csr, imm	Writes the corresponding bits in the lower 5 bits of the CSR.
fence	fence iorw, iorw	The synchronization instruction between memory and device.
j offset	jal x0, offset	The jump directly instruction
jal offset	jal x1, offset	The instruction for directly jumping to a subroutine.
jalr rs	jalr x1, rs, 0	The jump and link register instruction
jr rs	jalr x0, rs, 0	The jump to register instruction
la rd, symbol	auipc rd, symbol[31:12] addi rd, rd, symbol[11:0]	Loads the instruction address
li rd, immediate	Splits into multiple instructions based on the size of the immediate value	Loads an immediate.
l{b h w} rd, symbol, rt	auipc rt, symbol[31:12] l{b h w} rd, symbol[11:0](rt)	Loads the 4 GB address space.
mv rd, rs	addi rd, rs, 0	Transfers data
neg rd, rs	sub rd, x0, rs	Takes negate value from register and stores
nop	addi x0,x0,0	No operation
not rd, rs	xori rd, rs, -1	The bitwise complement register instruction
ret	jalr x0, x1,0	Returns from the subroutine
s{b h w} rd, symbol, rt	auipc rt, symbol[31:12] s{b h w} rd, symbol[11:0](rt)	Stores the 4 GiB address space
seqz rd, rs	sltiu rd, rs, 1	Sets the register value 0 to 1
sgtz rd, rs	slt rd, rs, x0, rs	Sets the if-greater-than-zero register value to 1
sltz rd, rs	slt rd, rs, rs, x0	sets the if-smaller-than-zero register value to 1
snez rd, rs	sltu rd, rs, x0, rs	sets the non-zero register value to 1
tail offset	auipc x6, offset[31:12] jalr x0, x6, offset[11:0]	Not link or jump to the subroutine

Appendix B Xuantie Extended Instructions

Apart from the RV32EMC instruction sets defined in the standard, E902 extends and implements custom cache instruction sets. The following section describes each instruction in details.

If the value of THEADISAEE in the MXSTATUS register is 0, executing the extended instructions in this section causes an illegal instruction exception.

15.1 Appendix B-1 Cache Instructions

Cache instruction subset has implemented the cache operation with 32-bit width for each instruction. The following instructions are listed in alphabetic order.

15.1.1 ICACHE.IALL—The Instruction to Invalidate All Table Entries in the I-Cache

Syntax:

icache.iall

Operation:

Invalidates all table entries in the I-Cache.

Execute Permission:

Machine Mode (M-mode)

Exception:

Illegal instruction exceptions

Note:

mxstatus.theadisaee=0, executing the instruction occurs an illegal instruction exception.

mxstatus.theadisaee=1, executing the instruction in User Mode (U-mode) occurs an illegal instruction exception.

Instruction format:

31	25	24	20	19	15	14	12	11	7	6	0
0000000		10000		00000		000		00000		0001011	

15.1.2 ICACHE.IPA—The Instruction that Invalidates Page Table Entries in the I-Cache that Match the Specified Physical Address

Syntax:

icache.ipa rs1

Operation:

Invalidates the page table entry that matches the specified physical address in the I-Cache of rs1.

Execute Permission:

M mode

Exception:

Illegal instruction exceptions

Note:

mxstatus.theadisaee=0, executing the instruction occurs an illegal instruction exception.

mxstatus.theadisaee=1, executing the instruction in U-mode occurs an illegal instruction exception.

Instruction format:

31	25	24	20	19	15	14	12	11	7	6	0
0000001		11000		rs1		000		00000		0001011	

Appendix C Machine Mode Control and Status Registers

Machine Mode (M-mode) Control and Status Registers (CSRs) are classified by features into M-mode information register group, M-mode exception configuration register group, M-mode exception handling register group, M-mode Physical Memory Protection (PMP) register group, M-mode counter register group, and M-mode extended register group.

16.1 M-mode Information Register Group

16.1.1 M-mode Vendor ID Register (MVENDORID)

The MVENDORID register stores the vendor IDs of Xuantie CPU, allocated by JEDEC, and the register value is currently bound to 0x5B7 in E902.

This register is 64-bit wide and read-only in M-mode. Accesses in non-machine mode and writes in M-mode will cause an illegal instruction exception.

16.1.2 M-mode Architecture ID register (MARCHID)

The MARCHID register stores the architecture IDs of CPU cores. Currently, E902 does not define this register, and the values are all zero.

This register is 32-bit wide and read-only in M-mode. Accesses in non-machine mode and writes in M-mode will cause an illegal instruction exception.

16.1.3 M-mode Hardware Implementation ID Register(MIMPID)

The MIMPID register stores hardware implementation IDs of CPU cores. E902 does not define this register, and the values are all zero.

This register is 32-bit wide and read-only in M-mode. Accesses in non-machine mode and writes in M-mode will cause an illegal instruction exception.

16.1.4 M-mode Hart ID Register (MHARTID)

MHARTID stores the hardware logic core number of CPU cores. Currently, E902 does not define this register, and the values are all zero.

This register is 32-bit wide and read-only in M-mode. Accesses in non-machine mode and writes in M-mode will cause an illegal instruction exception.

16.2 M-mode Exception Configuration Register Group

16.2.1 M-Mode Status Register (MSTATUS)

The MSTATUS register stores status and control information of the CPU in M-mode, including the global interrupt enable bit, exception preserve interrupt enable bit, exception preserve privilege mode bit and so on.

This register is 32-bit wide and is readable and writable in M-mode. Accesses in non-machine mode will cause an illegal instruction exception. The reset value of this register is 0x1800.

[illegible]

Fig. 16.1: M-mode Status Register (MSTATUS)

MIE: M-mode global interrupt enable bit:

When MIE is set to 0, interrupts are invalid.

When MIE is set to 1, interrupts are valid.

The reset value of this bit is 0. This bit is set to 0 when the CPU responds to an exception and is set to the value of MPIE when the CPU exits the exception.

MPiE: M-mode preserve interrupt enable bit

This bit stores the value of the MIE bit before the CPU enters the exception service routine.

The reset value of this bit is 0, and is set to 1 when the CPU exits the exception service routine.

MPP: M-mode preserve privilege mode bit

This bit stores the privilege status before the CPU enters the exception service routine.

When MPP is set to 2'b00, the CPU is in User Mode (U-mode) before it enters the exception service routine.

When MPP is set to 2'b11, the CPU is in Machine Mode (M-mode) before it enters the exception service routine.

The reset value of this bit is 2'b11.

MPRV: store privilege mode bit

When MPRV is set to 0, there are no special requirements on the translation and protection determination of the memory access address.

When MPRV is set to 1, the load and store instructions need to perform address translation and protection determination based on the privilege mode information stored in the MPP bit.

Address translation and protection determination of the fetch instruction are not affected by this bit.

The reset value of this bit is 0.

In Core Local Interrupt Controller (CLIC) mode, the CPU can access the MCAUSE register to obtain the MPP bit and MPIE bit.

16.2.2 M-mode Instruction Set Architecture Register (MISA)

The MISA register stores the information of the instruction set architecture supported by the CPU.

This register is 32-bit wide and is readable and writable in M-mode. Accesses in non-machine mode will cause an illegal instruction exception.

E902 supports the RV32E[M]C instruction set architecture, and the corresponding reset value of the MISA register is 0x40901105. For specific information about the value assignment rules, please refer to the official RISC-V document: *riscv-privileged* (Document Version 20190208-Priv-MSU-Ratified).

Dynamic configuration of the MISA register is not supported in E092. Writes to this register will not take effect.

16.2.3 M-mode Interrupt Enable Register (MIE)

The MIE register controls local masking of different types of CLINT interrupts in M-mode.

This register is 32-bit wide and is readable and writable in M-mode. Accesses in non-machine mode will cause an illegal instruction exception.

31	12	11	10	8	7	6	4	3	2	0
-										
MEIE MTIE MSIE										
Reset	0									
	0	0	0	0	0	0	0	0	0	0

Fig. 16.2: M-mode Interrupt Enable Register (MIE)

MSIE: M-mode software interrupt enable bit

- 0: M-mode software interrupts are not enabled.
- 1: M-mode software interrupts are enabled.

MTIE: M-mode timer interrupt enable bit

- 0: M-mode timer interrupts are not enabled.
- 1: M-mode timer interrupts are enabled.

MEIE: external interrupt enable bit

When MEIE is set to 0, external interrupts are invalid and the CPU cannot be awakened from the low-power mode by external interrupts.

When MEIE is set to 1, external interrupts are valid and the CPU can be awakened from the low-power mode by external interrupts.

When the CPU and the MTVEC register are configured to the CLIC mode (MTVEC.mode[1] is set to 1'b1), this register is set to 0. E902 is configured with a fixed CLIC mode.

16.2.4 M-Mode Exception Vector Base Address Register (MTVEC)

The MTVEC register is applied to configuring the entry address of the exception service routine and addressing mode for interrupt and exception service routine entry points.

This register is 32-bits wide and is readable and writable in M-mode. Accesses in non-machine mode will cause an illegal instruction exception.

31	2	1	0
Base			
			Mode
Reset	0		
	0		

Fig. 16.3: M-Mode Exception Vector Base Address Register (MTVEC)

BASE: vector base address bit

The BASE bit indicates the upper 30 bits of the entry address of the exception service routine. Combining the lower bits of the base address with 2'b00 will obtain the entry address of the exception service routine.

The reset value of this bit is 0.

MODE: vector entry mode bit

When MODE[1:0] is set to 2'b00, the BASE address is applied as the entry address for all exceptions and interrupts.

When MODE[1:0] is set to 2'b01, BASE base address is applied as the entry address for exceptions, and the value of $\text{BASE} \ll 2 + 4 * \text{interrupt ID}$ is applied as the entry address, with the interrupt ID for the vector ID of interrupts.

When configuring the CLIC mode in hardware, two additional settings, 2'b10 and 2'b11, are introduced for the MODE[1:0] bits. Under these two modes, the exception entry addresses are constrained to be 64-byte aligned within the CPU.

When MODE[1:0] is set to 2'b10, this bit is reserved.

When MODE[1:0] is set to 2'b11, the CPU uses MTVEC[31:6]<<6 as the entry address of the exception service routine and jumps to the entry address to execute. In hardware vector interrupt mode, the CPU first takes MTVT + 4*interrupt ID as the entry address of the interrupt service routine (ISR) and jumps to this entry address to execute. In non-hardware vector interrupt mode, the CPU takes MTVEC[31:6]<<6 as the entry address of the ISR and jumps to this entry address to execute. MTVT, the M-mode vector interrupt base address register is available in CLIC mode.

In E902, MODE[1:0] is fixed to 3 for hardware and unconfigurable for software.

16.2.5 M-Mode Counter Enable Register (MCOUNTEREN)

	31		3	2	1	0
	-			IR	TM	CY
Reset	0			0	0	0

Fig. 16.4: M-Mode Counter Enable Register (MCOUNTEREN)

CY: access control bit of the MCYCLe register in U-mode

When CY is set to 0, access to the MCYCLE register in U-mode causes an illegal instruction exception. When CY is 1, the CPU can access the MCYCLE register in U-mode normally.

TM: access control bit of the MTIME registers in U-mode

When TM is set to 0, access to the MTIME registers (MTIMEHI and MTIMELO) in U-mode causes an illegal instruction exception. When TM is set to 1, the CPU can access the MTIMEHI and MTIMELO registers in U-mode normally.

IR: access control bit of the MINSTRET register in U-mode

When IR is set to 0, access to the MINSTRET register in U-mode causes an illegal instruction exception. When IR is set to 1, the CPU can access the MINSTRET register in U-mode normally.

16.2.6 M-mode Vector Interrupt Base Address Register (MTVT)

The MTVT register is applied to configuring the entry address of the vector interrupt service routine.

This register is 32-bit wide and is readable and writable in M-mode. Accesses in non-machine mode will cause an illegal instruction exception.

	31		6	5	0
	Base				0
Reset	0				0

Fig. 16.5: M-mode Vector Interrupt Base Address Register (MTVT)

BASE: vector base address bit

The vector base address bit indicates the base address of the vector interrupt table. The processor calculates the entry address for each ISR in the table, by adding the offset ($\text{MTVT} + 4 * \text{Interrupt ID}$) to this base address, and then jumps to execute.

The reset value of the base address field is 0.

16.3 M-Mode Exception Handling Register Group

16.3.1 M-Mode Scratch Register (MSCRATCH)

The MSCRATCH register is applied to backing up temporary data in the exception service routine for CPU. It is usually used to store the entry pointer value of the local context space in M-mode.

This register is 32-bit wide and is readable and writable in M-mode. Accesses in non-machine mode will cause an illegal instruction exception.

16.3.2 M-Mode Multi-Mode Data Backup Register (MSCRATCHCSW)

The MSCRATCHCSW register is applied to accelerating interrupt handling in multi-privilege modes. In general, when the CPU switches between different privilege modes and MSCRATCH is not set to M-mode

prior to an interrupt, the MSCRATCH register supports exchanging values with the stack pointer. Otherwise, the value remains unchanged. The instruction syntax is as follows:

```
csrrw rd, mscratchcsw, rs1.
```

If the privilege mode of the CPU before it enters an interrupt is not the M-mode (mcause.mpp!=M-mode), the CPU executes:

```
t = rs1; rd = mscratch; mscratch = t;
```

Otherwise, it executes:

```
rd = rs1.
```

A common usage pattern is:

```
csrw sp, mscratchcsw, sp.
```

16.3.3 M-Mode Interrupt Data Backup Register (MSCRATCHCSWL)

The MSCRATCHCSWL register is applied to accelerating scenarios where both the pre- and post-interrupt CPU states differ, enhancing interrupt handling efficiency. This register supports exchanging the values of the MSCRATCH register and the stack pointer. The instruction syntax is as follows:

```
csrrw rd, mscratchcswl, rs1.
```

When the CPU does not handle an interrupt before it enters an interrupt ((mcause.pil == 0) != (minstatus.mil == 0)), the CPU executes:

```
t = rs1; rd = mscratch; mscratch = t;
```

Otherwise, executes rd = rs1.

A common usage pattern is:

```
csrw sp, mscratchcswl, sp.
```

16.3.4 M-mode Interrupt Controller Base Address Register (MCLICBASE)

The MCLICBASE register is applied to indicating the CLIC memory-mapped register address to the software. In E902, the MCLICBASE register is fixed to 32'hE0800000 for hardware and unwritable for software.

This register is 32-bit wide and is read-only in M-mode. Accesses in non-machine mode will cause an illegal instruction exception.

16.3.5 M-Mode Exception Program Counter (MEPC)

The MEPC register stores the program counter (PC) value to which the program should return upon exiting the exception service routine. E902 supports 16-bit instructions, and the PC value is

halfword-aligned. Therefore, the least significant bit of the MEPC register is fixed to 0, while the remaining upper 31 bits constitute the writable portion.

This register is 32-bit wide and is readable and writable in M-mode. Accesses in non-machine mode will cause an illegal instruction exception.

16.3.6 M-Mode Exception Cause Register (MCAUSE)

The MCAUSE register stores the vector event ID that trigger exceptions, which is applied to handling corresponding events in the exception service routine.

This register is 32-bit wide and is readable and writable in M-mode. Accesses in non-machine mode will cause an illegal instruction exception.

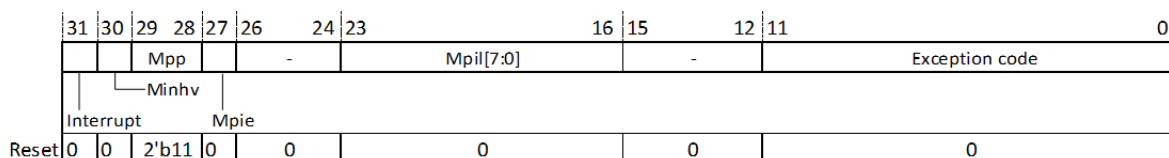


Fig. 16.6: M-Mode Exception Cause Register (MCAUSE)

Interrupt: interrupt bit

When the Interrupt bit is set to 0, it indicates that the source of the exception is not an interrupt, and the Exception Code should be interpreted according to the standard exception rules.

When the Interrupt bit is set to 1, it indicates that the source of the exception is an interrupt, and the Exception Code should be interpreted according to the interrupt rules.

The reset value of this bit is 0.

MINHV: vector interrupt jump indication bit

This bit is available only when the CPU is configured in CLIC.

It serves to indicate whether the CPU is currently fetching the entry address of a vector interrupt. When the CPU responds to a vector interrupt, this bit is set to high. It is then cleared back to 0 after successfully acquiring the entry point of the vector ISR.

The reset value of this bit is 0.

MPP: M-mode preserve privilege status bit

This bit is available only when the CPU is configured in CLIC.

This bit mirrors the value of MPP[1:0] in the MSTATUS register. That is, reads and writes to the MPP field in the MCAUSE register generate the same results as that in the MSTATUS register.

MPIE: M-mode preserve interrupt enable bit

This bit is available only when the CPU is configured in CLIC.

This bit mirrors the value of MSTATUS.MPIE.

MPIL: M-mode preserve interrupt priority bit

This bit is available only when the CPU is configured in CLIC.

This bit stores the interrupt priority before the CPU enters the ISR, effectively copying the MINTSTATUS.MIL bit to this bit. Upon executing the MRET instruction to return from an interrupt, the processor copies the MPIL bit back to the MIL bit within the MINTSTATUS register.

During exception handling, the value of this bit remains unchanged.

Exception Code: exception vector number bit

When the CPU encounters an exception, the Exception Code field is updated to the vector number of the exception source.

When the CPU is configured with the CLIC mode, this bit field is extended to 12 bits and can record up to 4,096 interrupt IDs.

When the CPU is not configured with the CLIC mode, this bit field is 4 bits. E902 is fixed to the CLIC mode.

The reset value of this bit is 0.

For specific information about the exception vector numbers, please refer to [Table 4.1](#).

16.3.7 M-Mode Pending Interrupt Vector Address and Interrupt Enable Register (MNXTI)

This register is available only when the CPU is configured with the CLIC mode.

Software applies this register to accelerate interrupt responding. In M-mode, when the priority of the current pending interrupt is higher than that of the reserved interrupt (MCAUSE.mpil), the software reads this register to obtain the entry address of the next interrupt.

The CPU can execute the CSRRSI or CSRRCI instruction to operate MNXTI register, obtain the vector table address of the next interrupt, and write the value to the MSTATUS register.

The CPU can set bit[3] in the MNXTI register to 1, to set the MIE bit in the MSTATUS register and enable the interrupt.

When a non-zero value is returned from reads to the MNXTI register, the CLIC standard defines the value as the entry address of a valid pending interrupt, that is, $MTVT[31:6] \ll 6 + 4 * \text{interrupt ID}$. At the same time, the CPU hardware updates the interrupt context as follows:

The MIL bit in the MINTSTATUS register is set to the priority of this valid pending interrupt.

The Exception Code field in the MCAUSE register is set to the ID of this valid pending interrupt.

When non-zero values are returned from reads to the MNXTI register, the conditions for obtaining valid interrupts are as follows:

1. The CPU is in M-mode.
2. The priority of the current pending interrupt is higher than that of the reserved interrupt (MCAUSE.mpil).
3. The interrupt does not belong to a hardware vector interrupt.

When zero is returned from reads to the MNXTI register, it indicates no valid pending interrupt to be handled, which includes one of the following scenarios:

1. There is no pending interrupts in CLIC.
2. The priority of the pending interrupt is greater than the value of the mpil bit in the MCAUSE register. But the interrupt belongs to hardware vector interrupt.

When zero is returned from reads to the MNXTI register, the MIL field in the MINTSTATUS register and the Exception Code field in the MCAUSE register are not updated.

16.3.8 M-Mode Interrupt Status Register (MINTSTATUS)

	31	24	23	0
	Mil[7:0]		-	
Reset	0		0	

Fig. 16.7: M-mode Interrupt Status Register (MINTSTATUS)

This register is available only when the CPU is configured with the CLIC mode. This register is read-only for software.

MIL: current interrupt priority bit

This field specifies the interrupt priority that is currently being handled by the CPU.

When the CPU responds to an interrupt, this field is updated to the priority of the responded interrupt. When the CPU executes the MRET instruction to return from the ISR, the CPU copies the value of the MPIL bit in the MCAUSE register to this bit.

When the CPU responds to an exception, this bit remains unchanged.

The reset value of this bit is 0.

16.3.9 M-Mode Exception Cause Register (MTVAL)

The MTVAL register is applied to storing the specific causes for triggering exception events, such as the incorrect access addresses for access error exceptions.

This register is 32-bit wide and is readable and writable in M-mode. Accesses in non-machine mode will cause an illegal instruction exception.

16.3.10 M-Mode Interrupt Pending Register (MIP)

The MIP register stores the pending states of interrupts for CPU. This register is meaningful only in non-CLIC modes. When an interrupt is pending, the corresponding bit in the MIP register will be set.

This register is 32-bit wide and is readable and writable in M-mode. Accesses in non-machine mode will cause an illegal instruction exception.

E902 implements the CLIC mode. Therefore, this register is set to 0, unwritable for software.

16.4 M-Mode PMP Register Group

M-mode physical memory protection (PMP) registers are related to the settings of the PMP unit. They are valid when the PMP unit is configured in the CPU.

M-mode PMP configuration register (PMPCFG)

The PMPCFG register configures the access permissions and address matching mode for physical memory.

This register is 32 bit-wide and readable and writable in M-mode. Accesses in non-machine mode will cause an illegal instruction exception.

For specific CSR definitions, please refer to *PMP CSR*.

M-mode PMP address register (PMPADDR)

The PMPADDR register configures the address range of each physical memory entry.

This register is 32 bit-wide and readable and writable in M-mode. Accesses in non-machine mode will cause an illegal instruction exception.

For specific CSR definitions, please refer to *PMP CSR*.

16.5 M-mode Exception Handling Register Group

M-mode counter register group is related to performance monitoring, and applied to collecting software and hardware information during program operation, for software development personnel to optimize programs.

16.5.1 M-mode Cycle Counter (MCYCLE)

The MCYCLE counter stores the cycles executed by the CPU. When the CPU is in the execution state (non-low power state), the MCYCLE register auto-increments the count by 1 upon each clock cycle.

The MCYCLE counter is 64-bit wide. In the RV32 architecture, it is implemented respectively by the MCYCLEH and MCYCLE registers. These two registers store the upper 32 bits and lower 32 bits of the MCYCLE counter.

The MCYCLEH and MCYCLE registers are 32-bit wide, readable and writable in M-mode. Accesses in non-machine mode will cause an illegal instruction exception.

The reset value of the MCYCLE counter is 0.

16.5.2 M-mode Retired Instruction Counter (MINSTRET)

The MINSTRET counter stores the number of retired instructions of the CPU, and auto-increments the count by 1 upon each retired instruction.

The MINSTRET counter is 64-bit wide. In RV32 architecture, it is implemented respectively by the MINSTRETH and MINSTRET registers. These two registers store the upper 32 bits and lower 32 bits of the MINSTRET counter.

The MINSTRETH and MINSTRET registers are 32-bit wide, readable and writable in M-mode. Accesses in non-machine mode will cause an illegal instruction exception.

The reset value of the MINSTRET counter is 0.

16.6 M-mode Extended Register Group

16.6.1 Extended Status Register (MXSTATUS)

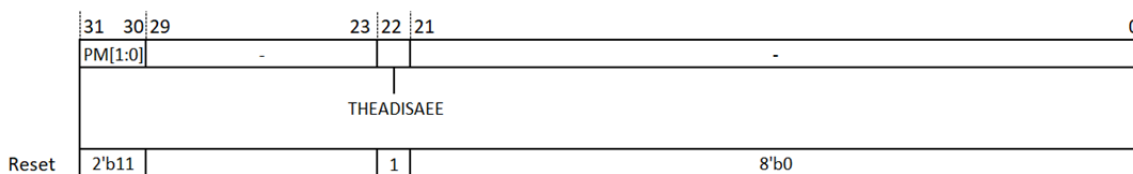


Fig. 16.8: MXSTATUS Register

THEADISAE: XuanTie extended instruction set enable bit

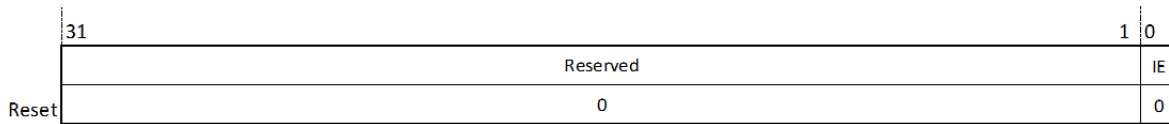
When THEADISAE is set to 0, executing all XuanTie custom extended instructions causes illegal instruction exceptions.

When THEADISAE is set to 1, the CPU can execute XuanTie custom extended instructions normally.

The reset value of this field is 0.

PM: current privilege mode bit

This field indicates the current privilege mode of the CPU. When this field is set to 2'b11, the CPU is in M-mode. When this field is set to 2'b00, the CPU is in U-mode.

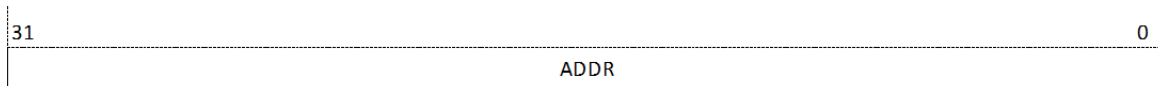
16.6.2 M-mode Hardware Configuration Register (MHCR)**Fig. 16.9: MHCR Register****IE: Instruction Cache (I-Cache) enable bit**

When IE is set to 0, the I-Cache is disabled;

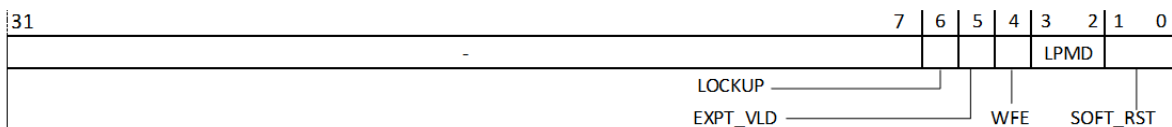
When IE is set to 1, the I-Cache is enabled.

16.6.3 Reset Start Address Register (MRADDR)

This register is only applied to identifying the size of the CPU reset start address specified by the SoC during hardware integration, and is read-only in M-mode with writes having no effect. Access to this register in U-mode will trigger an illegal instruction exception.

**Fig. 16.10: MRADDR Register****ADDR: reset start address**

The lower two bits of this field are fixed to zero.

16.6.4 Extended Exception Status Register (MEXSTATUS)**Fig. 16.11: MEXSTATUS Register****SOFT_RST: software reset indication bit**

2'b00: No reset request.

2'b01: Only the kernel is reset.

2'b10: The system is reset.

2'b11: Not implemented.

The reset value of this field is 2'b00. Writes to this field enable the CPU top-layer output signal `cpu_pad_rst[1:0]` to indicate corresponding values. SoC designers need to perform the related reset operations based on this value. Reset is not automatically performed in E902.

LPMD: low-power mode

2'b00: deep sleep

2'b01: light sleep

2'b10: not implemented. Writing is invalid.

2'b11: normal operation mode. Writing is invalid.

The reset value of this field is 2'b00. Setting this field instructs the SoC on the level of low-power mode the system should enter after the CPU executes the Wait For Interrupt (WFI) instruction. The CPU communicates the selected low-power mode through the top-level output signal `sysio_pad_lpmc_b[1:0]`. For the CPU, regardless of whether this field is set to 2'b00 or 2'b01, the low-power strategy adopted upon entering low-power mode via WFI remains consistent, entailing the shutdown of most registers' clocks that are unrelated to wake-up logic.

When the CPU is awakened, this field is 2'b11.

WFE: low-power wake-up mode indication bit

1'b0: When waking up the CPU via an interrupt, it is required that the interrupt's priority be higher than that indicated by `MINTSTATUS.MIL` in order to successfully awaken the CPU.

1'b1: When an interrupt request is applied for the wake-up of the CPU, there are no requirements on the interrupt priority. In addition, external events can also awaken the CPU.

The reset value of this field is 1'b1.

EXPT_VLD: exception status indication bit

1'b0: The CPU is not handling exceptions.

1'b1: The CPU is handling an exception.

The reset value of this field is 1'b0. This register is writable only in debug mode. While in non-debug mode, this register is read-only in M-mode.

LOCKUP: lockup indication bit

1'b0: The CPU is not locked.

1'b1: The CPU is locked.

The reset value of this field is 1'b0. This register is read-only in M-mode.

16.6.5 NMI Status Register (MNMICAUSE)

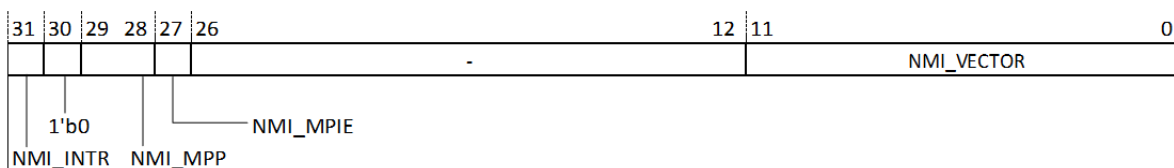


Fig. 16.12: NMI Status Register (MNMICAUSE)

NMI_VECTOR:

This bit stores the value of the Exception Code field in the MCAUSE register when the CPU responds to a Non-Maskable Interrupt (NMI).

NMI_MPIE:

This bit stores the value of the MPIE bit in the MSTATUS register when the CPU responds to a NMI.

NMI_MPP:

This bit stores the value of the MPP bit in the MSTATUS register when the CPU responds to a NMI.

NMI_INTR:

This bit saves the value of the INTR bit in the MCAUSE register when the CPU responds to a NMI.

16.6.6 NMI Exception Program Counter (MNMIPC)

This register is 32-bit wide and is readable and writable only in M-mode. Accesses in U-mode will cause an illegal instruction exception.

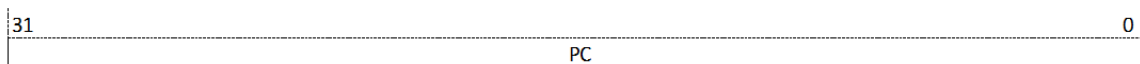


Fig. 16.13: NMI Exception Program Counter (MNMIPC)

PC: exception program counter

The least significant bit of this field is fixed to 0.

16.6.7 CPU Identifier Register

The MCPUID registers store CPU IDs. Its reset value is determined by the product.

16.7 Debug/Trace Register Group (Shared with Debug Mode)

16.7.1 Debug/Trace Trigger Selection Register (TSELECT)

The TSELECT register is designed to select one from multiple available triggers, to read/write the trigger.

	31		4	3	0
				SELECT	
Reset	0			0	

Fig. 16.14: TSELECT Register

SELECT: Debug/Trace Trigger Selection

Record the currently selected debug/trace trigger ID. For example, to configure Trigger Number 2, write 0x2 into the SELECT register.

16.7.2 Debug/Trace Trigger Data Register 1 (TDATA1)

	31	28	27	26	0
	TYPE		DEMODE	DATA	
	0x2 (mcontrol)		0	0	
Reset	0x5 (itrigger/etrigger/icount)				

Fig. 16.15: TDATA1 Register

TYPE: Debug/Trace Trigger Type Selection

- TYPE = 2, indicating the current trigger type is mcontrol;
- TYPE = 3, indicating the current trigger type is icount;
- TYPE = 4, indicating the current trigger type is itrigger;
- TYPE = 5, indicating the current trigger type is etrigger.

E902 supports 2 types of triggers:

1. mcontrol trigger, The TYPE field is hardwired to 0x2;

2. ittrigger/etrigger/icount configurable registers, and the TYPE field can be configured as 0x3, 0x4, or 0x5; its reset value is 0x5.

DEMODE: Control the write permissions for Debug/Trace Trigger Data Registers 1, 2, and 3 (TDATA1/TDATA2/TDATA3)

- When DEMODE is set to 0, Debug mode and M-mode both allow for writing to TDATA Registers;
- When DEMODE is set to 1, Only Debug mode allows for writing to TDATA Registers.

DATA: Control Debug/Trace Trigger Data Register 1

The specific meaning of DATA field is determined by TYPE. (For specific information, please refer to the section 5.2 in [RISC-V External Debug Support](#)).

16.7.3 Debug/Trace Trigger Data Register 2 (TDATA2)

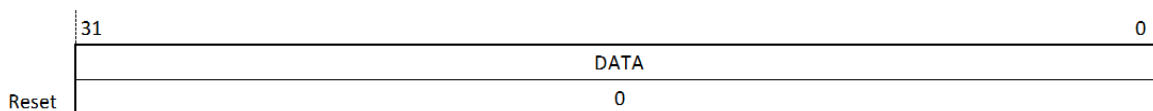


Fig. 16.16: Debug/Trace Trigger Data Register 2 (TDATA2)

DATA: Data of Debug/Trace Trigger Data Register 2

It is designed to set the trigger value, and the specific meaning is determined by the TYPE field of TDATA1.

16.7.4 Debug/Trace Trigger Data Register 3 (TDATA3)

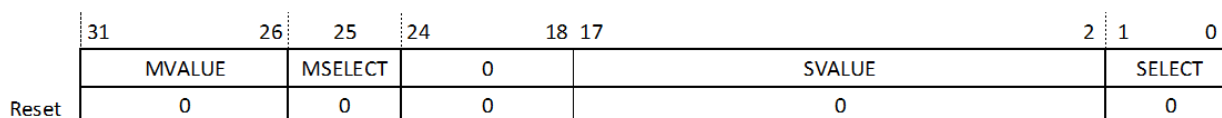


Fig. 16.17: Debug/Trace Trigger Data Register 3 (TDATA3)

MVALUE: Trigger M-mode Content Matching Data

MVALUE sets the desired value of content in M-mode to be matched.

MSELECT: Trigger S-Mode Content Matching

- When MSELECT is set to 0: disable trigger S-mode content matching.

- When MSELECT is set to 1: the trigger is enabled for CPU matching information as the M-mode Context Register (MCONTEXT) equals the MVALUE register.

SVALUE: Trigger S-Mode Content Matching Data

It is designed to set the desired value of content in S-mode to be matched.

SSELECT: Trigger S-mode Content Matching

- When SSELECT is set to 0: close trigger S-mode content matching.
- When SSELECT is set to 1: the trigger is capable of matching CPU information, as the S-mode Content Register(SCONTEXT) is equal to the Trigger S-mode Content Matching Data (SVALUE).
- When SSELECT is set to 2: the trigger is capable of matching CPU information, as the value of ASID field in satp register is equal to the trigger S-mode Content Matching data.

16.7.5 Debug/Trace Trigger Information Register (TINFO)

	31	16	15	0
	0		INFO	
Reset	0		0x100 (mcontrol) 0x111000 (itrigger/etrigger/icount)	

Fig. 16.18: TINFO Register

INFO: Types Supported by the Trigger

When bit[n] is set to 1, it indicates that the TYPE field of the trigger TDATA1 can be configured as n.

E902 supports 2 types of triggers:

1. mcontrol trigger, INFO is hardwired to 0x100, indicating the TYPE field in TDATA1 can only be set to 2;
2. itrigger/etrigger/icount configurable registers, and INFO is hardwired to 0x111000, indicating the TYPE field can be configured as 0x3, 0x4, 0x5

16.7.6 Debug/Trace Trigger CSR (TCONTROL)

	31	9	8	7	6	4	3	2	0
	0				MPTE	0	MTE	0	
Reset	0				0	0	0	0	

Fig. 16.19: TCONTROL Register

MPTE: M-mode Trigger Enable Backup

The hardware stores the value of MTE into MPTE, when entering a M-mode exception/interrupt handling routine.

MTE: M-mode Trigger Enable

- When MTE is set to 0: the trigger that generates a breakpoint exception can not be triggered in M-mode;
- When MTE is set to 1: the trigger can be triggered in M-mode.

The hardware sets MTE to 0 when entering a M-mode exception/interrupt handling routine;
The hardware resets MTE to the value of MPTE upon returning from the M-mode exception/interrupt handler handling routine.

16.7.7 M-mode Content Register (MCONTEXT)

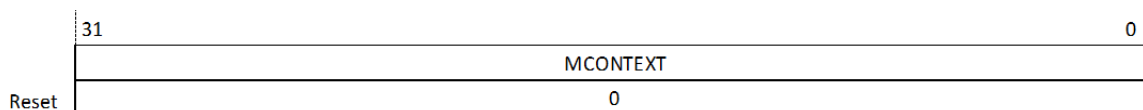


Fig. 16.20: MCONTEXT Register

MCONTEXT: M-mode Content

In M-mode, software can write to a specific context and control the trigger to be triggered only in specific M-mode contexts by combining the MSELECT with MVALUE in TDATA3.

16.8 Debug Mode Register Group

16.8.1 Debug CSR Register (DCSR)

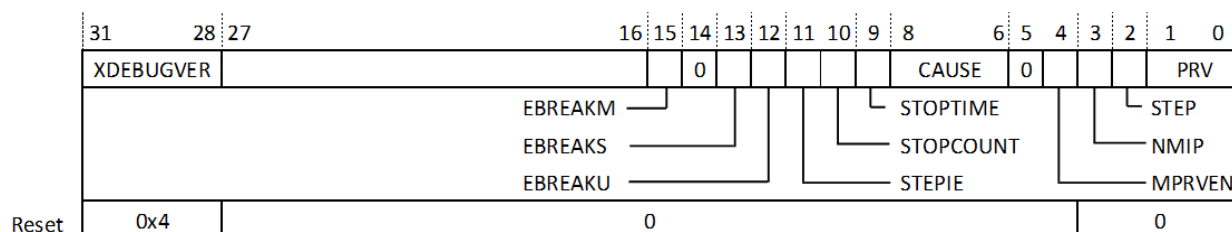


Fig. 16.21: DCSR Register

XDEBUGVER:

- 0: No debug system
- 4: the debug system, supporting riscv debug spec v0.13.2
- 15: the debug system, not supporting riscv debug spec v0.13.2

EBREAKM:

- 0: Execution of the ebreak instruction in M-mode generates a breakpoint exception.
- 1: Execution of the ebreak instruction in M-mode enters debug mode.

EBREAKS:

- 0: Execution of the ebreak instruction in S-mode generates a breakpoint exception.
- 1: Execution of the ebreak instruction in S-mode enters debug mode.

EBREAKU:

- 0: Execution of the ebreak instruction in u-mode generates a breakpoint exception.
- 1: Execution of the ebreak instruction in U-mode enters debug mode.

STEPIE:

- 0: does not respond to interrupts during single-step debugging
- 1: responds to interrupts during single-step debugging

STOPCOUNT:

- 0: Performance monitoring counters increment normally in debug mode.
- 1: In debug mode, performance monitoring counters, including MCYCLE and MINSTRET, do not increment.

STOPTIME:

- 0: In debug mode, CPU private clock counter counts normally.
- 1: In debug mode, CPU private clock counter does not count.

CAUSE:

- 1: Indicates that entry into debug mode is caused by the execution of an ebreak instruction.
- 2: Indicates that entry into debug mode is triggered by a trigger.
- 3: Indicates that entry into debug mode is due to a synchronous debug request.
- 4: Indicates that entry into debug mode is initiated by a single-step debug request.
- 5: Indicates that entry into debug mode is the result of a reset debug request.

MPRVN:

- 0: The MPRV field in the MSTATUS register is ineffective during debug mode.
- 1: The MPRV field in the MSTATUS register is effective during debug mode, enabling CPU to handle memory access instruction address translation and protection based on the settings of MPRV and MPP.

NMIP:

- 0: No NMI is in CPU.
- 1: NMI occurs in CPU.

STEP:

- 0: No single-step debugging.
- 1: Initiates single-step debugging mode.

PRV:

Upon entering debug mode, the CPU privilege mode is stored in PRV; upon exiting debug mode, the CPU privilege mode is set according to the value in the PRV field.

16.8.2 Debug Program Counter (DPC)

DPC[31:0]:

Upon entering debug mode, the hardware writes the address of the next instruction into the DPC; when exiting debug mode, CPU fetches instructions from the address stored within the DPC.

16.8.3 Debug Mode Temporary Data Backup Register 0 (DSCRATCH0)

DSCRATCH0[31:0]:

Reserved for data exchange between the debug system and CPU by the hardware.

16.8.4 Debug Mode Temporary Data Backup Register 1 (DSCRATCH1)

DSCRATCH1[31:0]:

Reserved for data exchange between the debug system and CPU by the hardware.

16.9 Dubug Extended Register Group

16.9.1 Xuantie Debug Cause Register (MHALTCAUSE)

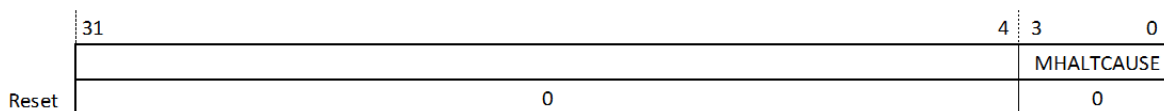


Fig. 16.22: MHALTCAUSE Register

MHALTCAUSE: indicates the cause of entering the debug mode

- When MHALTCAUSE is set to 1: Indicates that entry into debug mode is caused by the execution of an ebreak instruction.
- When MHALTCAUSE is set to 2: Indicates that entry into debug mode is triggered by a trigger.
- When MHALTCAUSE is set to 3: Indicates that entry into debug mode is due to a synchronous debug request.
- When MHALTCAUSE is set to 4: Indicates that entry into debug mode is initiated by a single-step debug request.
- When MHALTCAUSE is set to 5: Indicates that entry into debug mode is the result of a reset debug request.
- When MHALTCAUSE is set to 8: Indicates that entry into debug mode is the result of a asynchronous debug request.

16.9.2 Xunatie Debug Information Register (MDBGINFO)

MDBGINFO[31:0]:

It is applied to record debug information of CPU during the asynchronous debugging.

16.9.3 XuanTie Branch Target Address Record Register (MPCFIFO)

MPCFIFO[31:0]:

It records the target addresses of branch/jump instructions.